

J'ai une base de données postgres avec les tables suivantes :

formation :

id_formation (clé primaire sous forme de "formation" + nombre)

nom (str du nom de la formation)

jour_semaine (indique le jour de la semaine entre 0 et 6, 0 étant lundi et 6 étant dimanche)

heure_debut (l'heure du début de la formation)

heure_fin (heure de la fin de la formation)

chemins_image (liste de textes indiquant le chemin vers les images de l'événement)

description (description textuelle de l'événement)

cela représente l'une de mes tables

Merci pour ces informations sur votre table `formation` . Je vois que vous avez une structure bien définie pour gérer des formations avec leurs horaires, descriptions et images.

Voici un résumé de la structure de votre table :

Table : formation

Colonne	Type	Description
id_formation	TEXT (ou VARCHAR)	Clé primaire au format "formation" + nombre (ex: "formation1", "formation25")
nom	TEXT (ou VARCHAR)	Nom de la formation
jour_semaine	INTEGER	Jour de la semaine (0=lundi à 6=dimanche)
heure_debut	TIME	Heure de début de la formation
heure_fin	TIME	Heure de fin de la formation

Colonne	Type	Description
chemins_image	TEXT[] (array)	Liste des chemins vers les images de la formation
description	TEXT	Description textuelle de la formation

Exemple de données :

sql

```
INSERT INTO formation VALUES
('formation1', 'Formation Python', 1, '09:00', '17:00',
 '{"images/python1.jpg", "images/python2.jpg"}',
 'Formation complète sur la programmation Python');
```

Requêtes utiles :

1. Récupérer toutes les formations du mercredi (jour 2) :

sql

```
SELECT * FROM formation WHERE jour_semaine = 2;
```

2. Formations avec conversion du jour numérique en texte :

sql

```
SELECT id_formation, nom,
       CASE jour_semaine
         WHEN 0 THEN 'Lundi'
         WHEN 1 THEN 'Mardi'
         WHEN 2 THEN 'Mercredi'
         WHEN 3 THEN 'Jeudi'
         WHEN 4 THEN 'Vendredi'
         WHEN 5 THEN 'Samedi'
         WHEN 6 THEN 'Dimanche'
       END as jour,
       heure_debut, heure_fin
FROM formation;
```

3. Formations avec leurs images :

```
sql
```

```
SELECT id_formation, nom, UNNEST(chemins_image) as image_path
FROM formation;
```

Souhaitez-vous que je vous aide avec d'autres aspects de votre base de données, comme :

- Créer des requêtes spécifiques
- Optimiser la structure
- Ajouter des relations avec d'autres tables
- Implémenter des contraintes ou index ?

J'ai aussi une page d'accueil qui ressemble à ça à REACT js et node JS.

```
// AccueilFormations.jsx
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"]
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
```

```
  }) => {
    const [showAdminLogin, setShowAdminLogin] = useState(false);
    const [adminCredentials, setAdminCredentials] = useState({
      username: '',
      password: ''
    });

    const handleFormationClick = (formationId) => {
      if (onFormationSelect) {
        onFormationSelect(formationId);
      }
    };

    const handleAdminLogin = (e) => {
      e.preventDefault();
      // Simulation de connexion admin
      console.log('Tentative de connexion admin:', adminCredentials);
      if (adminCredentials.username && adminCredentials.password) {
        alert('Connexion administrateur réussie !');
        setShowAdminLogin(false);
        setAdminCredentials({ username: '', password: '' });
      } else {
        alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
      }
    };

    const handleAdminInputChange = (e) => {
      const { name, value } = e.target;
      setAdminCredentials(prev => ({
        ...prev,
        [name]: value
      }));
    };

    return (
      <div className="accueil-container">
        </* Header avec bouton admin */>
        <header className="accueil-header">
          <div className="header-content">
            <h1 className="accueil-title">Formations en Ligne</h1>
          </div>
        </header>
      </div>
    );
  }
}
```

```

<button
  className="admin-button"
  onClick={() => setShowAdminLogin(true)}
>
  🛡️ Administrateur
</button>
</div>
</header>

{/* Contenu principal */}
<main className="accueil-main">
  <div className="formations-grid">
    {Object.entries(formations).map(([formationId, formation]) =>
      (
        <div
          key={formationId}
          className="formation-card"
          onClick={() => handleFormationClick(formationId)}
        >
          <div className="formation-image-container">
            <img
              src={formation.images[0]}
              alt={formation.titre}
              className="formation-image"
              onError={(e) => {
                e.target.src =
`https://via.placeholder.com/300x200/667eea/ffffff?
text=${encodeURIComponent(formation.titre)}`;
              }}
            />
          </div>
          <div className="formation-info">
            <h3 className="formation-titre">{formation.titre}</h3>
            <div className="formation-overlay">
              <span className="voir-details">Voir les détails
→</span>
            </div>
          </div>
        </div>
      )
    )
  )
}

```

```
</div>
```

```
{Object.keys(formations).length === 0 && (  
  <div className="no-formations">  
    <p>Aucune formation disponible pour le moment.</p>  
  </div>  
  )}  
</main>
```

```
{/* Modal de connexion admin */}  
{showAdminLogin && (  
  <div className="admin-modal-overlay">  
    <div className="admin-modal">  
      <div className="admin-modal-header">  
        <h2>Connexion Administrateur</h2>  
        <button  
          className="close-button"  
          onClick={() => setShowAdminLogin(false)}  
        >  
          ×  
        </button>  
      </div>  
  
      <form className="admin-form" onSubmit=  
        {handleAdminLogin}>  
        <div className="form-group">  
          <label htmlFor="username">Nom d'utilisateur</label>  
          <input  
            type="text"  
            id="username"  
            name="username"  
            value={adminCredentials.username}  
            onChange={handleAdminInputChange}  
            placeholder="Entrez votre nom d'utilisateur"  
            required  
          />  
        </div>  
  
        <div className="form-group">  
          <label htmlFor="password">Mot de passe</label>
```

```

    <input
      type="password"
      id="password"
      name="password"
      value={adminCredentials.password}
      onChange={handleAdminInputChange}
      placeholder="Entrez votre mot de passe"
      required
    />
  </div>

  <div className="admin-form-buttons">
    <button type="submit" className="login-button">
      Se connecter
    </button>
    <button
      type="button"
      className="cancel-button"
      onClick={() => setShowAdminLogin(false)}
    >
      Annuler
    </button>
  </div>
</form>
</div>
</div>
  )}
</div>
);
};

export default AccueilFormations;

```

Excellent ! Voici comment vous pouvez connecter votre interface React avec votre base de données PostgreSQL. Je vais vous montrer comment créer l'API Node.js et adapter le composant React.

1. API Node.js (Express)

javascript

```
// server.js
const express = require('express');
const cors = require('cors');
const { Pool } = require('pg');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

// Configuration de la base de données
const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});

// Route pour récupérer toutes les formations
app.get('/api/formations', async (req, res) => {
  try {
    const query = `
      SELECT
        id_formation,
        nom as titre,
        chemins_image as images,
        description,
        jour_semaine,
        heure_debut,
        heure_fin
      FROM formation
      ORDER BY jour_semaine, heure_debut
    `;

    const result = await pool.query(query);

    // Transformer les données pour correspondre au format React
    const formations = {};
    result.rows.forEach(formation => {
      formations[formation.id_formation] = {
        titre: formation.titre,
```



```

        images: formation.images || [],
        description: formation.description,
        jour_semaine: formation.jour_semaine,
        heure_debut: formation.heure_debut,
        heure_fin: formation.heure_fin
    };
});

res.json(formations);
} catch (error) {
    console.error('Erreur lors de la récupération des formations:', error);
    res.status(500).json({ error: 'Erreur serveur' });
}
});

// Route pour récupérer une formation spécifique
app.get('/api/formations/:id', async (req, res) => {
    try {
        const { id } = req.params;
        const query = `
            SELECT * FROM formation WHERE id_formation = $1
        `;

        const result = await pool.query(query, [id]);

        if (result.rows.length === 0) {
            return res.status(404).json({ error: 'Formation non trouvée' });
        }

        res.json(result.rows[0]);
    } catch (error) {
        console.error('Erreur:', error);
        res.status(500).json({ error: 'Erreur serveur' });
    }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
    console.log(`Serveur démarré sur le port ${PORT}`);
});

```

2. Composant React mis à jour

jsx

```
// AccueilFormations.jsx
import React, { useState, useEffect } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
  const [formations, setFormations] = useState({});
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: '',
    password: ''
  });

  // Récupérer les formations depuis l'API
  useEffect(() => {
    const fetchFormations = async () => {
      try {
        setLoading(true);
        const response = await fetch('http://localhost:5000/api/formations');

        if (!response.ok) {
          throw new Error('Erreur lors du chargement des formations');
        }

        const data = await response.json();
        setFormations(data);
      } catch (err) {
        setError(err.message);
        console.error('Erreur:', err);
      } finally {
        setLoading(false);
      }
    };

    fetchFormations();
  }, []);
```

```
const handleFormationClick = (formationId) => {
  if (onFormationSelect) {
    onFormationSelect(formationId);
  }
};

const handleAdminLogin = async (e) => {
  e.preventDefault();
  try {
    // Ici vous pouvez appeler votre API d'authentification
    console.log('Tentative de connexion admin:', adminCredentials);

    // Simulation de connexion
    if (adminCredentials.username && adminCredentials.password) {
      alert('Connexion administrateur réussie !');
      setShowAdminLogin(false);
      setAdminCredentials({ username: '', password: '' });

      // Rediriger vers l'interface admin ou mettre à jour l'état
      // window.location.href = '/admin';
    } else {
      alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
    }
  } catch (err) {
    alert('Erreur de connexion');
  }
};

const handleAdminInputChange = (e) => {
  const { name, value } = e.target;
  setAdminCredentials(prev => ({
    ...prev,
    [name]: value
  }));
};

// Fonction pour formater l'heure
const formatHeure = (heure) => {
  if (!heure) return '';
  return heure.substring(0, 5); // Retourne HH:MM
};

// Fonction pour obtenir le nom du jour
const getJourSemaine = (jourNumero) => {
```

```

    const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche'];
    return jours[jourNumero] || '';
  };

  if (loading) {
    return (
      <div className="accueil-container">
        <div className="loading">Chargement des formations...</div>
      </div>
    );
  }

  if (error) {
    return (
      <div className="accueil-container">
        <div className="error">Erreur: {error}</div>
      </div>
    );
  }

  return (
    <div className="accueil-container">
      {/* Header avec bouton admin */}
      <header className="accueil-header">
        <div className="header-content">
          <h1 className="accueil-title">Formations en Ligne</h1>
          <button
            className="admin-button"
            onClick={() => setShowAdminLogin(true)}
          >
             Administrateur
          </button>
        </div>
      </header>

      {/* Contenu principal */}
      <main className="accueil-main">
        <div className="formations-grid">
          {Object.entries(formations).map(([formationId, formation]) => (
            <div
              key={formationId}
              className="formation-card"
              onClick={() => handleFormationClick(formationId)}
            >

```

```

>
<div className="formation-image-container">
  <img
    src={formation.images && formation.images[0]
      ? formation.images[0]
      : `https://via.placeholder.com/300x200/667eea/ffffff?text=${encodeURIComponent(formation.titre)}`}
    alt={formation.titre}
    className="formation-image"
    onError={(e) => {
      e.target.src = `https://via.placeholder.com/300x200/667eea/ffffff?text=${encodeURIComponent(formation.titre)}`;
    }}
  />
</div>
<div className="formation-info">
  <h3 className="formation-titre">{formation.titre}</h3>

  /* Informations supplémentaires depuis la BDD */
  <div className="formation-details">
    <span className="formation-jour">
      {getJourSemaine(formation.jour_semaine)}
    </span>
    <span className="formation-horaire">
      {formatHeure(formation.heure_debut)} - {formatHeure(formation.heu
re_fin)}
    </span>
  </div>

  <div className="formation-overlay">
    <span className="voir-details">Voir les détails →</span>
  </div>
</div>
</div>
  )}}
</div>

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

```

```
{/* Modal de connexion admin (inchangé) */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">
        <h2>Connexion Administrateur</h2>
        <button
          className="close-button"
          onClick={() => setShowAdminLogin(false)}
        >
          x
        </button>
      </div>

      <form className="admin-form" onSubmit={handleAdminLogin}>
        <div className="form-group">
          <label htmlFor="username">Nom d'utilisateur</label>
          <input
            type="text"
            id="username"
            name="username"
            value={adminCredentials.username}
            onChange={handleAdminInputChange}
            placeholder="Entrez votre nom d'utilisateur"
            required
          />
        </div>

        <div className="form-group">
          <label htmlFor="password">Mot de passe</label>
          <input
            type="password"
            id="password"
            name="password"
            value={adminCredentials.password}
            onChange={handleAdminInputChange}
            placeholder="Entrez votre mot de passe"
            required
          />
        </div>

        <div className="admin-form-buttons">
          <button type="submit" className="login-button">
```

```

        Se connecter
      </button>
      <button
        type="button"
        className="cancel-button"
        onClick={() => setShowAdminLogin(false)}
      >
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
)}
</div>
);
};

export default AccueilFormations;

```

3. Fichier .env pour la configuration

env

```

DB_USER=votre_utilisateur
DB_HOST=localhost
DB_NAME=votre_base_de_donnees
DB_PASSWORD=votre_mot_de_passe
DB_PORT=5432
PORT=5000

```

4. Package.json pour les dépendances

json

```

{
  "name": "formations-app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",

```

```

    "pg": "^8.11.3",
    "dotenv": "^16.3.1"
  },
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}

```

Points clés :

1. **API REST** : Votre serveur Node.js expose des endpoints pour récupérer les données
2. **Hook useEffect** : Récupère les données au chargement du composant
3. **Gestion d'état** : États pour le loading, les erreurs et les données
4. **Transformation des données** : Adaptation du format PostgreSQL au format attendu par React
5. **Gestion des images** : Fallback sur des images placeholder si besoin

Votre application est maintenant complètement connectée à la base de données !

J'ai un App.jsx qui ressemble à cela :

```

import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

```

```

// App.jsx
import React from 'react';
import PageAdministrateur from './components/PageAdministrateur';
import './App.css';

```

```

function App() {
  // Données de test pour l'admin
  const formationsAdmin = [
    {

```



```
id: "formation-react",
nom: "Formation React Avancé",
date: "2024-02-15",
heureDebut: "14:00",
inscrits: 28
},
{
id: "formation-js",
nom: "Formation JavaScript Moderne",
date: "2024-02-20",
heureDebut: "10:00",
inscrits: 15
},
{
id: "formation-node",
nom: "Formation Node.js & Express",
date: "2024-02-10",
heureDebut: "09:00",
inscrits: 30
},
{
id: "formation-angular",
nom: "Formation Angular",
date: "2024-02-25",
heureDebut: "16:00",
inscrits: 12
},
{
id: "formation-vue",
nom: "Formation Vue.js",
date: "2024-02-18",
heureDebut: "13:00",
inscrits: 25
},
{
id: "formation-python",
nom: "Formation Python Django",
date: "2024-02-22",
heureDebut: "11:00",
inscrits: 30
```

```

    }
  ];

  const handleRetourAccueil = () => {
    console.log('Retour à l\'accueil');
    // Pour l'instant, on reste sur la page admin
    alert('Fonctionnalité de retour à implémenter');
  };

  return (
    <PageAdministrateur
      formations={formationsAdmin}
      onRetourAccueil={handleRetourAccueil}
    />
  );
}

export default App;

```

Parfait ! Voici comment restructurer votre `App.jsx` pour gérer la navigation entre la page d'accueil des formations et la page administrateur, en connectant tout avec votre base de données :

App.jsx mis à jour

jsx

```

// App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import PageAdministrateur from './components/PageAdministrateur';
import './App.css';

function App() {
  const [currentPage, setCurrentPage] = useState('accueil'); // 'accueil' ou 'admin'
  const [selectedFormation, setSelectedFormation] = useState(null);
  const [formations, setFormations] = useState({});
  const [loading, setLoading] = useState(true);

  // Récupérer les formations depuis l'API
  useEffect(() => {

```

```
const fetchFormations = async () => {
  try {
    setLoading(true);
    const response = await fetch('http://localhost:5000/api/formations');

    if (!response.ok) {
      throw new Error('Erreur lors du chargement des formations');
    }

    const data = await response.json();
    setFormations(data);
  } catch (error) {
    console.error('Erreur:', error);
    // En cas d'erreur, utiliser des données de test
    setFormations(getFormationsTest());
  } finally {
    setLoading(false);
  }
};

fetchFormations();
}, []);

// Données de test en cas d'erreur API
const getFormationsTest = () => {
  return {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"],
      description: "Formation complète sur React avancé",
      jour_semaine: 1,
      heure_debut: "14:00",
      heure_fin: "17:00"
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"],
      description: "Formation sur les nouvelles fonctionnalités JavaScript",
      jour_semaine: 3,
      heure_debut: "10:00",
      heure_fin: "13:00"
    }
  };
};
```

```
// Gestionnaire pour la sélection d'une formation
const handleFormationSelect = (formationId) => {
  console.log('Formation sélectionnée:', formationId);
  setSelectedFormation(formationId);
  // Ici vous pouvez naviguer vers une page de détails si nécessaire
  // setCurrentPage('details');
};

// Gestionnaire pour la connexion admin
const handleAdminLogin = () => {
  setCurrentPage('admin');
};

// Gestionnaire pour retour à l'accueil
const handleRetourAccueil = () => {
  setCurrentPage('accueil');
  setSelectedFormation(null);
};

// Fonction pour rafraîchir les données depuis la BDD
const refreshFormations = async () => {
  try {
    setLoading(true);
    const response = await fetch('http://localhost:5000/api/formations');
    if (response.ok) {
      const data = await response.json();
      setFormations(data);
    }
  } catch (error) {
    console.error('Erreur lors du rafraîchissement:', error);
  } finally {
    setLoading(false);
  }
};

// Données formatées pour l'admin (avec statistiques simulées)
const getFormationsAdmin = () => {
  return Object.entries(formations).map(([id, formation]) => ({
    id: id,
    nom: formation.titre,
    date: getProchaineDate(formation.jour_semaine),
    heureDebut: formation.heure_debut?.substring(0, 5) || '09:00',
    inscrits: Math.floor(Math.random() * 30) + 10 // Simulation
  }));
};
```

```

    }));
  };

  // Fonction utilitaire pour calculer la prochaine date basée sur le jour de la semaine
  const getProchaineDate = (jourSemaine) => {
    const today = new Date();
    const daysUntilNext = (jourSemaine - today.getDay() + 7) % 7;
    const nextDate = new Date(today);
    nextDate.setDate(today.getDate() + daysUntilNext);
    return nextDate.toISOString().split('T')[0];
  };

  // Affichage conditionnel selon la page courante
  const renderCurrentPage = () => {
    switch (currentPage) {
      case 'admin':
        return (
          <PageAdministrateur
            formations={getFormationsAdmin()}
            onRetourAccueil={handleRetourAccueil}
            onRefresh={refreshFormations}
          />
        );
      case 'accueil':
      default:
        return (
          <AccueilFormations
            formations={formations}
            onFormationSelect={handleFormationSelect}
            onAdminLogin={handleAdminLogin}
            loading={loading}
          />
        );
    }
  };

  return (
    <div className="App">
      {renderCurrentPage()}
    </div>
  );
}

```

```
export default App;
```

AccueilFormations.jsx mis à jour

jsx

```
// AccueilFormations.jsx
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {},
  onFormationSelect = () => {},
  onAdminLogin = () => {},
  loading = false
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: '',
    password: ''
  });

  const handleFormationClick = (formationId) => {
    onFormationSelect(formationId);
  };

  const handleAdminLogin = (e) => {
    e.preventDefault();
    // Simulation de vérification des credentials
    if (adminCredentials.username && adminCredentials.password) {
      // Ici, vous devriez appeler votre API d'authentification
      console.log('Connexion admin réussie');
      onAdminLogin(); // Notifie Le parent de passer en mode admin
      setShowAdminLogin(false);
      setAdminCredentials({ username: '', password: '' });
    } else {
      alert('Veuillez entrer un nom d\'utilisateur et un mot de passe');
    }
  };

  const handleAdminInputChange = (e) => {
    const { name, value } = e.target;
```

```

    setAdminCredentials(prev => ({
      ...prev,
      [name]: value
    }));
  });

  // Fonction pour obtenir le nom du jour
  const getJourSemaine = (jourNumero) => {
    const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche'];
    return jours[jourNumero] || '';
  };

  if (loading) {
    return (
      <div className="accueil-container">
        <div className="loading">Chargement des formations...</div>
      </div>
    );
  }

  return (
    <div className="accueil-container">
      { /* Header avec bouton admin */ }
      <header className="accueil-header">
        <div className="header-content">
          <h1 className="accueil-title">Formations en Ligne</h1>
          <button
            className="admin-button"
            onClick={() => setShowAdminLogin(true)}
          >
             Administrateur
          </button>
        </div>
      </header>

      { /* Contenu principal */ }
      <main className="accueil-main">
        <div className="formations-grid">
          {Object.entries(formations).map(([formationId, formation]) => (
            <div
              key={formationId}
              className="formation-card"
              onClick={() => handleFormationClick(formationId)}
            >

```

```

>
<div className="formation-image-container">
  <img
    src={formation.images && formation.images[0]
      ? formation.images[0]
      : `https://via.placeholder.com/300x200/667eea/ffffff?text=${encodeURIComponent(formation.titre)}`}
    alt={formation.titre}
    className="formation-image"
  />
</div>
<div className="formation-info">
  <h3 className="formation-titre">{formation.titre}</h3>

  {/* Informations supplémentaires depuis la BDD */}
  <div className="formation-details">
    <span className="formation-jour">
      {getJourSemaine(formation.jour_semaine)}
    </span>
    <span className="formation-horaire">
      {formation.heure_debut?.substring(0, 5)} - {formation.heure_fin?.
substring(0, 5)}
    </span>
  </div>

  <div className="formation-overlay">
    <span className="voir-details">Voir les détails →</span>
  </div>
</div>
  )}
</div>

{Object.keys(formations).length === 0 && !loading && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

{/* Modal de connexion admin */}
{showAdminLogin && (
  <div className="admin-modal-overlay">

```



```
<div className="admin-modal">
  <div className="admin-modal-header">
    <h2>Connexion Administrateur</h2>
    <button
      className="close-button"
      onClick={() => setShowAdminLogin(false)}
    >
      x
    </button>
  </div>

  <form className="admin-form" onSubmit={handleAdminLogin}>
    <div className="form-group">
      <label htmlFor="username">Nom d'utilisateur</label>
      <input
        type="text"
        id="username"
        name="username"
        value={adminCredentials.username}
        onChange={handleAdminInputChange}
        placeholder="Entrez votre nom d'utilisateur"
        required
      />
    </div>

    <div className="form-group">
      <label htmlFor="password">Mot de passe</label>
      <input
        type="password"
        id="password"
        name="password"
        value={adminCredentials.password}
        onChange={handleAdminInputChange}
        placeholder="Entrez votre mot de passe"
        required
      />
    </div>

    <div className="admin-form-buttons">
      <button type="submit" className="login-button">
        Se connecter
      </button>
      <button
        type="button"

```

```

        className="cancel-button"
        onClick={() => setShowAdminLogin(false)}
      >
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
);
};

export default AccueilFormations;

```

PageAdministrateur.jsx mis à jour

jsx

```

// PageAdministrateur.jsx
import React from 'react';
import './PageAdministrateur.css';

const PageAdministrateur = ({
  formations = [],
  onRetourAccueil = () => {},
  onRefresh = () => {}
}) => {
  return (
    <div className="admin-container">
      <header className="admin-header">
        <div className="admin-header-content">
          <h1>Espace Administrateur</h1>
          <div className="admin-actions">
            <button className="refresh-button" onClick={onRefresh}>
              🔄 Actualiser
            </button>
            <button className="logout-button" onClick={onRetourAccueil}>
              ← Retour à l'accueil
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

</header>

<main className="admin-main">
  <div className="stats-overview">
    <div className="stat-card">
      <h3>Total Formations</h3>
      <p className="stat-number">{formations.length}</p>
    </div>
    <div className="stat-card">
      <h3>Total Inscrits</h3>
      <p className="stat-number">
        {formations.reduce((total, formation) => total + formation.inscrits,
0)}
      </p>
    </div>
    <div className="stat-card">
      <h3>Moyenne par formation</h3>
      <p className="stat-number">
        {formations.length > 0
          ? Math.round(formations.reduce((total, formation) => total + formatio
n.inscrits, 0) / formations.length)
          : 0
        }
      </p>
    </div>
  </div>

  <div className="formations-list">
    <h2>Liste des Formations</h2>
    <div className="formations-table">
      {formations.map(formation => (
        <div key={formation.id} className="formation-row">
          <div className="formation-info">
            <h4>{formation.nom}</h4>
            <p>ID: {formation.id}</p>
          </div>
          <div className="formation-date">
            <p>Date: {formation.date}</p>
            <p>Heure: {formation.heureDebut}</p>
          </div>
          <div className="formation-stats">
            <p className="inscrits-count">{formation.inscrits} inscrits</p>
          </div>
          <div className="formation-actions">

```

```

        <button className="edit-btn">Modifier</button>
        <button className="delete-btn">Supprimer</button>
      </div>
    </div>
  )})
</div>
</div>
</main>
</div>
);
};

export default PageAdministrateur;

```

Fonctionnalités implémentées :

1. **Navigation complète** entre accueil et admin
2. **Récupération des données** depuis votre API PostgreSQL
3. **Gestion des états** de chargement et d'erreur
4. **Rafraîchissement des données** depuis la BDD
5. **Système d'authentification** basique pour l'admin
6. **Affichage dynamique** des informations depuis la base

Votre application est maintenant complètement fonctionnelle et connectée à votre base de données !

Lorsque j'exporte ma table formation, cela me donne le résultat suivant lorsqu'on le met en Json :

```

[
  {
    "id_formation": "formation1",
    "nom": "Formation Blender 3D",
    "jour_semaine": 1,
    "heure_debut": "14:00:00",
    "heure_fin": "17:00:00",
    "chemins_images": [
      "Blender/blender1.png",
      "Blender/blender2.jpg",

```

```
"Blender/blender3.jpg",
"Blender/blender4.jpg"
],
"description": "Découvrez la magie de la création 3D ! Blender est
un logiciel gratuit et puissant pour modéliser, animer et rendre vos
idées en trois dimensions. Apprenez à créer des objets,
personnages et environnements virtuels, à maîtriser l'éclairage et les
textures, et à produire des animations époustouflantes. Parfait pour
les artistes, designers et passionnés d'animation."
},
{
  "id_formation": "formation2",
  "nom": "Formation Krita - Dessin Digital",
  "jour_semaine": 2,
  "heure_debut": "18:00:00",
  "heure_fin": "21:00:00",
  "chemins_images": [
    "Krita/Krita1.jpg",
    "Krita/krita2.png",
    "Krita/krita3.webp",
    "Krita/krita4.png"
  ],
  "description": "Libérez votre créativité numérique ! Krita est un
studio de peinture numérique open-source conçu pour les artistes.
Apprenez les techniques de dessin digital, la manipulation des
calques, les pinceaux personnalisés et la colorisation
professionnelle. Idéal pour l'illustration, le concept art et la bande
dessinée."
},
{
  "id_formation": "formation3",
  "nom": "Formation Kdenlive - Montage Vidéo",
  "jour_semaine": 3,
  "heure_debut": "19:00:00",
  "heure_fin": "22:00:00",
  "chemins_images": [
    "Kdenlive/kdenlive.png",
    "Kdenlive/kdenlive1.webp",
    "Kdenlive/kdenlive2.png",
    "Kdenlive/kdenlive3.png"
```

```
],
```

```
  "description": "Devenez monteur vidéo ! Kdenlive est un logiciel de montage vidéo gratuit et complet. Maîtrisez le découpage, les transitions, les effets visuels, le traitement audio et l'export de vidéos de qualité professionnelle. Parfait pour les YouTubeurs, vidéastes amateurs et créateurs de contenu."
```

```
},
```

```
{
```

```
  "id_formation": "formation5",
```

```
  "nom": "Formation Ollama - Intelligence Artificielle",
```

```
  "jour_semaine": 0,
```

```
  "heure_debut": "09:00:00",
```

```
  "heure_fin": "12:00:00",
```

```
  "chemins_images": [
```

```
    "Ollama/Ollama1.png",
```

```
    "Ollama/Ollama2.png",
```

```
    "Ollama/Ollama3.png",
```

```
    "Ollama/Ollama4.png"
```

```
  ],
```

```
  "description": "Explorez l'intelligence artificielle locale ! Ollama vous permet d'exécuter des modèles de langage avancés directement sur votre ordinateur. Découvrez comment interagir avec l'IA, personnaliser les modèles et développer des applications intelligentes sans dépendre d'internet."
```

```
},
```

```
{
```

```
  "id_formation": "formation4",
```

```
  "nom": "Formation DaVinci Resolve - Post-production",
```

```
  "jour_semaine": 4,
```

```
  "heure_debut": "16:00:00",
```

```
  "heure_fin": "19:00:00",
```

```
  "chemins_images": [
```

```
    "Davinci_resolve/davinci_resolve1.jpeg",
```

```
    "Davinci_resolve/davinci_resolve2.jpg",
```

```
    "Davinci_resolve/davinci_resolve3.jpg",
```

```
    "Davinci_resolve/davinci_resolve4.webp"
```

```
  ],
```

```
  "description": "Passez au niveau professionnel du montage vidéo ! DaVinci Resolve combine montage, correction colorimétrique, effets visuels et post-production audio dans un seul outil. Apprenez
```

les techniques utilisées par les studios hollywoodiens pour donner vie à vos projets vidéo."

```

    },
    {
      "id_formation": "formation6",
      "nom": "Formation Obsidian - Organisation des Connaissances",
      "jour_semaine": 1,
      "heure_debut": "19:00:00",
      "heure_fin": "22:00:00",
      "chemins_images": [
        "Obsidian/Obsidian1.png",
        "Obsidian/Obsidian2.png",
        "Obsidian/Obsidian3.webp",
        "Obsidian/Obsidian4.webp"
      ],
      "description": "Organisez vos pensées comme jamais ! Obsidian est un carnet de notes puissant qui vous aide à connecter vos idées. Apprenez à créer une base de connaissances personnelle, à établir des liens entre vos notes et à booster votre productivité grâce au zettelkasten."
    },
    {
      "id_formation": "formation7",
      "nom": "Formation ONLY Office - Bureautique Collaborative",
      "jour_semaine": 2,
      "heure_debut": "14:00:00",
      "heure_fin": "17:00:00",
      "chemins_images": [
        "Onlyoffice/Onlyoffice1.png",
        "Onlyoffice/Onlyoffice2.png",
        "Onlyoffice/Onlyoffice3.png",
        "Onlyoffice/Onlyoffice4.png"
      ],
      "description": "Maîtrisez la suite bureautique collaborative ! ONLY Office intègre traitement de texte, tableur et présentation dans une interface moderne. Apprenez à créer des documents professionnels, à travailler en équipe en temps réel et à gérer vos projets efficacement."
    },
    {

```

```
"id_formation": "formation8",
"nom": "Formation Linux Ubuntu - Système d'Exploitation",
"jour_semaine": 3,
"heure_debut": "10:00:00",
"heure_fin": "13:00:00",
"chemins_images": [
  "Ubuntu/Ubuntu1.webp",
  "Ubuntu/Ubuntu2.webp",
  "Ubuntu/Ubuntu3.webp",
  "Ubuntu/Ubuntu4.png"
],
"description": "Découvrez un nouveau monde informatique !
Ubuntu est un système d'exploitation libre, sécurisé et performant.
Apprenez les bases de Linux, la gestion des fichiers en terminal,
l'installation de logiciels et les avantages d'un système open-source
pour votre quotidien numérique."
},
{
  "id_formation": "formation9",
  "nom": "Formation Bitwarden - Gestion des Mots de Passe",
  "jour_semaine": 4,
  "heure_debut": "13:00:00",
  "heure_fin": "16:00:00",
  "chemins_images": [
    "Bitwarden/Bitwarden.png",
    "Bitwarden/Bitwarden1.png",
    "Bitwarden/Bitwarden3.jpg",
    "Bitwarden/Bitwarden4.png"
  ],
  "description": "Prenez le contrôle de votre sécurité en ligne !
Bitwarden est un gestionnaire de mots de passe open-source.
Apprenez à générer des mots de passe forts, à synchroniser vos
identifiants entre appareils et à protéger vos comptes contre les
cybermenaces."
},
{
  "id_formation": "formation10",
  "nom": "Formation Notion - Organisation et Productivité",
  "jour_semaine": 5,
  "heure_debut": "10:00:00",
```



```
"heure_fin": "13:00:00",
"chemins_images": [
  "Notion/Notion1.webp",
  "Notion/Notion2.png",
  "Notion/Notion3.avif",
  "Notion/Notion4.webp"
],
"description": "Centralisez votre vie numérique ! Notion est un
espace de travail tout-en-un pour notes, bases de données,
planning et collaboration. Apprenez à créer des pages
personnalisées, des systèmes d'organisation et à automatiser vos
workflows."
},
{
  "id_formation": "formation11",
  "nom": "Formation SlimeVR - Réalité Virtuelle Corporelle",
  "jour_semaine": 5,
  "heure_debut": "14:00:00",
  "heure_fin": "17:00:00",
  "chemins_images": [
    "SlimeVR/Slimevr1.webp",
    "SlimeVR/Slimevr2.png",
    "SlimeVR/Slimevr3.jpg",
    "SlimeVR/Slimevr4.png"
  ],
  "description": "Plongez dans la réalité virtuelle corporelle !
SlimeVR est un système de tracking corporel DIY et abordable.
Apprenez à configurer des capteurs, à calibrer vos mouvements et à
vivre une immersion VR complète pour le gaming et les applications
sociales."
},
{
  "id_formation": "formation12",
  "nom": "Formation Inkscape - Dessin Vectoriel",
  "jour_semaine": 0,
  "heure_debut": "14:00:00",
  "heure_fin": "17:00:00",
  "chemins_images": [
    "Inkscape/Inkscape1.png",
    "Inkscape/Inkscape2.jpg",
```

```
"Inkscape/Inkscape3.png",
"Inkscape/Inkscape4.avif"
],
"description": "Créez des graphiques vectoriels professionnels !
Inkscape est l'alternative libre à Illustrator. Maîtrisez le dessin
vectoriel, la manipulation des chemins, la typographie et la création
de logos, icônes et illustrations évolutives."
},
{
  "id_formation": "formation13",
  "nom": "Formation Draw.io - Création de Diagrammes",
  "jour_semaine": 1,
  "heure_debut": "10:00:00",
  "heure_fin": "13:00:00",
  "chemins_images": [
    "Draw.io/Draw.io1.png",
    "Draw.io/Draw.io2.png",
    "Draw.io/Draw.io3.jpg",
    "Draw.io/Draw.io4.png"
  ],
  "description": "Visualisez vos idées complexes ! Draw.io est un
outil de diagramming gratuit et intuitif. Apprenez à créer des
organigrammes, plans architecturaux, réseaux informatiques et
schémas techniques pour communiquer efficacement."
},
{
  "id_formation": "formation14",
  "nom": "Formation Freemocap - Capture de Mouvement",
  "jour_semaine": 2,
  "heure_debut": "10:00:00",
  "heure_fin": "13:00:00",
  "chemins_images": [
    "Freemocap/freemocap1.jpg",
    "Freemocap/Freemocap2.jpg",
    "Freemocap/Freemocap3.jpg",
    "Freemocap/Freemocap4.png"
  ],
  "description": "Capturez le mouvement humain facilement !
Freemocap transforme des caméras standards en système de
motion capture. Apprenez à enregistrer des mouvements, analyser
```

la biomécanique et créer des animations réalistes sans équipement coûteux."

```

    },
    {
      "id_formation": "formation15",
      "nom": "Formation Ibis Paint - Dessin sur Mobile",
      "jour_semaine": 6,
      "heure_debut": "13:00:00",
      "heure_fin": "16:00:00",
      "chemins_images": [
        "Ibis_paint/Ibis_paint1.jpg",
        "Ibis_paint/Ibis_paint2.jpg",
        "Ibis_paint/Ibis_paint3.avif",
        "Ibis_paint/Ibis_paint4.webp"
      ],
      "description": "Peignez sur mobile comme un pro ! Ibis Paint est
une application de dessin complète pour smartphones et tablettes.
Maîtrisez les outils de dessin digital, les calques, les filtres et créez
des œuvres d'art où que vous soyez."
    }
  ]

```

et maintenant Voici la liste de mes images dans le dossier

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/src/assets/images\$ tree

```

.
├── Bitwarden
│   ├── birwarden2.png
│   ├── bitwarden1.png
│   ├── bitwarden3.png
│   └── bitwarden4.png
├── Blender
│   ├── blender1.png
│   ├── blender2.jpg
│   ├── blender3.jpg
│   └── blender4.png
├── Davinci_resolve
│   ├── davinci1.webp
│   └── davinci2.jpg

```

- | | — davinci3.jpg
- | | — davinci4.jpg
- | — Draw.io
- | | — draw1.png
- | | — draw2.jpg
- | | — draw3.jpg
- | | — draw4.png
- | — Freemocap
- | | — freemocap1.jpg
- | | — freemocap2.jpg
- | | — freemocap3.jpg
- | | — freemocap4.png
- | — Ibis_paint
- | | — ibispaint1.jpg
- | | — ibispaint2.jpg
- | | — ibispaint3.jpg
- | | — ibispaint4.jpg
- | — Inkscape
- | | — inkscape1.png
- | | — inkscape2.webp
- | | — inkscape3.png
- | | — inkscape4.jpg
- | — Kdenlive
- | | — kdenlive1.webp
- | | — kdenlive2.png
- | | — kdenlive3.png
- | | — kdenlive4.jpg
- | — Krita
- | | — krita1.avif
- | | — krita2.webp
- | | — krita3.png
- | | — krita4.webp
- | — Notion
- | | — notion1.png
- | | — notion2.png
- | | — notion3.jpg
- | | — notion4.avif
- | — Obsidian
- | | — obsidian1.png
- | | — obsidian2.jpg

```

|   |— obsidian3.png
|   |— obsidian4.png
|— Ollama
|   |— ollama1.png
|   |— ollama2.png
|   |— ollama3.png
|   |— ollama4.png
|— Onlyoffice
|   |— onlyoffice1.png
|   |— onlyoffice2.png
|   |— onlyoffice3.png
|   |— onlyoffice4.png
|— SlimeVR
|   |— slimevr1.webp
|   |— slimevr2.jpg
|   |— slimevr3.webp
|   |— slimevr4.jpg
|— Ubuntu
|   |— ubuntu1.jpg
|   |— ubuntu2.webp
|   |— ubuntu3.jpeg
|   |— ubuntu4.png

```

16 directories, 60 files

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/src/assets/images\$

je voudrais que tu mettes à jour la colonne chemins_images de la base de données pour que les chemins soient respectés. Prends en terme de root le dossier où est-ce que j'ai exécuté la commande pour obtenir le chemin de toutes les images.

Je vois le problème ! Les chemins dans votre base de données ne correspondent pas aux noms réels de vos fichiers. Voici un script SQL pour mettre à jour la colonne chemins_images avec les bons chemins :

Script de mise à jour PostgreSQL

```
sql
```

-- Mise à jour des chemins d'images pour correspondre à votre structure de dossiers

UPDATE formation SET chemins_images =

CASE id_formation

-- Formation Blender 3D

```
WHEN 'formation1' THEN ARRAY[
    'Blender/blender1.png',
    'Blender/blender2.jpg',
    'Blender/blender3.jpg',
    'Blender/blender4.png'
]
```

-- Formation Krita - Dessin Digital

```
WHEN 'formation2' THEN ARRAY[
    'Krita/krita1.avif',
    'Krita/krita2.webp',
    'Krita/krita3.png',
    'Krita/krita4.webp'
]
```

-- Formation Kdenlive - Montage Vidéo

```
WHEN 'formation3' THEN ARRAY[
    'Kdenlive/kdenlive1.webp',
    'Kdenlive/kdenlive2.png',
    'Kdenlive/kdenlive3.png',
    'Kdenlive/kdenlive4.jpg'
]
```

-- Formation DaVinci Resolve - Post-production

```
WHEN 'formation4' THEN ARRAY[
    'Davinci_resolve/davinci1.webp',
    'Davinci_resolve/davinci2.jpg',
    'Davinci_resolve/davinci3.jpg',
    'Davinci_resolve/davinci4.jpg'
]
```

-- Formation Ollama - Intelligence Artificielle

```
WHEN 'formation5' THEN ARRAY[
    'Ollama/ollama1.png',
    'Ollama/ollama2.png',
    'Ollama/ollama3.png',
    'Ollama/ollama4.png'
]
```

-- *Formation Obsidian - Organisation des Connaissances*

```
WHEN 'formation6' THEN ARRAY[
  'Obsidian/obsidian1.png',
  'Obsidian/obsidian2.jpg',
  'Obsidian/obsidian3.png',
  'Obsidian/obsidian4.png'
]
```

-- *Formation ONLY Office - Bureautique Collaborative*

```
WHEN 'formation7' THEN ARRAY[
  'Onlyoffice/onlyoffice1.png',
  'Onlyoffice/onlyoffice2.png',
  'Onlyoffice/onlyoffice3.png',
  'Onlyoffice/onlyoffice4.png'
]
```

-- *Formation Linux Ubuntu - Système d'Exploitation*

```
WHEN 'formation8' THEN ARRAY[
  'Ubuntu/ubuntu1.jpg',
  'Ubuntu/ubuntu2.webp',
  'Ubuntu/ubuntu3.jpeg',
  'Ubuntu/ubuntu4.png'
]
```

-- *Formation Bitwarden - Gestion des Mots de Passe*

```
WHEN 'formation9' THEN ARRAY[
  'Bitwarden/bitwarden1.png',
  'Bitwarden/birwarden2.png',
  'Bitwarden/bitwarden3.png',
  'Bitwarden/bitwarden4.png'
]
```

-- *Formation Notion - Organisation et Productivité*

```
WHEN 'formation10' THEN ARRAY[
  'Notion/notion1.png',
  'Notion/notion2.png',
  'Notion/notion3.jpg',
  'Notion/notion4.avif'
]
```

-- *Formation SlimeVR - Réalité Virtuelle Corporelle*

```
WHEN 'formation11' THEN ARRAY[
  'SlimeVR/slimevr1.webp',
  'SlimeVR/slimevr2.jpg',

```

```

        'SlimeVR/slimevr3.webp',
        'SlimeVR/slimevr4.jpg'
    ]

    -- Formation Inkscape - Dessin Vectoriel
    WHEN 'formation12' THEN ARRAY[
        'Inkscape/inkscape1.png',
        'Inkscape/inkscape2.webp',
        'Inkscape/inkscape3.png',
        'Inkscape/inkscape4.jpg'
    ]

    -- Formation Draw.io - Création de Diagrammes
    WHEN 'formation13' THEN ARRAY[
        'Draw.io/draw1.png',
        'Draw.io/draw2.jpg',
        'Draw.io/draw3.jpg',
        'Draw.io/draw4.png'
    ]

    -- Formation Freemocap - Capture de Mouvement
    WHEN 'formation14' THEN ARRAY[
        'Freemocap/freemocap1.jpg',
        'Freemocap/freemocap2.jpg',
        'Freemocap/freemocap3.jpg',
        'Freemocap/freemocap4.png'
    ]

    -- Formation Ibis Paint - Dessin sur Mobile
    WHEN 'formation15' THEN ARRAY[
        'Ibis_paint/ibispaint1.jpg',
        'Ibis_paint/ibispaint2.jpg',
        'Ibis_paint/ibispaint3.jpg',
        'Ibis_paint/ibispaint4.jpg'
    ]

    ELSE chemins_images
END;
```

Version alternative avec une seule requête UPDATE

sql

-- Correction des chemins d'images en une seule requête

UPDATE formation

SET chemins_images = ARRAY[

CASE

```
WHEN id_formation = 'formation1' THEN 'Blender/blender1.png'
WHEN id_formation = 'formation2' THEN 'Krita/krita1.avif'
WHEN id_formation = 'formation3' THEN 'Kdenlive/kdenlive1.webp'
WHEN id_formation = 'formation4' THEN 'Davinci_resolve/davinci1.webp'
WHEN id_formation = 'formation5' THEN 'Ollama/ollama1.png'
WHEN id_formation = 'formation6' THEN 'Obsidian/obsidian1.png'
WHEN id_formation = 'formation7' THEN 'Onlyoffice/onlyoffice1.png'
WHEN id_formation = 'formation8' THEN 'Ubuntu/ubuntu1.jpg'
WHEN id_formation = 'formation9' THEN 'Bitwarden/bitwarden1.png'
WHEN id_formation = 'formation10' THEN 'Notion/notion1.png'
WHEN id_formation = 'formation11' THEN 'SlimeVR/slimevr1.webp'
WHEN id_formation = 'formation12' THEN 'Inkscape/inkscape1.png'
WHEN id_formation = 'formation13' THEN 'Draw.io/draw1.png'
WHEN id_formation = 'formation14' THEN 'Freemocap/freemocap1.jpg'
WHEN id_formation = 'formation15' THEN 'Ibis_paint/ibispaint1.jpg'
```

END,

CASE

```
WHEN id_formation = 'formation1' THEN 'Blender/blender2.jpg'
WHEN id_formation = 'formation2' THEN 'Krita/krita2.webp'
WHEN id_formation = 'formation3' THEN 'Kdenlive/kdenlive2.png'
WHEN id_formation = 'formation4' THEN 'Davinci_resolve/davinci2.jpg'
WHEN id_formation = 'formation5' THEN 'Ollama/ollama2.png'
WHEN id_formation = 'formation6' THEN 'Obsidian/obsidian2.jpg'
WHEN id_formation = 'formation7' THEN 'Onlyoffice/onlyoffice2.png'
WHEN id_formation = 'formation8' THEN 'Ubuntu/ubuntu2.webp'
WHEN id_formation = 'formation9' THEN 'Bitwarden/birwarden2.png'
WHEN id_formation = 'formation10' THEN 'Notion/notion2.png'
WHEN id_formation = 'formation11' THEN 'SlimeVR/slimevr2.jpg'
WHEN id_formation = 'formation12' THEN 'Inkscape/inkscape2.webp'
WHEN id_formation = 'formation13' THEN 'Draw.io/draw2.jpg'
WHEN id_formation = 'formation14' THEN 'Freemocap/freemocap2.jpg'
WHEN id_formation = 'formation15' THEN 'Ibis_paint/ibispaint2.jpg'
```

END,

CASE

```
WHEN id_formation = 'formation1' THEN 'Blender/blender3.jpg'
WHEN id_formation = 'formation2' THEN 'Krita/krita3.png'
WHEN id_formation = 'formation3' THEN 'Kdenlive/kdenlive3.png'
WHEN id_formation = 'formation4' THEN 'Davinci_resolve/davinci3.jpg'
WHEN id_formation = 'formation5' THEN 'Ollama/ollama3.png'
```

```

WHEN id_formation = 'formation6' THEN 'Obsidian/obsidian3.png'
WHEN id_formation = 'formation7' THEN 'Onlyoffice/onlyoffice3.png'
WHEN id_formation = 'formation8' THEN 'Ubuntu/ubuntu3.jpeg'
WHEN id_formation = 'formation9' THEN 'Bitwarden/bitwarden3.png'
WHEN id_formation = 'formation10' THEN 'Notion/notion3.jpg'
WHEN id_formation = 'formation11' THEN 'SlimeVR/slimevr3.webp'
WHEN id_formation = 'formation12' THEN 'Inkscape/inkscape3.png'
WHEN id_formation = 'formation13' THEN 'Draw.io/draw3.jpg'
WHEN id_formation = 'formation14' THEN 'Freemocap/freemocap3.jpg'
WHEN id_formation = 'formation15' THEN 'Ibis_paint/ibispaint3.jpg'
END,
CASE
WHEN id_formation = 'formation1' THEN 'Blender/blender4.png'
WHEN id_formation = 'formation2' THEN 'Krita/krita4.webp'
WHEN id_formation = 'formation3' THEN 'Kdenlive/kdenlive4.jpg'
WHEN id_formation = 'formation4' THEN 'Davinci_resolve/davinci4.jpg'
WHEN id_formation = 'formation5' THEN 'Ollama/ollama4.png'
WHEN id_formation = 'formation6' THEN 'Obsidian/obsidian4.png'
WHEN id_formation = 'formation7' THEN 'Onlyoffice/onlyoffice4.png'
WHEN id_formation = 'formation8' THEN 'Ubuntu/ubuntu4.png'
WHEN id_formation = 'formation9' THEN 'Bitwarden/bitwarden4.png'
WHEN id_formation = 'formation10' THEN 'Notion/notion4.avif'
WHEN id_formation = 'formation11' THEN 'SlimeVR/slimevr4.jpg'
WHEN id_formation = 'formation12' THEN 'Inkscape/inkscape4.jpg'
WHEN id_formation = 'formation13' THEN 'Draw.io/draw4.png'
WHEN id_formation = 'formation14' THEN 'Freemocap/freemocap4.png'
WHEN id_formation = 'formation15' THEN 'Ibis_paint/ibispaint4.jpg'
END
];

```

Vérification après mise à jour

```

sql

-- Vérifier que Les chemins ont été mis à jour
SELECT id_formation, nom, chemins_images
FROM formation
ORDER BY id_formation;

```

Mise à jour de votre API Node.js

N'oubliez pas de mettre à jour votre serveur pour servir les images statiques :

javascript

```
// Ajouter cette ligne dans votre server.js
app.use('/images', express.static('src/assets/images'));

// Ou selon votre structure :
app.use('/images', express.static(path.join(__dirname, 'src/assets/images')));
```

Dans votre composant React

Assurez-vous que les URLs des images pointent vers votre serveur :

jsx

```
// Dans AccueilFormations.jsx
const imageUrl = `http://localhost:5000/images/${formation.images[0]}`;
```

Maintenant vos chemins d'images dans la base de données correspondent exactement à la structure de vos dossiers !

Le fichier serveur dot server.js qui se situe dans le dossier Backend/ par rapport à la racine de mon projet ressemble à cela :

```
// server.js
const express = require('express');
const cors = require('cors');
const formationRoutes = require('./routes/formations');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
```

```
console.log(`Serveur démarré sur le port ${PORT}`);  
});
```

montre-moi ce que je devrais modifier

Voici les modifications à apporter à votre `server.js` pour servir les images statiques et bien configurer votre API :

server.js modifié

javascript

```
// server.js  
const express = require('express');  
const cors = require('cors');  
const path = require('path');  
const formationRoutes = require('./routes/formations');  
  
const app = express();  
const PORT = process.env.PORT || 3000;  
  
app.use(cors());  
app.use(express.json());  
  
// Servir les images statiques depuis le dossier src/assets/images  
// Note: Adaptez le chemin selon votre structure de projet  
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));  
  
// Route de test pour vérifier que le serveur fonctionne  
app.get('/api/health', (req, res) => {  
  res.json({ status: 'OK', message: 'Serveur formations démarré' });  
});  
  
// Routes  
app.use('/api', formationRoutes);  
  
// Gestion des routes non trouvées  
app.use('*', (req, res) => {  
  res.status(404).json({ error: 'Route non trouvée' });  
});
```

```
// Gestion des erreurs
app.use((err, req, res, next) => {
  console.error('Erreur serveur:', err);
  res.status(500).json({ error: 'Erreur interne du serveur' });
});

app.listen(PORT, () => {
  console.log(`✅ Serveur démarré sur le port ${PORT}`);
  console.log(`📁 Dossier images: ${path.join(__dirname, '../src/assets/images')}`);
});
```

Vérification de la structure de votre projet

Assurez-vous que votre structure de projet ressemble à ceci :

text

```
votre-projet/
├─ Backend/
│   ├─ server.js
│   ├─ routes/
│   │   └─ formations.js
│   └─ package.json
├─ src/
│   └─ assets/
│       └─ images/
│           ├─ Bitwarden/
│           ├─ Blender/
│           ├─ Davinci_resolve/
│           └─ ... (tous vos dossiers)
└─ (autres dossiers frontend)
```

routes/formations.js mis à jour

Voici comment votre fichier de routes devrait ressembler :

javascript

```
// routes/formations.js
const express = require('express');
const { Pool } = require('pg');
```

```
const router = express.Router();

// Configuration de la base de données
const pool = new Pool({
  user: process.env.DB_USER || 'votre_utilisateur',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'votre_base',
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',
  port: process.env.DB_PORT || 5432,
});

// Route pour récupérer toutes les formations
router.get('/formations', async (req, res) => {
  try {
    const query = `
      SELECT
        id_formation,
        nom as titre,
        chemins_image as images,
        description,
        jour_semaine,
        heure_debut,
        heure_fin
      FROM formation
      ORDER BY jour_semaine, heure_debut
    `;

    const result = await pool.query(query);

    // Transformer les données pour correspondre au format React
    const formations = {};
    result.rows.forEach(formation => {
      formations[formation.id_formation] = {
        titre: formation.titre,
        images: formation.images || [],
        description: formation.description,
        jour_semaine: formation.jour_semaine,
        heure_debut: formation.heure_debut,
        heure_fin: formation.heure_fin
      };
    });

    res.json(formations);
  } catch (error) {
```

```

    console.error('Erreur lors de la récupération des formations:', error);
    res.status(500).json({ error: 'Erreur serveur lors de la récupération des formations' });
  }
});

// Route pour récupérer une formation spécifique
router.get('/formations/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const query = `
      SELECT * FROM formation WHERE id_formation = $1
    `;

    const result = await pool.query(query, [id]);

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Formation non trouvée' });
    }

    res.json(result.rows[0]);
  } catch (error) {
    console.error('Erreur:', error);
    res.status(500).json({ error: 'Erreur serveur' });
  }
});

module.exports = router;

```

Fichier .env pour la configuration

Créez un fichier .env dans votre dossier Backend/ :

```

env

DB_USER=votre_utilisateur_postgres
DB_HOST=localhost
DB_NAME=nom_de_votre_base
DB_PASSWORD=votre_mot_de_passe
DB_PORT=5432
PORT=3000

```

Test de votre API

Une fois les modifications faites, testez avec :

1. **Santé du serveur** : GET `http://localhost:3000/api/health`
2. **Formations** : GET `http://localhost:3000/api/formations`
3. **Images** : GET `http://localhost:3000/images/Blender/blender1.png`

Mise à jour dans React

Dans votre composant React, les URLs d'images seront maintenant :

jsx

```
// Dans AccueilFormations.jsx
const imageUrl = `http://localhost:3000/images/${formation.images[0]}`;

// Ou en production :
const imageUrl = `/images/${formation.images[0]}`;
```

Les principales modifications sont :

1. **Ajout de** `path` pour gérer les chemins de fichiers
2. **Middleware static** pour servir les images
3. **Route de santé** pour tester l'API
4. **Gestion d'erreurs** améliorée
5. **Logs** plus informatifs

Votre API est maintenant prête à servir à la fois les données et les images !

Je voudrais maintenant appliquez cela à la première visualisation de mon application. Je voudrais que lorsque l'utilisateur entre sur le site il soit tout de suite redirigé vers la page d'accueil des formations qui se trouve dans le dossier `src/components/AccueilFormations.jsx`. Le fichier `App.jsx` se trouve dans `src/App.jsx`. Il y a aussi le fichier `Backend/server.js` et `routes/formation.js`. Il y a aussi le fichier `Backend/formationController.js` qui contient des fonction utile dans la base de données. Je veux que lorsque l'utilisateur arrive sur le site il soit redirigé vers la page d'accueil qui présente toutes les

formations selon la base de données. Je ne veux pas que tu modifies le design des pages seulement l'information qui est présentée.

Voici ce qui se trouve dans le fichier Backend/server.js

```
// server.js
const express = require('express');
const cors = require('cors');
const formationRoutes = require('./routes/formations');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

voici ce qui se trouve dans le fichier Backend/formationController.js :

```
// formationController.js
const pool = require('./db');

// Obtenir toutes les formations
async function obtenirToutesFormations() {
  try {
    const result = await pool.query('SELECT * FROM formation
ORDER BY nom');
    return result.rows;
  } catch (error) {
    console.error('Erreur lors de la récupération des formations:',
error);
    throw error;
  }
}
```

```
}  
}  
  
// Obtenir une formation par ID  
async function obtenirFormationParId(idFormation) {  
  try {  
    const result = await pool.query('SELECT * FROM formation  
WHERE id_formation = $1', [idFormation]);  
    return result.rows[0];  
  } catch (error) {  
    console.error('Erreur lors de la récupération de la formation:',  
error);  
    throw error;  
  }  
}  
  
// Créer une nouvelle formation  
async function creerFormation(formationData) {  
  const { nom, jour_semaine, heure_debut, heure_fin,  
chemins_images, description } = formationData;  
  
  try {  
    const result = await pool.query(  
      `INSERT INTO formation (nom, jour_semaine, heure_debut,  
heure_fin, chemins_images, description)  
VALUES ($1, $2, $3, $4, $5, $6)  
RETURNING *`,  
      [nom, jour_semaine, heure_debut, heure_fin, chemins_images,  
description]  
    );  
    return result.rows[0];  
  } catch (error) {  
    console.error('Erreur lors de la création de la formation:', error);  
    throw error;  
  }  
}  
  
// Mettre à jour une formation  
async function mettreAJourFormation(idFormation, formationData) {  
  const { nom, jour_semaine, heure_debut, heure_fin,
```

```
chemins_images, description } = formationData;
```

```
try {
  const result = await pool.query(
    `UPDATE formation
     SET nom = $1, jour_semaine = $2, heure_debut = $3, heure_fin
     = $4, chemins_images = $5, description = $6
     WHERE id_formation = $7
     RETURNING *`,
    [nom, jour_semaine, heure_debut, heure_fin, chemins_images,
description, idFormation]
  );
  return result.rows[0];
} catch (error) {
  console.error('Erreur lors de la mise à jour de la formation:', error);
  throw error;
}
}
```

```
// Ajouter une image à une formation
```

```
async function ajouterImageFormation(idFormation, cheminImage) {
  try {
    const result = await pool.query(
      'SELECT ajouter_image_formation($1, $2)',
      [idFormation, cheminImage]
    );
    return { success: true, message: 'Image ajoutée avec succès' };
  } catch (error) {
    console.error('Erreur lors de l\'ajout de l\'image:', error);
    throw error;
  }
}
```

```
// Supprimer une image d'une formation
```

```
async function supprimerImageFormation(idFormation,
cheminImage) {
  try {
    const result = await pool.query(
      'SELECT supprimer_image_formation($1, $2)',
      [idFormation, cheminImage]
    );
  }
}
```

```

    );
    return { success: true, message: 'Image supprimée avec succès'
  };
} catch (error) {
  console.error('Erreur lors de la suppression de l\'image:', error);
  throw error;
}
}
}

```

```

// Obtenir la première image d'une formation
async function obtenirPremiereImageFormation(idFormation) {
  try {
    const result = await pool.query(
      'SELECT obtenir_premiere_image_formation($1) as
premiere_image',
      [idFormation]
    );
    return result.rows[0].premiere_image;
  } catch (error) {
    console.error('Erreur lors de la récupération de la première
image:', error);
    throw error;
  }
}

```

voici ce qui se trouve dans le fichier routes/formation.js :

```

// routes/formations.js
const express = require('express');
const router = express.Router();
const formationController =
require('../controllers/formationController');
const formationInstanceController =
require('../controllers/formationInstanceController');
const reservationController =
require('../controllers/reservationController');

// Routes pour les formations
router.get('/formations', async (req, res) => {

```

```
try {
  const formations = await
formationController.obtenirToutesFormations();
  res.json(formations);
} catch (error) {
  res.status(500).json({ error: error.message });
}
});

router.get('/formations/:id', async (req, res) => {
  try {
    const formation = await
formationController.obtenirFormationParId(req.params.id);
    if (!formation) {
      return res.status(404).json({ error: 'Formation non trouvée' });
    }
    res.json(formation);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Routes pour les instances
router.get('/instances', async (req, res) => {
  try {
    const instances = await
formationInstanceController.obtenirInstancesDetaillees();
    res.json(instances);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/instances/:id', async (req, res) => {
  try {
    const instance = await
formationInstanceController.obtenirInstanceParId(req.params.id);
    if (!instance) {
      return res.status(404).json({ error: 'Instance non trouvée' });
    }
  }
});
```

```
    res.json(instance);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Route pour les inscriptions
router.post('/reservations', async (req, res) => {
  try {
    const idReservation = await
reservationController.inscrireFormation(req.body);
    res.json({
      success: true,
      message: 'Inscription réussie',
      id_reservation: idReservation
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// Route pour les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const formations = await
reservationController.listerFormationsDisponibles();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

voici ce qui se trouve dans le fichier
src/components/AccueilFormations.jsx :

```
// AccueilFormations.jsx
```

```
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"]
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: "",
    password: ""
  });

  const handleFormationClick = (formationId) => {
    if (onFormationSelect) {
      onFormationSelect(formationId);
    }
  };

  const handleAdminLogin = (e) => {
    e.preventDefault();
    // Simulation de connexion admin
    console.log('Tentative de connexion admin:', adminCredentials);
    if (adminCredentials.username && adminCredentials.password) {
      alert('Connexion administrateur réussie !');
    }
  };
}
```

```

    setShowAdminLogin(false);
    setAdminCredentials({ username: '', password: '' });
  } else {
    alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
  }
};

```

```

const handleAdminInputChange = (e) => {
  const { name, value } = e.target;
  setAdminCredentials(prev => ({
    ...prev,
    [name]: value
  }));
};

```

```

return (
  <div className="accueil-container">
    { /* Header avec bouton admin */ }
    <header className="accueil-header">
      <div className="header-content">
        <h1 className="accueil-title">Formations en Ligne</h1>
        <button
          className="admin-button"
          onClick={() => setShowAdminLogin(true)}
        >
           Administrateur
        </button>
      </div>
    </header>

    { /* Contenu principal */ }
    <main className="accueil-main">
      <div className="formations-grid">
        {Object.entries(formations).map(([formationId, formation]) =>
          (
            <div
              key={formationId}
              className="formation-card"
              onClick={() => handleFormationClick(formationId)}
            >

```



```

<div className="formation-image-container">
  <img
    src={formation.images[0]}
    alt={formation.titre}
    className="formation-image"
    onError={(e) => {
      e.target.src =
`https://via.placeholder.com/300x200/667eea/ffffff?
text=${encodeURIComponent(formation.titre)}`;
    }}
  />
</div>
<div className="formation-info">
  <h3 className="formation-titre">{formation.titre}</h3>
  <div className="formation-overlay">
    <span className="voir-details">Voir les détails
→</span>
  </div>
</div>
</div>
  )}
</div>

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

{/* Modal de connexion admin */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">
        <h2>Connexion Administrateur</h2>
        <button
          className="close-button"
          onClick={() => setShowAdminLogin(false)}
        >

```

```
      x
    </button>
  </div>
```

```
    <form className="admin-form" onSubmit=
{handleAdminLogin}>
      <div className="form-group">
        <label htmlFor="username">Nom d'utilisateur</label>
        <input
          type="text"
          id="username"
          name="username"
          value={adminCredentials.username}
          onChange={handleAdminInputChange}
          placeholder="Entrez votre nom d'utilisateur"
          required
        />
      </div>
```

```
      <div className="form-group">
        <label htmlFor="password">Mot de passe</label>
        <input
          type="password"
          id="password"
          name="password"
          value={adminCredentials.password}
          onChange={handleAdminInputChange}
          placeholder="Entrez votre mot de passe"
          required
        />
      </div>
```

```
      <div className="admin-form-buttons">
        <button type="submit" className="login-button">
          Se connecter
        </button>
        <button
          type="button"
          className="cancel-button"
          onClick={() => setShowAdminLogin(false)}
        >
```

```

        >
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  })
</div>
);
};

```

export default AccueilFormations;

et finalement voici ce qui se trouve dans le fichier src/App.jsx :

```

import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

```

```

// App.jsx
import React from 'react';
import PageAdministrateur from './components/PageAdministrateur';
import './App.css';

```

```

function App() {
  // Données de test pour l'admin
  const formationsAdmin = [
    {
      id: "formation-react",
      nom: "Formation React Avancé",
      date: "2024-02-15",
      heureDebut: "14:00",
      inscrits: 28
    },
    {

```

```
id: "formation-js",
nom: "Formation JavaScript Moderne",
date: "2024-02-20",
heureDebut: "10:00",
inscrits: 15
},
{
id: "formation-node",
nom: "Formation Node.js & Express",
date: "2024-02-10",
heureDebut: "09:00",
inscrits: 30
},
{
id: "formation-angular",
nom: "Formation Angular",
date: "2024-02-25",
heureDebut: "16:00",
inscrits: 12
},
{
id: "formation-vue",
nom: "Formation Vue.js",
date: "2024-02-18",
heureDebut: "13:00",
inscrits: 25
},
{
id: "formation-python",
nom: "Formation Python Django",
date: "2024-02-22",
heureDebut: "11:00",
inscrits: 30
}
];

const handleRetourAccueil = () => {
  console.log('Retour à l\'accueil');
  // Pour l'instant, on reste sur la page admin
  alert('Fonctionnalité de retour à implémenter');
```

```
};

return (
  <PageAdministrateur
    formations={formationsAdmin}
    onRetourAccueil={handleRetourAccueil}
  />
);
}

export default App;
```

si tu veux tu peux dire uniquement les places qui sont à modifier.
Pas obligé de tout réécrire

Parfait ! Voici uniquement les modifications nécessaires :

1. Modifier Backend/server.js - Ajouter le service des images

javascript

```
// server.js
const express = require('express');
const cors = require('cors');
const path = require('path');
const formationRoutes = require('./routes/formations');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());
```

```
// ✅ AJOUT: Servir Les images statiques
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

2. Modifier src/App.jsx - Afficher l'accueil des formations

javascript

```
// App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import PageAdministrateur from './components/PageAdministrateur';
import './App.css';

function App() {
  const [currentPage, setCurrentPage] = useState('accueil'); // ✅ MODIFIÉ: 'accueil' par défaut
  const [formations, setFormations] = useState({});

  // ✅ AJOUT: Récupérer Les formations depuis L'API
  useEffect(() => {
    const fetchFormations = async () => {
      try {
        const response = await fetch('http://localhost:3000/api/formations');
        if (response.ok) {
          const data = await response.json();
          setFormations(data);
        }
      } catch (error) {
        console.error('Erreur:', error);
      }
    };
    fetchFormations();
  }, []);

  const handleAdminLogin = () => {
```

```

    setCurrentPage('admin');
  };

  const handleRetourAccueil = () => {
    setCurrentPage('accueil');
  };

  // ✅ MODIFIÉ: Afficher l'accueil par défaut
  if (currentPage === 'admin') {
    return (
      <PageAdministrateur
        formations={[]} // Vous pouvez adapter si nécessaire
        onRetourAccueil={handleRetourAccueil}
      />
    );
  }

  return (
    <AccueilFormations
      formations={formations}
      onFormationSelect={(id) => console.log('Formation sélectionnée:', id)}
      onAdminLogin={handleAdminLogin}
    />
  );
}

export default App;

```

3. Modifier src/components/AccueilFormations.jsx - Adapter au format API

javascript

```

// AccueilFormations.jsx
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {}, // ✅ MODIFIÉ: objet vide par défaut
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  },

```

```

    onAdminLogin = () => {} // ✅ AJOUT: callback pour l'admin
  }) => {
    const [showAdminLogin, setShowAdminLogin] = useState(false);
    const [adminCredentials, setAdminCredentials] = useState({
      username: '',
      password: ''
    });

    const handleFormationClick = (formationId) => {
      onFormationSelect(formationId);
    };

    const handleAdminLogin = (e) => {
      e.preventDefault();
      if (adminCredentials.username && adminCredentials.password) {
        onAdminLogin(); // ✅ MODIFIÉ: Utiliser le callback parent
        setShowAdminLogin(false);
        setAdminCredentials({ username: '', password: '' });
      } else {
        alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
      }
    };

    // ✅ AJOUT: Fonction pour construire l'URL de l'image
    const getImageUrl = (imagePath) => {
      return `http://localhost:3000/images/${imagePath}`;
    };

    return (
      <div className="accueil-container">
        {/* Contenu principal */}
        <main className="accueil-main">
          <div className="formations-grid">
            {Object.entries(formations).map(([formationId, formation]) => (
              <div
                key={formationId}
                className="formation-card"
                onClick={() => handleFormationClick(formationId)}
              >
                <div className="formation-image-container">
                  <img
                    src={formation.images && formation.images[0]
                      ? getImageUrl(formation.images[0]) // ✅ MODIFIÉ: URL complète
                      : `https://via.placeholder.com/300x200/667eea/ffffff?text=${encodeURIComponent(formation.title)}`
                    alt={formation.title}
                  />
                </div>
                <div>
                  <h3>{formation.title}</h3>
                  <p>{formation.description}</p>
                  <p>{formation.price}</p>
                </div>
              </div>
            ))}
          </div>
        </main>
      </div>
    );
  });

```



```

    eURIComponent(formation.titre)}`
    }
    alt={formation.titre}
    className="formation-image"
    onError={(e) => {
      e.target.src = `https://via.placeholder.com/300x200/667eea/ffffff
f?text=${encodeURIComponent(formation.titre)}`;
    }}
  />
</div>
<div className="formation-info">
  <h3 className="formation-titre">{formation.titre}</h3>
  <div className="formation-overlay">
    <span className="voir-details">Voir les détails →</span>
  </div>
</div>
</div>
  )})
</div>

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

{/* Modal de connexion admin (inchangé) */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    {/* ... Le reste du modal reste identique ... */}
  </div>
)}
</div>
);
};

export default AccueilFormations;

```

Résumé des modifications :

1. Backend : Ajout du service des images statiques

2. **App.jsx** : Affichage de l'accueil par défaut avec récupération des données API
3. **AccueilFormations.jsx** : Adaptation au format des données API et URLs d'images

Maintenant quand l'utilisateur arrive sur le site, il verra directement les formations depuis votre base de données !

Excuse-moi finalement je ne veux que le fait que App.jsx appelle la page d'accueil avec les paramètres qui sont demandés pour toutes les formations. La seule chose qu'il faut faire avant c'est utiliser la base de données pour aller chercher cette information.

Parfait ! Voici **seulement les modifications minimales** nécessaires :

1. Modifier Backend/server.js - Ajouter le service des images

javascript

```
// server.js
const express = require('express');
const cors = require('cors');
const path = require('path');
const formationRoutes = require('./routes/formations');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// ✅ AJOUT: Servir Les images statiques
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

2. Modifier src/App.jsx - Appeler l'accueil avec les données

javascript

```
// App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import './App.css';

function App() {
  const [formations, setFormations] = useState({});

  // ✅ AJOUT: Récupérer les formations depuis l'API
  useEffect(() => {
    const fetchFormations = async () => {
      try {
        const response = await fetch('http://localhost:3000/api/formations');
        if (response.ok) {
          const data = await response.json();
          setFormations(data);
        }
      } catch (error) {
        console.error('Erreur:', error);
      }
    };
    fetchFormations();
  }, []);

  // ✅ MODIFIÉ: Appeler AccueilFormations avec les données
  return (
    <AccueilFormations
      formations={formations}
      onFormationSelect={(formationId) => {
        console.log('Formation sélectionnée:', formationId);
      }}
    />
  );
}

export default App;
```

C'est tout ! Seulement ces 2 fichiers à modifier. Votre `AccueilFormations.jsx` reste exactement le même, il recevra automatiquement les données de la base de données.

Ça ne marche pas ça dit que il n'y a aucune formation disponible. Je te rappelle que je suis sur une base de données postgres qui roule sur le port 5431.

Il faut configurer la connexion à la base de données PostgreSQL sur le port 5431. Voici les corrections :

1. Créer/modifier le fichier `Backend/.env`

`env`

```
DB_USER=postgres
DB_HOST=localhost
DB_NAME=ton_nom_de_base
DB_PASSWORD=ton_mot_de_passe
DB_PORT=5431
PORT=3000
```

2. Modifier `Backend/db.js` (ou le fichier de configuration de la base)

Si vous avez un fichier `db.js` dans `Backend/` :

`javascript`

```
// Backend/db.js
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT || 5431, // ✅ Port 5431
});
```

```
module.exports = pool;
```

3. Vérifier que `formationController.js` utilise la bonne connexion

Dans `Backend/formationController.js`, assurez-vous qu'il utilise bien le pool :

javascript

```
// formationController.js
const pool = require('./db'); // ✅ Doit pointer vers votre configuration

// Obtenir toutes les formations
async function obtenirToutesFormations() {
  try {
    const result = await pool.query('SELECT * FROM formation ORDER BY nom');
    return result.rows;
  } catch (error) {
    console.error('Erreur lors de la récupération des formations:', error);
    throw error;
  }
}
```

4. Tester l'API directement

Ouvrez votre navigateur et allez à :

`http://localhost:3000/api/formations`

Vous devriez voir le JSON avec vos formations. Si ça ne marche pas, le problème vient du backend.

5. Vérifier les logs du serveur

Lancez le backend et regardez les logs. Si vous voyez une erreur de connexion à la base de données, c'est que les paramètres de connexion ne sont pas bons.

Les problèmes possibles :

- Mauvais mot de PostgreSQL
- Base de données n'existe pas
- PostgreSQL n'écoute pas sur le port 5431

Testez d'abord l'API directement dans le navigateur pour isoler le problème !

J'ai essayé de tester le API et ça dit que le site est inaccessible

Le problème est que votre serveur backend ne démarre pas ou n'écoute pas sur le port 3000. Voici la procédure de dépannage :

1. Vérifier que le backend est démarré

Ouvrez un terminal dans le dossier `Backend/` et exécutez :

```
bash
```

```
cd Backend
```

```
npm start
```

ou

```
bash
```

```
cd Backend
```

```
node server.js
```

Vous devriez voir : Serveur démarré sur le port 3000

2. Vérifier les dépendances

Assurez-vous que `package.json` dans `Backend/` contient :

```
json
```

```
{  
  "dependencies": {  
    "express": "^4.18.2",  
    "cors": "^2.8.5",  
    "pg": "^8.11.3",
```

```
"dotenv": "^16.3.1"  
}  
}
```

Si non, installez-les :

```
bash
```

```
cd Backend
```

```
npm install express cors pg dotenv
```

3. Vérifier la structure des fichiers

Votre dossier Backend/ doit contenir :

```
text
```

```
Backend/  
├─ server.js  
├─ .env  
├─ db.js  
├─ routes/  
│   └─ formations.js  
├─ controllers/  
│   └─ formationController.js  
└─ package.json
```

4. Créer le fichier Backend/db.js si manquant

```
javascript
```

```
// Backend/db.js  
const { Pool } = require('pg');  
require('dotenv').config();  
  
const pool = new Pool({  
  user: process.env.DB_USER || 'postgres',  
  host: process.env.DB_HOST || 'localhost',  
  database: process.env.DB_NAME || 'votre_base',  
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',  
  port: process.env.DB_PORT || 5431,  
});
```

```
});
```

```
module.exports = pool;
```

5. Vérifier que PostgreSQL tourne sur le port 5431

```
bash
```

```
# Vérifier si PostgreSQL écoute sur le port 5431  
sudo netstat -tulpn | grep 5431
```

6. Tester la connexion à la base manuellement

Créez un fichier `test-db.js` dans `Backend/` :

```
javascript
```

```
// test-db.js  
const pool = require('./db');  
  
async function testConnection() {  
  try {  
    const result = await pool.query('SELECT NOW()');  
    console.log('✅ Connexion BD réussie:', result.rows[0]);  
  
    const formations = await pool.query('SELECT COUNT(*) FROM formation');  
    console.log('✅ Nombre de formations:', formations.rows[0].count);  
  } catch (error) {  
    console.error('❌ Erreur connexion BD:', error);  
  }  
}  
  
testConnection();
```

Exécutez : `node test-db.js`

7. Démarrer le backend en mode debug

Modifiez `server.js` temporairement :

javascript

```
// server.js
const express = require('express');
const cors = require('cors');
const path = require('path');
const formationRoutes = require('./routes/formations');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Route de test
app.get('/test', (req, res) => {
  res.json({ message: 'Backend fonctionne!' });
});

app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`✅ Serveur démarré sur http://localhost:${PORT}`);
});
```

Testez d'abord : <http://localhost:3000/test>

Quelle étape échoue ?

```
wizcolas@Purple-Neon:/mnt/c/Users/nicol/Desktop/softghost$ cd Backend
wizcolas@Purple-Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$ npm start
npm error Missing script: "start"
npm error
npm error Did you mean one of these?
npm error   npm star # Mark your favorite packages
npm error   npm stars # View packages marked as favorites
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in:
```

```
/home/wizcolas/.npm/_logs/2025-11-04T02_34_59_863Z-debug-0.log
wizcolas@Purple-Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$ node server.js
file:///mnt/c/Users/nicol/Desktop/softghost/Backend/server.js:2
const express = require('express');
                ^
```

ReferenceError: require is not defined in ES module scope, you can use import instead

This file is being treated as an ES module because it has a '.js' file extension and '/mnt/c/Users/nicol/Desktop/softghost/package.json' contains "type": "module". To treat it as a CommonJS script, rename it to use the '.cjs' file extension.

```
at
file:///mnt/c/Users/nicol/Desktop/softghost/Backend/server.js:2:17
at ModuleJob.run
(node:internal/modules/esm/module_job:343:25)
at async onImport.tracePromise.__proto__
(node:internal/modules/esm/loader:665:26)
at async asyncRunEntryPointWithESMLoader
(node:internal/modules/run_main:117:5)
```

Node.js v22.21.0

Le problème est que votre projet utilise ES modules (type: "module") mais votre code backend est en CommonJS. Voici les corrections :

Solution 1 : Convertir en ES modules (Recommandé)

1. Modifier Backend/server.js

javascript

```
// Backend/server.js
import express from 'express';
import cors from 'cors';
import path from 'path';
import { fileURLToPath } from 'url';
```

```
import formationRoutes from './routes/formations.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

2. Modifier Backend/routes/formations.js

javascript

```
// Backend/routes/formations.js
import express from 'express';
import formationController from '../controllers/formationController.js';

const router = express.Router();

// Routes pour les formations
router.get('/formations', async (req, res) => {
  try {
    const formations = await formationController.obtenirToutesFormations();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// ... autres routes

export default router;
```

3. Modifier Backend/controllers/formationController.js

javascript

```
// Backend/controllers/formationController.js
import pool from '../db.js';

// Obtenir toutes les formations
async function obtenirToutesFormations() {
  try {
    const result = await pool.query('SELECT * FROM formation ORDER BY nom');
    return result.rows;
  } catch (error) {
    console.error('Erreur lors de la récupération des formations:', error);
    throw error;
  }
}

// ... autres fonctions

export default {
  obtenirToutesFormations,
  obtenirFormationParId,
  creerFormation,
  mettreAJourFormation,
  ajouterImageFormation,
  supprimerImageFormation,
  obtenirPremiereImageFormation
};
```

4. Créer Backend/db.js

javascript

```
// Backend/db.js
import pg from 'pg';
const { Pool } = pg;

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'votre_base',
```

```
password: process.env.DB_PASSWORD || 'votre_mot_de_passe',  
port: process.env.DB_PORT || 5431,  
});  
  
export default pool;
```

5. Créer Backend/package.json

json

```
{  
  "name": "backend",  
  "type": "module",  
  "scripts": {  
    "start": "node server.js",  
    "dev": "node --watch server.js"  
  },  
  "dependencies": {  
    "express": "^4.18.2",  
    "cors": "^2.8.5",  
    "pg": "^8.11.3"  
  }  
}
```

6. Installer les dépendances dans le dossier Backend

bash

```
cd Backend  
npm install express cors pg
```

Solution 2 : Forcer CommonJS (Alternative)

Si vous préférez garder CommonJS, créez un package.json dans Backend/ :

json

```
{  
  "name": "backend",  
  "type": "commonjs",
```

```
"scripts": {  
  "start": "node server.js"  
}  
}
```

Et renommez `server.js` en `server.cjs`

Démarrer le serveur

```
bash
```

```
cd Backend
```

```
npm start
```

Maintenant votre API devrait fonctionner sur `http://localhost:3000/api/formations`

```
wizcolas@Purple-
```

```
Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$ tree
```

```
.  
├── db.js  
├── emailer.js  
├── formationController.js  
├── formationInstanceController.js  
├── reservationController.js  
├── routes  
│   └── formations.js  
└── server.js
```

```
2 directories, 7 files
```

```
wizcolas@Purple-
```

```
Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$
```

le dossier backend ressemble à ça

Parfait ! Voici la solution complète pour convertir tous vos fichiers en ES modules :

1. Modifier Backend/server.js

javascript

```
// Backend/server.js
import express from 'express';
import cors from 'cors';
import path from 'path';
import { fileURLToPath } from 'url';
import formationRoutes from './routes/formations.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

2. Modifier Backend/db.js

javascript

```
// Backend/db.js
import pg from 'pg';
const { Pool } = pg;

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'votre_base',
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',
  port: process.env.DB_PORT || 5431,
});
```

```
export default pool;
```

3. Modifier Backend/formationController.js

```
javascript
```

```
// Backend/formationController.js
```

```
import pool from './db.js';
```

```
// Obtenir toutes les formations
```

```
async function obtenirToutesFormations() {  
  try {  
    const result = await pool.query('SELECT * FROM formation ORDER BY nom');  
    return result.rows;  
  } catch (error) {  
    console.error('Erreur lors de la récupération des formations:', error);  
    throw error;  
  }  
}
```

```
// Obtenir une formation par ID
```

```
async function obtenirFormationParId(idFormation) {  
  try {  
    const result = await pool.query('SELECT * FROM formation WHERE id_formation =  
$1', [idFormation]);  
    return result.rows[0];  
  } catch (error) {  
    console.error('Erreur lors de la récupération de la formation:', error);  
    throw error;  
  }  
}
```

```
// Créer une nouvelle formation
```

```
async function creerFormation(formationData) {  
  const { nom, jour_semaine, heure_debut, heure_fin, chemins_images, description } =  
formationData;  
  
  try {  
    const result = await pool.query(  
      `INSERT INTO formation (nom, jour_semaine, heure_debut, heure_fin, chemins_images, description)  
VALUES ($1, $2, $3, $4, $5, $6)`  
    );  
  }  
}
```



```

        RETURNING *`,
        [nom, jour_semaine, heure_debut, heure_fin, chemins_images, description]
    );
    return result.rows[0];
} catch (error) {
    console.error('Erreur lors de la création de la formation:', error);
    throw error;
}
}

// Mettre à jour une formation
async function mettreAJourFormation(idFormation, formationData) {
    const { nom, jour_semaine, heure_debut, heure_fin, chemins_images, description } =
    formationData;

    try {
        const result = await pool.query(
            `UPDATE formation
             SET nom = $1, jour_semaine = $2, heure_debut = $3, heure_fin = $4, chemins_images = $5, description = $6
             WHERE id_formation = $7
             RETURNING *`,
            [nom, jour_semaine, heure_debut, heure_fin, chemins_images, description, idFormation]
        );
        return result.rows[0];
    } catch (error) {
        console.error('Erreur lors de la mise à jour de la formation:', error);
        throw error;
    }
}

// Ajouter une image à une formation
async function ajouterImageFormation(idFormation, cheminImage) {
    try {
        const result = await pool.query(
            'SELECT ajouter_image_formation($1, $2)',
            [idFormation, cheminImage]
        );
        return { success: true, message: 'Image ajoutée avec succès' };
    } catch (error) {
        console.error('Erreur lors de l\'ajout de l\'image:', error);
        throw error;
    }
}

```

```
}

// Supprimer une image d'une formation
async function supprimerImageFormation(idFormation, cheminImage) {
  try {
    const result = await pool.query(
      'SELECT supprimer_image_formation($1, $2)',
      [idFormation, cheminImage]
    );
    return { success: true, message: 'Image supprimée avec succès' };
  } catch (error) {
    console.error('Erreur lors de la suppression de l\'image:', error);
    throw error;
  }
}

// Obtenir la première image d'une formation
async function obtenirPremiereImageFormation(idFormation) {
  try {
    const result = await pool.query(
      'SELECT obtenir_premiere_image_formation($1) as premiere_image',
      [idFormation]
    );
    return result.rows[0].premiere_image;
  } catch (error) {
    console.error('Erreur lors de la récupération de la première image:', error);
    throw error;
  }
}

export default {
  obtenirToutesFormations,
  obtenirFormationParId,
  creerFormation,
  mettreAJourFormation,
  ajouterImageFormation,
  supprimerImageFormation,
  obtenirPremiereImageFormation
};
```

4. Modifier Backend/routes/formations.js

javascript

```
// Backend/routes/formations.js
import express from 'express';
import formationController from '../controllers/formationController.js';

const router = express.Router();

// Routes pour Les formations
router.get('/formations', async (req, res) => {
  try {
    const formations = await formationController.obtenirToutesFormations();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/formations/:id', async (req, res) => {
  try {
    const formation = await formationController.obtenirFormationParId(req.params.id);
    if (!formation) {
      return res.status(404).json({ error: 'Formation non trouvée' });
    }
    res.json(formation);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

export default router;
```

5. Créer Backend/package.json

json

```
{
  "name": "backend",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
```

```
  },  
  "dependencies": {  
    "express": "^4.18.2",  
    "cors": "^2.8.5",  
    "pg": "^8.11.3"  
  }  
}
```

6. Installer les dépendances

```
bash
```

```
cd Backend  
npm install
```

7. Démarrer le serveur

```
bash
```

```
cd Backend  
npm start
```

Maintenant testez : <http://localhost:3000/api/formations>

Si vous avez les autres contrôleurs (`formationInstanceController.js` , `reservationController.js` , `emailer.js`), vous devrez aussi les convertir en ES modules de la même manière.

```
{"error":"connect ECONNREFUSED 127.0.0.1:5432"}
```

L'erreur indique que Node.js essaie de se connecter à PostgreSQL sur le port 5432, mais votre base de données est sur le port 5431. Voici les corrections :

1. Créer/modifier Backend/.env

```
env
```

```
DB_USER=postgres  
DB_HOST=localhost
```

```
DB_NAME=votre_nom_de_base
DB_PASSWORD=votre_mot_de_passe
DB_PORT=5431
PORT=3000
```

2. Vérifier Backend/db.js

javascript

```
// Backend/db.js
import pg from 'pg';
const { Pool } = pg;
import dotenv from 'dotenv';

dotenv.config();

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'votre_base',
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',
  port: process.env.DB_PORT || 5431, // ✅ FORCER Le port 5431
});

export default pool;
```

3. Tester la connexion à la base manuellement

Créez Backend/test-db.js :

javascript

```
// Backend/test-db.js
import pg from 'pg';
const { Pool } = pg;
import dotenv from 'dotenv';

dotenv.config();

const pool = new Pool({
  user: process.env.DB_USER,
```

```

    host: process.env.DB_HOST,
    database: process.env.DB_NAME,
    password: process.env.DB_PASSWORD,
    port: process.env.DB_PORT || 5431,
  });

  async function testConnection() {
    try {
      console.log('🔍 Test de connexion à PostgreSQL...');
      console.log('Paramètres:', {
        host: process.env.DB_HOST,
        port: process.env.DB_PORT,
        database: process.env.DB_NAME,
        user: process.env.DB_USER
      });

      const result = await pool.query('SELECT NOW()');
      console.log('✅ Connexion BD réussie:', result.rows[0]);

      const formations = await pool.query('SELECT COUNT(*) FROM formation');
      console.log('✅ Nombre de formations:', formations.rows[0].count);

      await pool.end();
    } catch (error) {
      console.error('❌ Erreur connexion BD:', error.message);
    }
  }

  testConnection();

```

Exécutez : `cd Backend && node test-db.js`

4. Vérifier que PostgreSQL écoute sur le port 5431

bash

```

# Vérifier les processus PostgreSQL
ps aux | grep postgres

```

```

# Vérifier les ports écoutés
sudo netstat -tulpn | grep 543

```

```
# Ou avec ss  
sudo ss -tulpn | grep 543
```

5. Si PostgreSQL n'écoute pas sur 5431

Option A : Démarrer PostgreSQL sur le port 5431

```
bash  
  
# Redémarrer PostgreSQL sur le port 5431  
sudo systemctl restart postgresql
```

Option B : Modifier la configuration PostgreSQL

```
bash  
  
# Éditer le fichier de configuration  
sudo nano /etc/postgresql/*/main/postgresql.conf  
  
# Chercher la ligne port et modifier :  
port = 5431  
  
# Redémarrer PostgreSQL  
sudo systemctl restart postgresql
```

6. Vérifier les informations de connexion

Assurez-vous que dans votre `.env` vous avez les bonnes valeurs :

- **DB_USER** : généralement `postgres`
- **DB_PASSWORD** : votre mot de passe PostgreSQL
- **DB_NAME** : le nom de votre base de données
- **DB_PORT** : `5431`

7. Test final

Une fois la connexion réussie avec `test-db.js`, redémarrez le serveur :

```
bash
```

```
cd Backend
```

```
npm start
```

Et testez : `http://localhost:3000/api/formations`

Qu'est-ce que le test `test-db.js` affiche ?

```
wizcolas@Purple-
Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$ node test-
db.js
[dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ specify
custom .env file path with { path: '/custom/path/.env' }
🔥 Test de connexion à PostgreSQL...
Paramètres: {
  host: undefined,
  port: undefined,
  database: undefined,
  user: undefined
}
❌ Erreur connexion BD: SASL: SCRAM-SERVER-FIRST-MESSAGE:
client password must be a string
^C
wizcolas@Purple-
Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$
```

Le problème est que le fichier `.env` n'est pas chargé correctement. Voici la solution :

1. Créer le fichier Backend/.env avec le bon chemin

```
bash
```

```
cd Backend
```

```
nano .env
```

2. Contenu du fichier .env

env

```
DB_USER=postgres
DB_HOST=localhost
DB_NAME=softghost
DB_PASSWORD=votre_mot_de_passe_postgres
DB_PORT=5431
PORT=3000
```

Remplacez `votre_mot_de_passe_postgres` par votre vrai mot de PostgreSQL

3. Modifier Backend/db.js pour charger dotenv correctement

javascript

```
// Backend/db.js
import pg from 'pg';
const { Pool } = pg;
import dotenv from 'dotenv';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Charger le .env depuis le dossier Backend
dotenv.config({ path: path.join(__dirname, '.env') });

console.log('🔧 Configuration DB:', {
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER
});

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: parseInt(process.env.DB_PORT) || 5431,
```

```
});
```

```
export default pool;
```

4. Vérifier que le fichier `.env` existe

```
bash
```

```
cd Backend
```

```
ls -la | grep .env
```

Vous devriez voir `.env` dans la liste.

5. Tester à nouveau

```
bash
```

```
cd Backend
```

```
node test-db.js
```

6. Si ça ne marche toujours pas, testez avec les valeurs en dur

Modifiez temporairement `Backend/db.js` :

```
javascript
```

```
// Backend/db.js (version temporaire pour test)
```

```
import pg from 'pg';
```

```
const { Pool } = pg;
```

```
const pool = new Pool({
```

```
  user: 'postgres',
```

```
  host: 'localhost',
```

```
  database: 'softghost', // ou Le nom de votre base
```

```
  password: 'VOTRE_MOT_DE_PASSE', // remplacez par votre vrai mot de passe
```

```
  port: 5431,
```

```
});
```

```
export default pool;
```

7. Vérifier les informations de votre base PostgreSQL

Connectez-vous à PostgreSQL pour vérifier :

```
bash

# Se connecter à PostgreSQL sur le port 5431
psql -U postgres -p 5431 -h localhost

# Dans psql, lister les bases de données
\l

# Vérifier les tables dans votre base
\c softghost
\dt
```

Quel est le mot de passe de votre utilisateur PostgreSQL ? C'est probablement le problème principal.

C'est beau ça marche mais lorsque je fais npm run dev. Il ne m'affiche pas la liste des formations. Cependant lorsque je vais à l'adresse <http://localhost:3000/api/formations> lorsque je fais npm start dans le dossier backend il m'affiche le JSON avec tous les détails.

Parfait ! L'API fonctionne donc le backend est OK. Le problème est dans le frontend. Voici les corrections :

1. Vérifier que src/App.jsx appelle correctement l'API

```
javascript

// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import './App.css';
```

```
function App() {
  const [formations, setFormations] = useState({});
  const [loading, setLoading] = useState(true);

  // Récupérer Les formations depuis L'API
  useEffect(() => {
    const fetchFormations = async () => {
      try {
        console.log('🔄 Chargement des formations...');
        const response = await fetch('http://localhost:3000/api/formations');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('✅ Formations chargées:', data);
        setFormations(data);
      } catch (error) {
        console.error('❌ Erreur chargement formations:', error);
        // Données de test en cas d'erreur
        setFormations({
          "formation1": {
            titre: "Formation Test",
            images: [],
            description: "Formation de test"
          }
        });
      } finally {
        setLoading(false);
      }
    };

    fetchFormations();
  }, []);

  if (loading) {
    return (
      <div className="loading-container">
        <p>Chargement des formations...</p>
      </div>
    );
  }
}
```

```
    return (  
      <AccueilFormations  
        formations={formations}  
        onFormationSelect={(formationId) => {  
          console.log('Formation sélectionnée:', formationId);  
        }}  
      />  
    );  
  }  
  
  export default App;
```

2. Vérifier la console du navigateur

Ouvrez les Developer Tools (F12) et regardez dans l'onglet **Console** et **Network** :

- Y a-t-il des erreurs CORS ?
- La requête vers `http://localhost:3000/api/formations` fonctionne-t-elle ?

3. Vérifier que les deux serveurs tournent

Terminal 1 - Backend:

```
bash  
  
cd Backend  
npm start
```

Terminal 2 - Frontend:

```
bash  
  
npm run dev
```

4. Vérifier l'URL dans le navigateur

Votre frontend devrait être sur un port différent, par exemple `http://localhost:5173` (Vite)

5. Si CORS bloque, modifier Backend/server.js

javascript

```
// Backend/server.js
import express from 'express';
import cors from 'cors';
import path from 'path';
import { fileURLToPath } from 'url';
import formationRoutes from './routes/formations.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT || 3000;

// CORS plus permissif pour le développement
app.use(cors({
  origin: ['http://localhost:5173', 'http://localhost:3000'], // Ports Vite + Backend
  credentials: true
}));

app.use(express.json());
app.use('/images', express.static(path.join(__dirname, '../src/assets/images')));

// Routes
app.use('/api', formationRoutes);

app.listen(PORT, () => {
  console.log(`✅ Backend démarré sur http://localhost:${PORT}`);
});
```

6. Test de débogage

Ajoutez un console.log dans AccueilFormations.jsx :

javascript

```
// Dans AccueilFormations.jsx
console.log('🖼️ Formations reçues:', formations);
```

Qu'est-ce qui s'affiche dans la console du navigateur ?

- Si vous voyez les données : le problème est d'affichage
- Si vous voyez un objet vide : le problème est dans App.jsx
- Si vous voyez une erreur : le problème est CORS ou réseau

Quel port utilise votre frontend (`npm run dev`) ?

Lorsque le backend et le frontend roulent ensemble et que je vais à l'adresse `http://localhost:5173`, cela m'affiche une page complètement grise

Le problème est que votre frontend n'affiche rien (page grise). Voici le dépannage étape par étape :

1. Vérifier la console du navigateur

Ouvrez les Developer Tools (F12) et regardez les erreurs dans l'onglet **Console**.

2. Vérifier le fichier `src/main.jsx` ou `src/main.js`

Assurez-vous qu'il monte correctement l'application :

```
javascript
```

```
// src/main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

3. Vérifier le fichier `index.html`

Assurez-vous qu'il y a bien un élément avec id="root" :

html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>SoftGhost</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

4. Vérifier que AccueilFormations.css existe

Si le fichier CSS est manquant, ça peut causer une page blanche/grise.

5. Test minimal - Modifier temporairement App.jsx

Remplacez App.jsx par ce code minimal pour tester :

javascript

```
// src/App.jsx
import React from 'react';
import './App.css';

function App() {
  return (
    <div style={{ padding: '20px', background: 'white' }}>
      <h1>Test - L'application fonctionne</h1>
      <p>Si vous voyez ce texte, React fonctionne.</p>
      <button onClick={() => alert('Test réussi!')}>
        Cliquez-moi
      </button>
    </div>
```



```
);  
}
```

```
export default App;
```

6. Si le test minimal fonctionne, vérifiez `AccueilFormations.jsx`

Le problème peut venir du composant lui-même. Vérifiez :

- Que le fichier `AccueilFormations.css` existe
- Qu'il n'y a pas d'erreurs de syntaxe
- Que tous les imports sont corrects

7. Vérifier les logs du terminal frontend

Quand vous faites `npm run dev`, voyez-vous des erreurs de compilation ?

Quelles erreurs voyez-vous dans la console du navigateur ? Cela nous indiquera exactement où est le problème.

Le test minimal a fonctionné mais lorsque je reviens à la fonction initiale cela m'affiche encore une page grise :
voici le contenu du fichier `components/AccueilFormations.jsx` :

```
// AccueilFormations.jsx  
import React, { useState } from 'react';  
import './AccueilFormations.css';  
  
const AccueilFormations = ({  
  formations = {  
    "formation-1": {  
      titre: "Formation React Avancé",  
      images: ["/images/react1.jpg", "/images/react2.jpg"]  
    },  
    "formation-2": {  
      titre: "Formation JavaScript Moderne",  
      images: ["/images/js1.jpg", "/images/js2.jpg"]  
    },  
  },  
}) => {  
  // ...  
}
```

```
"formation-3": {
  titre: "Formation Node.js & Express",
  images: ["/images/node1.jpg", "/images/node2.jpg"]
},
onFormationSelect = (formationId) => {
  console.log('Formation sélectionnée:', formationId);
}
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: '',
    password: ''
  });

  const handleFormationClick = (formationId) => {
    if (onFormationSelect) {
      onFormationSelect(formationId);
    }
  };

  const handleAdminLogin = (e) => {
    e.preventDefault();
    // Simulation de connexion admin
    console.log('Tentative de connexion admin:', adminCredentials);
    if (adminCredentials.username && adminCredentials.password) {
      alert('Connexion administrateur réussie !');
      setShowAdminLogin(false);
      setAdminCredentials({ username: '', password: '' });
    } else {
      alert('Veuillez entrer un nom d\'utilisateur et un mot de passe');
    }
  };

  const handleAdminInputChange = (e) => {
    const { name, value } = e.target;
    setAdminCredentials(prev => ({
      ...prev,
      [name]: value
    }));
  };
}
```

```

};

return (
  <div className="accueil-container">
    {/* Header avec bouton admin */}
    <header className="accueil-header">
      <div className="header-content">
        <h1 className="accueil-title">Formations en Ligne</h1>
        <button
          className="admin-button"
          onClick={() => setShowAdminLogin(true)}
        >
           Administrateur
        </button>
      </div>
    </header>

    {/* Contenu principal */}
    <main className="accueil-main">
      <div className="formations-grid">
        {Object.entries(formations).map(([formationId, formation]) =>
          (
            <div
              key={formationId}
              className="formation-card"
              onClick={() => handleFormationClick(formationId)}
            >
              <div className="formation-image-container">
                <img
                  src={formation.images[0]}
                  alt={formation.titre}
                  className="formation-image"
                  onError={(e) => {
                    e.target.src =
`https://via.placeholder.com/300x200/667eea/ffffff?
text=${encodeURIComponent(formation.titre)}`;
                  }}
                />
              </div>
              <div className="formation-info">

```

```

    <h3 className="formation-titre">{formation.titre}</h3>
    <div className="formation-overlay">
      <span className="voir-details">Voir les détails
→</span>
    </div>
  </div>
</div>
  )}}
</div>

```

```

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

```

```

{ /* Modal de connexion admin */ }
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">
        <h2>Connexion Administrateur</h2>
        <button
          className="close-button"
          onClick={() => setShowAdminLogin(false)}
        >
          ×
        </button>
      </div>

      <form className="admin-form" onSubmit=
{handleAdminLogin}>
        <div className="form-group">
          <label htmlFor="username">Nom d'utilisateur</label>
          <input
            type="text"
            id="username"
            name="username"
            value={adminCredentials.username}

```

```
      onChange={handleAdminInputChange}
      placeholder="Entrez votre nom d'utilisateur"
      required
    />
  </div>

  <div className="form-group">
    <label htmlFor="password">Mot de passe</label>
    <input
      type="password"
      id="password"
      name="password"
      value={adminCredentials.password}
      onChange={handleAdminInputChange}
      placeholder="Entrez votre mot de passe"
      required
    />
  </div>

  <div className="admin-form-buttons">
    <button type="submit" className="login-button">
      Se connecter
    </button>
    <button
      type="button"
      className="cancel-button"
      onClick={() => setShowAdminLogin(false)}
    >
      Annuler
    </button>
  </div>
</form>
</div>
</div>
  )}
</div>
);
};
```

```
export default AccueilFormations;
```

et voici le contenu du fichier components/AccueilFormations.css :

```
/* AccueilFormations.css */
.accueil-container {
  min-height: 100vh;
  min-width: 100vw;
  background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  margin: 0;
  padding: 0;
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  overflow-y: auto;
}

.accueil-header {
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(10px);
  padding: 20px 0;
  box-shadow: 0 2px 20px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 100;
  width: 100%;
}

.header-content {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
.accueil-title {
  color: #333;
  font-size: 2.5em;
  font-weight: 700;
  margin: 0;
  background: linear-gradient(135deg, #667eea, #764ba2);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.admin-button {
  background: linear-gradient(135deg, #ff6b6b, #ee5a24);
  color: white;
  border: none;
  padding: 12px 24px;
  border-radius: 25px;
  font-size: 1em;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
  box-shadow: 0 4px 15px rgba(255, 107, 107, 0.3);
}

.admin-button:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(255, 107, 107, 0.4);
}

.accueil-main {
  max-width: 1200px;
  margin: 0 auto;
  padding: 40px 20px;
  min-height: calc(100vh - 80px);
  box-sizing: border-box;
}

.formations-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
```

```
gap: 30px;
margin-top: 40px;
}

.formation-card {
  background: white;
  border-radius: 20px;
  overflow: hidden;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
  cursor: pointer;
  transition: all 0.3s ease;
  position: relative;
}

.formation-card:hover {
  transform: translateY(-10px);
  box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
}

.formation-image-container {
  width: 100%;
  height: 200px;
  overflow: hidden;
  position: relative;
}

.formation-image {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: transform 0.3s ease;
}

.formation-card:hover .formation-image {
  transform: scale(1.05);
}

.formation-info {
  padding: 25px;
  position: relative;
```



```
}

.formation-titre {
  color: #333;
  font-size: 1.4em;
  font-weight: 600;
  margin: 0 0 15px 0;
  line-height: 1.3;
}

.formation-overlay {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: linear-gradient(135deg, rgba(102, 126, 234, 0.9),
    rgba(118, 75, 162, 0.9));
  display: flex;
  align-items: center;
  justify-content: center;
  opacity: 0;
  transition: opacity 0.3s ease;
}

.formation-card:hover .formation-overlay {
  opacity: 1;
}

.voir-details {
  color: white;
  font-size: 1.2em;
  font-weight: 600;
  text-align: center;
}

.no-formations {
  text-align: center;
  padding: 60px 20px;
  color: white;
}
```

```
font-size: 1.2em;
height: 50vh;
display: flex;
align-items: center;
justify-content: center;
}

/* Modal de connexion admin */
.admin-modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.7);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
  padding: 20px;
}

.admin-modal {
  background: white;
  border-radius: 20px;
  padding: 40px;
  max-width: 450px;
  width: 100%;
  box-shadow: 0 25px 50px rgba(0, 0, 0, 0.3);
  animation: modalAppear 0.3s ease;
}

@keyframes modalAppear {
  from {
    opacity: 0;
    transform: scale(0.9) translateY(-20px);
  }
  to {
    opacity: 1;
    transform: scale(1) translateY(0);
  }
}
```

```
}  
}
```

```
.admin-modal-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 30px;  
}
```

```
.admin-modal-header h2 {  
  color: #333;  
  margin: 0;  
  font-size: 1.8em;  
}
```

```
.close-button {  
  background: none;  
  border: none;  
  font-size: 2em;  
  cursor: pointer;  
  color: #666;  
  padding: 0;  
  width: 40px;  
  height: 40px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 50%;  
  transition: all 0.3s ease;  
}
```

```
.close-button:hover {  
  background: #f8f9fa;  
  color: #333;  
}
```

```
.admin-form .form-group {  
  margin-bottom: 20px;  
}
```

```
.admin-form label {  
  display: block;  
  margin-bottom: 8px;  
  color: #333;  
  font-weight: 600;  
}
```

```
.admin-form input {  
  width: 100%;  
  padding: 15px;  
  border: 2px solid #e9ecef;  
  border-radius: 8px;  
  font-size: 1em;  
  transition: all 0.3s ease;  
  box-sizing: border-box;  
}
```

```
.admin-form input:focus {  
  outline: none;  
  border-color: #667eea;  
  box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);  
}
```

```
.admin-form-buttons {  
  display: flex;  
  gap: 15px;  
  margin-top: 30px;  
}
```

```
.login-button {  
  flex: 2;  
  background: linear-gradient(135deg, #4CAF50, #45a049);  
  color: white;  
  border: none;  
  padding: 15px;  
  border-radius: 8px;  
  font-size: 1.1em;  
  font-weight: 600;  
  cursor: pointer;
```

```
    transition: all 0.3s ease;
  }

.login-button:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(76, 175, 80, 0.4);
}

.admin-form-buttons .cancel-button {
  flex: 1;
  background: #6c757d;
  color: white;
  border: none;
  padding: 15px;
  border-radius: 8px;
  font-size: 1em;
  cursor: pointer;
  transition: all 0.3s ease;
}

.admin-form-buttons .cancel-button:hover {
  background: #5a6268;
  transform: translateY(-2px);
}

/* Personnalisation de la barre de scroll */
.accueil-container::-webkit-scrollbar {
  width: 12px;
}

.accueil-container::-webkit-scrollbar-track {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.accueil-container::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.3);
  border-radius: 6px;
  border: 2px solid #667eea;
}
```

```
.accueil-container::-webkit-scrollbar-thumb:hover {  
  background: rgba(255, 255, 255, 0.5);  
}  
  
/* Pour Firefox */  
.accueil-container {  
  scrollbar-width: thin;  
  scrollbar-color: rgba(255, 255, 255, 0.3) #667eea;  
}  
  
/* Responsive */  
@media (max-width: 768px) {  
  .header-content {  
    flex-direction: column;  
    gap: 15px;  
    text-align: center;  
  }  
  
  .accueil-title {  
    font-size: 2em;  
  }  
  
  .formations-grid {  
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));  
    gap: 20px;  
  }  
  
  .admin-modal {  
    padding: 30px 20px;  
    margin: 10px;  
  }  
  
  .admin-form-buttons {  
    flex-direction: column;  
  }  
  
  .accueil-main {  
    padding: 30px 15px;  
    min-height: calc(100vh - 70px);  
  }  
}
```

```
}  
}  
  
@media (max-width: 480px) {  
  .accueil-title {  
    font-size: 1.7em;  
  }  
  
  .formations-grid {  
    grid-template-columns: 1fr;  
  }  
  
  .formation-info {  
    padding: 20px;  
  }  
  
  .formation-titre {  
    font-size: 1.2em;  
  }  
  
  .accueil-main {  
    padding: 20px 10px;  
  }  
}
```

je ne veux pas que tu modifies le design des pages s'il te plaît je veux qu'elle ressemble exactement à la même chose

Le problème vient du CSS ! Votre composant utilise `position: fixed` et `min-width: 100vw` ce qui peut causer une page grise. Voici les corrections **uniquement sur le CSS** sans changer le design :

Modifier `components/AccueilFormations.css`

Changez seulement ces lignes :

CSS

```
/* AccueilFormations.css */
.accueil-container {
  min-height: 100vh;
  /* SUPPRIMER: min-width: 100vw; */
  /* SUPPRIMER: position: fixed; */
  /* SUPPRIMER: top: 0; */
  /* SUPPRIMER: left: 0; */
  /* SUPPRIMER: right: 0; */
  /* SUPPRIMER: bottom: 0; */
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  margin: 0;
  padding: 0;
  overflow-y: auto; /* Garder cette ligne */
}

.accueil-header {
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(10px);
  padding: 20px 0;
  box-shadow: 0 2px 20px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 100;
  width: 100%;
  /* AJOUTER: */
  margin: 0;
}

.accueil-main {
  max-width: 1200px;
  margin: 0 auto;
  padding: 40px 20px;
  min-height: calc(100vh - 80px);
  box-sizing: border-box;
  /* AJOUTER: */
  width: 100%;
}
```

Seulement ces modifications - supprimez les propriétés problématiques qui empêchent l'affichage. Le design restera **exactement le même** mais fonctionnera correctement.

Alternative plus simple

Si vous voulez être sûr de ne rien casser, remplacez **seulement** la classe

`.accueil-container :`

CSS

```
.accueil-container {  
  min-height: 100vh;  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  margin: 0;  
  padding: 0;  
}
```

Le problème était que `position: fixed + min-width: 100vw + top/left/right/bottom: 0` créait un conteneur qui couvrait tout mais empêchait le rendu correct.

Testez maintenant - votre page devrait s'afficher avec le même design mais sans la page grise !

Je ne pense pas que le problème vient du CSS parce que lorsque je roule le frontend sans le backend l'interface graphique est affichée parfaitement à l'exception qu'il n'y a aucune formation d'afficher

Vous avez raison ! Le problème vient des données. Lorsque le backend est démarré, votre `App.jsx` envoie un objet **vide** ou **avec un format différent** à `AccueilFormations`

.

1. Vérifiez le format des données de l'API

Allez à `http://localhost:3000/api/formations` et regardez le format exact. Comparez avec ce que attend `AccueilFormations` :

Format attendu par `AccueilFormations` :

javascript

```

{
  "formation-1": {
    titre: "Formation React Avancé",
    images: ["/images/react1.jpg", "/images/react2.jpg"]
  },
  "formation-2": {
    titre: "Formation JavaScript Moderne",
    images: ["/images/js1.jpg", "/images/js2.jpg"]
  }
}

```

2. Ajoutez du débogage dans App.jsx

Modifiez temporairement App.jsx :

javascript

```

// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import './App.css';

function App() {
  const [formations, setFormations] = useState({});

  useEffect(() => {
    const fetchFormations = async () => {
      try {
        console.log('🔄 Chargement des formations...');
        const response = await fetch('http://localhost:3000/api/formations');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('✅ Données reçues de l API:', data);
        console.log('📁 Type:', typeof data);
        console.log('🔑 Nombre de clés:', Object.keys(data).length);
        console.log('🔍 Première clé:', Object.keys(data)[0]);
        console.log('📄 Première formation:', data[Object.keys(data)[0]]);
      }
    };
  });
}

```

```

        setFormations(data);
    } catch (error) {
        console.error('✖ Erreur chargement formations:', error);
    }
};

fetchFormations();
}, []);

return (
    <AccueilFormations
        formations={formations}
        onFormationSelect={({formationId}) => {
            console.log('Formation sélectionnée:', formationId);
        }}
    />
);
}

export default App;

```

3. Vérifiez la console du navigateur

Ouvrez F12 → Console et regardez ce qui s'affiche. Les logs vous diront :

- Si les données sont chargées
- Le format exact des données
- Si c'est un tableau au lieu d'un objet

4. Problème probable : format de l'API

Si votre API retourne un **tableau** au lieu d'un **objet**, modifiez App.jsx :

```

javascript

// Si l'API retourne un tableau, convertissez-le en objet
const data = await response.json();

// Si data est un tableau :
if (Array.isArray(data)) {
    const formationsObject = {};

```

```

data.forEach(formation => {
  formationsObject[formation.id_formation] = {
    titre: formation.nom,
    images: formation.chemins_images || []
  };
});
setFormations(formationsObject);
} else {
  setFormations(data);
}

```

Qu'est-ce que vous voyez dans la console du navigateur quand le backend tourne ? Cela nous dira exactement quel est le format des données.

Je pense savoir d'où vient le problème :

Premièrement le format de données que retourne l'api est ce dernier :

```

[{"id_formation":"formation9","nom":"Formation Bitwarden - Gestion des Mots de Passe","jour_semaine":4,"heure_debut":"13:00:00","heure_fin":"16:00:00","chemins_images":["Bitwarden/bitwarden1.png","Bitwarden/birwarden2.png","Bitwarden/bitwarden3.png","Bitwarden/bitwarden4.png"],"description":"Prenez le contrôle de votre sécurité en ligne ! Bitwarden est un gestionnaire de mots de passe open-source. Apprenez à générer des mots de passe forts, à synchroniser vos identifiants entre appareils et à protéger vos comptes contre les cybermenaces."}, {"id_formation":"formation1","nom":"Formation Blender 3D","jour_semaine":1,"heure_debut":"14:00:00","heure_fin":"17:00:00","chemins_images":["Blender/blender1.png","Blender/blender2.jpg","Blender/blender3.jpg","Blender/blender4.png"],"description":"Découvrez la magie de la création 3D ! Blender est un logiciel gratuit et puissant pour modéliser, animer et rendre vos idées en trois dimensions. Apprenez à créer des objets, personnages et environnements virtuels, à maîtriser l'éclairage et les textures, et à produire des animations époustouflantes. Parfait pour les artistes, designers et passionnés d'animation."}, {"id_formation":"formation4","nom":"Formation DaVinci Resolve - Post-

```

production","jour_semaine":4,"heure_debut":"16:00:00","heure_fin":"19:00:00","chemins_images":
 ["Davinci_resolve/davinci1.webp","Davinci_resolve/davinci2.jpg","Davinci_resolve/davinci3.jpg","Davinci_resolve/davinci4.jpg"],"description":"Passez au niveau professionnel du montage vidéo ! DaVinci Resolve combine montage, correction colorimétrique, effets visuels et post-production audio dans un seul outil. Apprenez les techniques utilisées par les studios hollywoodiens pour donner vie à vos projets vidéo."},{ "id_formation":"formation13","nom":"Formation Draw.io - Création de Diagrammes","jour_semaine":1,"heure_debut":"10:00:00","heure_fin":"13:00:00","chemins_images":
 ["Draw.io/draw1.png","Draw.io/draw2.jpg","Draw.io/draw3.jpg","Draw.io/draw4.png"],"description":"Visualisez vos idées complexes ! Draw.io est un outil de diagramming gratuit et intuitif. Apprenez à créer des organigrammes, plans architecturaux, réseaux informatiques et schémas techniques pour communiquer efficacement."},{ "id_formation":"formation14","nom":"Formation Freemocap - Capture de Mouvement","jour_semaine":2,"heure_debut":"10:00:00","heure_fin":"13:00:00","chemins_images":
 ["Freemocap/freemocap1.jpg","Freemocap/freemocap2.jpg","Freemocap/freemocap3.jpg","Freemocap/freemocap4.png"],"description":"Capturez le mouvement humain facilement ! Freemocap transforme des caméras standards en système de motion capture. Apprenez à enregistrer des mouvements, analyser la biomécanique et créer des animations réalistes sans équipement coûteux."},{ "id_formation":"formation15","nom":"Formation Ibis Paint - Dessin sur Mobile","jour_semaine":6,"heure_debut":"13:00:00","heure_fin":"16:00:00","chemins_images":
 ["Ibis_paint/ibispaint1.jpg","Ibis_paint/ibispaint2.jpg","Ibis_paint/ibispaint3.jpg","Ibis_paint/ibispaint4.jpg"],"description":"Peignez sur mobile comme un pro ! Ibis Paint est une application de dessin complète pour smartphones et tablettes. Maîtrisez les outils de dessin digital, les calques, les filtres et créez des œuvres d'art où que vous soyez."},{ "id_formation":"formation12","nom":"Formation Inkscape - Dessin Vectoriel","jour_semaine":0,"heure_debut":"14:00:00","heure_fin":"17:00:00","chemins_images":

["Inkscape/inkscape1.png","Inkscape/inkscape2.webp","Inkscape/inkscape3.png","Inkscape/inkscape4.jpg"],"description":"Créez des graphiques vectoriels professionnels ! Inkscape est l'alternative libre à Illustrator. Maîtrisez le dessin vectoriel, la manipulation des chemins, la typographie et la création de logos, icônes et illustrations évolutives."}, {"id_formation":"formation3","nom":"Formation Kdenlive - Montage Vidéo","jour_semaine":3,"heure_debut":"19:00:00","heure_fin":"22:00:00","chemins_images":["Kdenlive/kdenlive1.webp","Kdenlive/kdenlive2.png","Kdenlive/kdenlive3.png","Kdenlive/kdenlive4.jpg"],"description":"Devenez monteur vidéo ! Kdenlive est un logiciel de montage vidéo gratuit et complet. Maîtrisez le découpage, les transitions, les effets visuels, le traitement audio et l'export de vidéos de qualité professionnelle. Parfait pour les YouTubeurs, vidéastes amateurs et créateurs de contenu."}, {"id_formation":"formation2","nom":"Formation Krita - Dessin Digital","jour_semaine":2,"heure_debut":"18:00:00","heure_fin":"21:00:00","chemins_images":["Krita/krita1.avif","Krita/krita2.webp","Krita/krita3.png","Krita/krita4.webp"],"description":"Libérez votre créativité numérique ! Krita est un studio de peinture numérique open-source conçu pour les artistes. Apprenez les techniques de dessin digital, la manipulation des calques, les pinceaux personnalisés et la colorisation professionnelle. Idéal pour l'illustration, le concept art et la bande dessinée."}, {"id_formation":"formation8","nom":"Formation Linux Ubuntu - Système d'Exploitation","jour_semaine":3,"heure_debut":"10:00:00","heure_fin":"13:00:00","chemins_images":["Ubuntu/ubuntu1.jpg","Ubuntu/ubuntu2.webp","Ubuntu/ubuntu3.jpg","Ubuntu/ubuntu4.png"],"description":"Découvrez un nouveau monde informatique ! Ubuntu est un système d'exploitation libre, sécurisé et performant. Apprenez les bases de Linux, la gestion des fichiers en terminal, l'installation de logiciels et les avantages d'un système open-source pour votre quotidien numérique."}, {"id_formation":"formation10","nom":"Formation Notion - Organisation et Productivité","jour_semaine":5,"heure_debut":"10:00:00","heure_fin":"13:00:00","chemins_images":["Notion/notion1.png","Notion/notion2.png","Notion/notion3.jpg","Notion/notion4.png"],"description":"Maîtrisez l'art de l'organisation et de la productivité avec Notion ! Notion est un outil tout-en-un pour gérer vos tâches, vos projets et vos connaissances. Apprenez à structurer vos idées, à collaborer efficacement et à automatiser vos workflows. Parfait pour les entrepreneurs, les étudiants et les professionnels."}]

ion/notion4.avif"],"description":"Centralisez votre vie numérique ! Notion est un espace de travail tout-en-un pour notes, bases de données, planning et collaboration. Apprenez à créer des pages personnalisées, des systèmes d'organisation et à automatiser vos workflows."},{ "id_formation":"formation6","nom":"Formation Obsidian - Organisation des Connaissances","jour_semaine":1,"heure_debut":"19:00:00","heure_fin":"22:00:00","chemins_images":["Obsidian/obsidian1.png","Obsidian/obsidian2.jpg","Obsidian/obsidian3.png","Obsidian/obsidian4.png"],"description":"Organisez vos pensées comme jamais ! Obsidian est un carnet de notes puissant qui vous aide à connecter vos idées. Apprenez à créer une base de connaissances personnelle, à établir des liens entre vos notes et à booster votre productivité grâce au zettelkasten."},{ "id_formation":"formation5","nom":"Formation Ollama - Intelligence Artificielle","jour_semaine":0,"heure_debut":"09:00:00","heure_fin":"12:00:00","chemins_images":["Ollama/ollama1.png","Ollama/ollama2.png","Ollama/ollama3.png","Ollama/ollama4.png"],"description":"Explorez l'intelligence artificielle locale ! Ollama vous permet d'exécuter des modèles de langage avancés directement sur votre ordinateur. Découvrez comment interagir avec l'IA, personnaliser les modèles et développer des applications intelligentes sans dépendre d'internet."},{ "id_formation":"formation7","nom":"Formation ONLY Office - Bureautique Collaborative","jour_semaine":2,"heure_debut":"14:00:00","heure_fin":"17:00:00","chemins_images":["Onlyoffice/onlyoffice1.png","Onlyoffice/onlyoffice2.png","Onlyoffice/onlyoffice3.png","Onlyoffice/onlyoffice4.png"],"description":"Maîtrisez la suite bureautique collaborative ! ONLY Office intègre traitement de texte, tableur et présentation dans une interface moderne. Apprenez à créer des documents professionnels, à travailler en équipe en temps réel et à gérer vos projets efficacement."},{ "id_formation":"formation11","nom":"Formation SlimeVR - Réalité Virtuelle Corporelle","jour_semaine":5,"heure_debut":"14:00:00","heure_fin":"17:00:00","chemins_images":["SlimeVR/slimevr1.webp","SlimeVR/slimevr2.jpg","SlimeVR/slimevr3.webp","SlimeVR/slimevr4.jpg"],"description":"Plongez dans la réalité virtuelle corporelle ! SlimeVR est un système de tracking corporel

DIY et abordable. Apprenez à configurer des capteurs, à calibrer vos mouvements et à vivre une immersion VR complète pour le gaming et les applications sociales."}]

il faudrait l'adapter aux données d'entrée que prend la page d'accueil

la 2e chose est que les images sont situées à un endroit très précis du système de fichiers :

voici l'architecture du dossier src :

wizcolas@Purple-Neon:/mnt/c/Users/nicol/Desktop/softghost/src\$ tree

```
.
├── App.css
├── App.jsx
├── assets
│   ├── images
│   │   ├── Bitwarden
│   │   │   ├── birwarden2.png
│   │   │   ├── bitwarden1.png
│   │   │   ├── bitwarden3.png
│   │   │   └── bitwarden4.png
│   │   ├── Blender
│   │   │   ├── blender1.png
│   │   │   ├── blender2.jpg
│   │   │   ├── blender3.jpg
│   │   │   └── blender4.png
│   │   ├── Davinci_resolve
│   │   │   ├── davinci1.webp
│   │   │   ├── davinci2.jpg
│   │   │   ├── davinci3.jpg
│   │   │   └── davinci4.jpg
│   │   ├── Draw.io
│   │   │   ├── draw1.png
│   │   │   ├── draw2.jpg
│   │   │   ├── draw3.jpg
│   │   │   └── draw4.png
│   │   └── Freemocap
```


- | | | | — freemocap1.jpg
- | | | | — freemocap2.jpg
- | | | | — freemocap3.jpg
- | | | | — freemocap4.png
- | | | — Ibis_paint
- | | | | — ibispaint1.jpg
- | | | | — ibispaint2.jpg
- | | | | — ibispaint3.jpg
- | | | | — ibispaint4.jpg
- | | | — Inkscape
- | | | | — inkscape1.png
- | | | | — inkscape2.webp
- | | | | — inkscape3.png
- | | | | — inkscape4.jpg
- | | | — Kdenlive
- | | | | — kdenlive1.webp
- | | | | — kdenlive2.png
- | | | | — kdenlive3.png
- | | | | — kdenlive4.jpg
- | | | — Krita
- | | | | — krita1.avif
- | | | | — krita2.webp
- | | | | — krita3.png
- | | | | — krita4.webp
- | | | — Notion
- | | | | — notion1.png
- | | | | — notion2.png
- | | | | — notion3.jpg
- | | | | — notion4.avif
- | | | — Obsidian
- | | | | — obsidian1.png
- | | | | — obsidian2.jpg
- | | | | — obsidian3.png
- | | | | — obsidian4.png
- | | | — Ollama
- | | | | — ollama1.png
- | | | | — ollama2.png
- | | | | — ollama3.png
- | | | | — ollama4.png
- | | | — Onlyoffice

```

| | | | — onlyoffice1.png
| | | | — onlyoffice2.png
| | | | — onlyoffice3.png
| | | | — onlyoffice4.png
| | | — SlimeVR
| | | | — slimevr1.webp
| | | | — slimevr2.jpg
| | | | — slimevr3.webp
| | | | — slimevr4.jpg
| | — Ubuntu
| | | — ubuntu1.jpg
| | | — ubuntu2.webp
| | | — ubuntu3.jpeg
| | | — ubuntu4.png
| — react.svg
— components
  — AccueilFormations.css
  — AccueilFormations.jsx
  — ConfirmationPage.css
  — ConfirmationPage.jsx
  — ConfirmationReservation.css
  — ConfirmationReservation.jsx
  — DescriptionFormation.css
  — DescriptionFormation.jsx
  — ErreurReservation.css
  — ErreurReservation.jsx
  — PageAdministrateur.css
  — PageAdministrateur.jsx
— index.css
— main.jsx

```

19 directories, 77 files

il faudrait juste modifier le fichier App.jsx pour satisfaire ces conditions

tout cela en sachant que la page d'accueil prend ses paramètres :

```

// AccueilFormations.jsx
import React, { useState } from 'react';

```

```
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"]
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: '',
    password: ''
  });

  const handleFormationClick = (formationId) => {
    if (onFormationSelect) {
      onFormationSelect(formationId);
    }
  };

  const handleAdminLogin = (e) => {
    e.preventDefault();
    // Simulation de connexion admin
    console.log('Tentative de connexion admin:', adminCredentials);
    if (adminCredentials.username && adminCredentials.password) {
      alert('Connexion administrateur réussie !');
      setShowAdminLogin(false);
    }
  };
}
```

```

    setAdminCredentials({ username: '', password: '' });
  } else {
    alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
  }
};

```

```

const handleAdminInputChange = (e) => {
  const { name, value } = e.target;
  setAdminCredentials(prev => ({
    ...prev,
    [name]: value
  }));
};

```

```

return (
  <div className="accueil-container">
    { /* Header avec bouton admin */ }
    <header className="accueil-header">
      <div className="header-content">
        <h1 className="accueil-title">Formations en Ligne</h1>
        <button
          className="admin-button"
          onClick={() => setShowAdminLogin(true)}
        >
           Administrateur
        </button>
      </div>
    </header>

    { /* Contenu principal */ }
    <main className="accueil-main">
      <div className="formations-grid">
        {Object.entries(formations).map(([formationId, formation]) =>
          (
            <div
              key={formationId}
              className="formation-card"
              onClick={() => handleFormationClick(formationId)}
            >
              <div className="formation-image-container">

```

```

    <img
      src={formation.images[0]}
      alt={formation.titre}
      className="formation-image"
      onError={(e) => {
        e.target.src =
`https://via.placeholder.com/300x200/667eea/ffffff?
text=${encodeURIComponent(formation.titre)}`;
      }}
    />
  </div>
  <div className="formation-info">
    <h3 className="formation-titre">{formation.titre}</h3>
    <div className="formation-overlay">
      <span className="voir-details">Voir les détails
→</span>
    </div>
  </div>
</div>
  )}}
</div>

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

{/* Modal de connexion admin */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">
        <h2>Connexion Administrateur</h2>
        <button
          className="close-button"
          onClick={() => setShowAdminLogin(false)}
        >
      </div>
    </div>
  </div>
)}

```

```
</button>
```

```
</div>
```

```
<form className="admin-form" onSubmit={
handleAdminLogin}>
  <div className="form-group">
    <label htmlFor="username">Nom d'utilisateur</label>
    <input
      type="text"
      id="username"
      name="username"
      value={adminCredentials.username}
      onChange={handleAdminInputChange}
      placeholder="Entrez votre nom d'utilisateur"
      required
    />
  </div>
```

```
<div className="form-group">
  <label htmlFor="password">Mot de passe</label>
  <input
    type="password"
    id="password"
    name="password"
    value={adminCredentials.password}
    onChange={handleAdminInputChange}
    placeholder="Entrez votre mot de passe"
    required
  />
</div>
```

```
<div className="admin-form-buttons">
  <button type="submit" className="login-button">
    Se connecter
  </button>
  <button
    type="button"
    className="cancel-button"
    onClick={() => setShowAdminLogin(false)}
  >
```

```

        Annuler
      </button>
    </div>
  </form>
</div>
</div>
</div>
  )}
</div>
);
};

export default AccueilFormations;

```

Parfait ! Je vois le problème. Votre API retourne un **tableau** mais `AccueilFormations` attend un **objet**. Voici le `App.jsx` corrigé :

src/App.jsx modifié

javascript

```

// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import './App.css';

function App() {
  const [formations, setFormations] = useState({});

  useEffect(() => {
    const fetchFormations = async () => {
      try {
        console.log('📡 Chargement des formations depuis l API...');
        const response = await fetch('http://localhost:3000/api/formations');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('✅ Données brutes reçues:', data);
      } catch (error) {
        console.error('Erreur lors du chargement des formations:', error);
      }
    };
    fetchFormations();
  }, []);

  return (
    <div>
      <AccueilFormations formations={formations} setFormations={setFormations} />
    </div>
  );
}

export default App;

```

```

// Transformer Le tableau en objet avec le format attendu par AccueilFormatio
ns

const formationsFormatees = {};
data.forEach(formation => {
  formationsFormatees[formation.id_formation] = {
    titre: formation.nom,
    images: formation.chemins_images.map(img => `http://localhost:3000/image
s/${img}`)
  };
});

console.log('📋 Formations formatées:', formationsFormatees);
setFormations(formationsFormatees);

} catch (error) {
  console.error('❌ Erreur chargement formations:', error);
  // En cas d'erreur, utiliser des données de test
  setFormations({
    "formation-test": {
      titre: "Formation Test",
      images: ["https://via.placeholder.com/300x200/667eea/ffffff?text=Formatio
n+Test"]
    }
  });
}
});

fetchFormations();
}, []);

return (
  <AccueilFormations
    formations={formations}
    onFormationSelect={({formationId) => {
      console.log('Formation sélectionnée:', formationId);
    }}
  />
);
}

export default App;

```


Explication des modifications :

1. **Transformation du format** : L'API retourne un tableau, on le transforme en objet avec `id_formation` comme clé
2. **Champs mappés** :
 - `formation.nom` → `formation.titre`
 - `formation.chemins_images` → `formation.images`
3. **URLs d'images complètes** : On préfixe les chemins d'images avec `http://localhost:3000/images/`
4. **Gestion d'erreur** : Si l'API échoue, on affiche une formation test

Format de sortie maintenant :

javascript

```
{
  "formation1": {
    "titre": "Formation Blender 3D",
    "images": [
      "http://localhost:3000/images/Blender/blender1.png",
      "http://localhost:3000/images/Blender/blender2.jpg",
      // ...
    ]
  },
  "formation2": {
    "titre": "Formation Krita - Dessin Digital",
    "images": [
      "http://localhost:3000/images/Krita/krita1.avif",
      // ...
    ]
  }
  // ...
}
```

Testez maintenant - votre page d'accueil devrait afficher toutes les formations depuis la base de données avec les images correctes !

Maintenant je veux qu'à partir de la page d'accueil on soit capable de cliquer sur une icône et de se rendre à la description de la

formation.

Voici le fichier src/components/DescriptionFormation.jsx

```
// DescriptionFormation.jsx
import React, { useState } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de
maîtriser les concepts avancés de React, incluant les hooks
personnalisés, le contexte avancé, les performances et les bonnes
pratiques de développement.",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1, // 0=lundi, 6=dimanche
  images = [], // Tableau d'images importées
  onRetour = () => {
    // Fonction par défaut qui utilise l'historique du navigateur
    window.history.back();
  }
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
  const [errors, setErrors] = useState({});

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];

  // Fonction pour générer les prochaines dates correspondant au
jour de la semaine
  const genererDatesDisponibles = () => {
    const dates = [];
```

```
const aujourdHui = new Date();

// Générer les 8 prochaines occurrences du jour de la semaine
for (let i = 0; i < 56; i++) { // 8 semaines pour être sûr d'avoir 8
dates
  const date = new Date(aujourdHui);
  date.setDate(aujourdHui.getDate() + i);

  // Vérifier si c'est le bon jour de la semaine
  if (date.getDay() === jourSemaine) {
    dates.push(new Date(date));
    if (dates.length >= 8) break; // On veut 8 dates maximum
  }
}

return dates;
};

const datesDisponibles = genererDatesDisponibles();

const formatDatePourSelect = (date) => {
  return date.toISOString().split('T')[0]; // Format YYYY-MM-DD
pour la valeur
};

const formatDateAffichage = (date) => {
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
};

const nextImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev + 1) % images.length);
  }
};
```

```
const prevImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev - 1 + images.length) %
images.length);
  }
};

const validateName = (name) => {
  return /^[A-Za-zÀ-ÿ\ - ]+$/ .test(name);
};

const validateEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email);
};

const validatePhone = (phone) => {
  if (!phone) return true; // Optionnel
  return /^[+]?([]?[0-9]{1,4}[ ])?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/ .test(phone);
};

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
  // Effacer l'erreur du champ quand l'utilisateur tape
  if (errors[name]) {
    setErrors(prev => ({
      ...prev,
      [name]: ''
    }));
  }
};

const handleSubmit = (e) => {
  e.preventDefault();
  const newErrors = {};
```

```
// Validation
if (!formData.nom.trim()) {
  newErrors.nom = 'Le nom est requis';
} else if (!validateName(formData.nom)) {
  newErrors.nom = 'Le nom ne doit contenir que des lettres et des
traits d\'union';
}

if (!formData.prenom.trim()) {
  newErrors.prenom = 'Le prénom est requis';
} else if (!validateName(formData.prenom)) {
  newErrors.prenom = 'Le prénom ne doit contenir que des lettres
et des traits d\'union';
}

if (!formData.email.trim()) {
  newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length === 0) {
  // Récupérer la date formatée pour l'affichage
  const dateSelectionnee = datesDisponibles.find(date =>
    formatDatePourSelect(date) === formData.dateSelectionnee
  );
  const dateFormatee = dateSelectionnee ?
formatDateAffichage(dateSelectionnee) :
formData.dateSelectionnee;

  // Simulation d'envoi des données
  console.log('Données soumises:', formData);
```

```
    alert(`Inscription réussie pour le ${dateFormatee} ! Un email de
confirmation vous a été envoyé.`);
    setFormData({
      nom: "",
      prenom: "",
      email: "",
      telephone: "",
      dateSelectionnee: ""
    });
    setShowForm(false);
    setErrors({});
  } else {
    setErrors(newErrors);
  }
};

const handleBack = () => {
  // Utilise la fonction onRetour passée en props
  if (onRetour) {
    onRetour();
  }
};

const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({
    nom: "",
    prenom: "",
    email: "",
    telephone: "",
    dateSelectionnee: ""
  });
};

return (
  <div className="formation-container">
    <div className="formation-content">
      {/* BOUTON MODIFIÉ POUR RETOURNER À L'ACCUEIL */}
      <button className="back-button" onClick={handleBack}>
```

← Retour aux formations

</button>

```

<div className="formation-grid">
  /* Section Galerie d'images */
  <div className="gallery-section">
    <div className="gallery-container">
      {images.length > 0 ? (
        <>
          <div className="image-main-container">
            <img
              src={images[imageIndex]}
              alt={`${titreEvenement} - Image ${imageIndex + 1}`}
              className="gallery-image"
            />

            {images.length > 1 && (
              <>
                <button className="nav-button prev" onClick=
{prevImage}>
                  <
                </button>
                <button className="nav-button next" onClick=
{nextImage}>
                  >
                </button>

                <div className="image-counter">
                  {imageIndex + 1} / {images.length}
                </div>
              </>
            )}
          </div>

          {images.length > 1 && (
            <div className="thumbnail-container">
              {images.map((img, index) => (
                <img
                  key={index}
                  src={img}

```

```

        alt={`Miniature ${index + 1}`}
        className={`thumbnail ${index === imageIndex ?
'active' : ''}`}
        onClick={() => setImageIndex(index)}
      />
    ))}
  </div>
)}
</>
): (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

{/* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>
  <div className="description-section">
    <h2>Description</h2>
    <p className="description-text">{descriptionEvenement}
  </p>
  </div>

  <button

```



```

        className="inscription-button"
        onClick={() => setShowForm(true)}
      >
        S'inscrire à la formation
      </button>
    </div>
  </div>

```

```

{ /* Formulaire d'inscription en overlay */
{showForm && (

```

```

  <div className="form-fullscreen-overlay">
    <div className="form-container">
      <div className="form-header">
        <h3>Inscription à la formation</h3>
      </div>

```

```

        <form className="inscription-form" onSubmit=
{handleSubmit}>
          <div className="form-group">
            <label htmlFor="prenom">Prénom *</label>
            <input
              type="text"
              id="prenom"
              name="prenom"
              value={formData.prenom}
              onChange={handleInputChange}
              className={errors.prenom ? 'error' : ''}
              placeholder="Entrez votre prénom"
            />
            {errors.prenom && <span className="error-message">
{errors.prenom}</span>}
          </div>

```

```

          <div className="form-group">
            <label htmlFor="nom">Nom *</label>
            <input
              type="text"
              id="nom"
              name="nom"
              value={formData.nom}

```

```

        onChange={handleInputChange}
        className={errors.nom ? 'error' : ''}
        placeholder="Entrez votre nom"
      />
      {errors.nom && <span className="error-message">
{errors.nom}</span>}
    </div>

<div className="form-group">
  <label htmlFor="email">Email *</label>
  <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
  />
  {errors.email && <span className="error-message">
{errors.email}</span>}
</div>

{/* SÉLECTEUR DE DATE AMÉLIORÉ */}
<div className="form-group">
  <label htmlFor="dateSelectionnee">Date de formation *
</label>
  <select
    id="dateSelectionnee"
    name="dateSelectionnee"
    value={formData.dateSelectionnee}
    onChange={handleInputChange}
    className={errors.dateSelectionnee ? 'error' : ''}
  >
    <option value="">Sélectionnez une date</option>
    {datesDisponibles.map((date, index) => (
      <option
        key={index}
        value={formatDatePourSelect(date)}
      >

```

```

        {formatDateAffichage(date)}
      </option>
    )})
  </select>
  {/* AFFICHAGE DE LA DATE SÉLECTIONNÉE */}
  {formData.dateSelectionnee && (
    <div className="selected-date-display">
      Date sélectionnée : <strong>
        {formatDateAffichage(
          datesDisponibles.find(date =>
            formatDatePourSelect(date) ===
formData.dateSelectionnee
          )
        )}
      </strong>
    </div>
  )}
  {errors.dateSelectionnee && <span className="error-
message">{errors.dateSelectionnee}</span>}
</div>

<div className="form-group">
  <label htmlFor="telephone">Téléphone (optionnel)
</label>
  <input
    type="tel"
    id="telephone"
    name="telephone"
    value={formData.telephone}
    onChange={handleInputChange}
    className={errors.telephone ? 'error' : ''}
    placeholder="+33 1 23 45 67 89"
  />
  {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
</div>

<div className="form-buttons">
  <button type="submit" className="submit-button">
    Confirmer l'inscription

```

```

        </button>
        <button type="button" className="cancel-button"
onClick={closeForm}>
            Annuler
        </button>
    </div>
</form>
</div>
</div>
    )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

je veux donc un peu la même chose c'est-à-dire que l'on passe en paramètre les bonnes choses afin de pouvoir se rendre à la description de la formation. À l'exception que cette fois-ci je veux y accéder depuis la page d'accueil et non pas en entrant dans l'application web

Parfait ! Voici les modifications pour connecter la page d'accueil à la description de formation :

1. Modifier src/App.jsx pour gérer la navigation

```
javascript
```

```

// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import DescriptionFormation from './components/DescriptionFormation';
import './App.css';

function App() {
    const [currentPage, setCurrentPage] = useState('accueil'); // 'accueil' ou 'description'
    const [selectedFormation, setSelectedFormation] = useState(null);
    const [formations, setFormations] = useState({});

```

```

useEffect(() => {
  const fetchFormations = async () => {
    try {
      console.log('🔄 Chargement des formations depuis l API...');
      const response = await fetch('http://localhost:3000/api/formations');

      if (!response.ok) {
        throw new Error(`Erreur HTTP: ${response.status}`);
      }

      const data = await response.json();
      console.log('✅ Données brutes reçues:', data);

      // Transformer le tableau en objet avec le format attendu par AccueilFormatio
ns

      const formationsFormatees = {};
      data.forEach(formation => {
        formationsFormatees[formation.id_formation] = {
          titre: formation.nom,
          images: formation.chemins_images.map(img => `http://localhost:3000/image
s/${img}`),
          description: formation.description,
          jour_semaine: formation.jour_semaine,
          heure_debut: formation.heure_debut,
          heure_fin: formation.heure_fin
        };
      });

      console.log('📋 Formations formatées:', formationsFormatees);
      setFormations(formationsFormatees);

    } catch (error) {
      console.error('❌ Erreur chargement formations:', error);
      // En cas d'erreur, utiliser des données de test
      setFormations({
        "formation-test": {
          titre: "Formation Test",
          images: ["https://via.placeholder.com/300x200/667eea/ffffff?text=Formatio
n+Test"],
          description: "Description de test",
          jour_semaine: 1,
          heure_debut: "14:00:00",
          heure_fin: "17:00:00"
        }
      });
    }
  };
});

```

```

    }
  });
}
};

fetchFormations();
}, []);

const handleFormationSelect = (formationId) => {
  console.log('Formation sélectionnée:', formationId);
  setSelectedFormation(formationId);
  setCurrentPage('description');
};

const handleRetourAccueil = () => {
  setCurrentPage('accueil');
  setSelectedFormation(null);
};

// Afficher la page de description si une formation est sélectionnée
if (currentPage === 'description' && selectedFormation && formations[selectedFormation]) {
  const formation = formations[selectedFormation];
  return (
    <DescriptionFormation
      titreEvenement={formation.titre}
      descriptionEvenement={formation.description}
      heureEvenement={` ${formation.heure_debut?.substring(0, 5)} - ${formation.heure_fin?.substring(0, 5)} `}
      jourSemaine={formation.jour_semaine}
      images={formation.images}
      onRetour={handleRetourAccueil}
    />
  );
}

// Sinon afficher la page d'accueil
return (
  <AccueilFormations
    formations={formations}
    onFormationSelect={handleFormationSelect}
  />
);
}

```

```
export default App;
```

2. Vérifier que AccueilFormations.jsx appelle bien la fonction

Votre AccueilFormations.jsx actuel est déjà correct - il appelle onFormationSelect quand on clique sur une carte :

```
javascript
```

```
// Dans AccueilFormations.jsx - déjà présent
const handleFormationClick = (formationId) => {
  if (onFormationSelect) {
    onFormationSelect(formationId);
  }
};

// Et dans Le JSX :
<div
  key={formationId}
  className="formation-card"
  onClick={() => handleFormationClick(formationId)} // ✅ Déjà là
>
```

3. Fonctionnement

Maintenant quand vous cliquez sur une formation dans la page d'accueil :

1. **AccueilFormations** → appelle onFormationSelect(formationId)
2. **App.jsx** → handleFormationSelect met à jour l'état et change de page
3. **DescriptionFormation** → s'affiche avec les données de la formation sélectionnée
4. **Retour** → Le bouton "Retour aux formations" dans DescriptionFormation appelle onRetour qui ramène à l'accueil

4. Données passées à DescriptionFormation

- `titreEvenement` : Nom de la formation
- `descriptionEvenement` : Description complète
- `heureEvenement` : Heure de début et fin formatées
- `jourSemaine` : Jour de la semaine (0-6)
- `images` : Tableau des URLs des images
- `onRetour` : Fonction pour retourner à l'accueil

Testez maintenant - vous devriez pouvoir cliquer sur n'importe quelle formation dans l'accueil et être redirigé vers sa page de description avec toutes les informations !

M'en fichier de la page d'accueil ressemble à cela pour l'instant, j'ai déjà modifié le fichier `App.jsx` :

```
// AccueilFormations.jsx
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"]
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
```



```
    username: '',  
    password: ''  
  });
```

```
const handleFormationClick = (formationId) => {  
  if (onFormationSelect) {  
    onFormationSelect(formationId);  
  }  
};
```

```
const handleAdminLogin = (e) => {  
  e.preventDefault();  
  // Simulation de connexion admin  
  console.log('Tentative de connexion admin:', adminCredentials);  
  if (adminCredentials.username && adminCredentials.password) {  
    alert('Connexion administrateur réussie !');  
    setShowAdminLogin(false);  
    setAdminCredentials({ username: '', password: '' });  
  } else {  
    alert('Veuillez entrer un nom d\'utilisateur et un mot de passe');  
  }  
};
```

```
const handleAdminInputChange = (e) => {  
  const { name, value } = e.target;  
  setAdminCredentials(prev => ({  
    ...prev,  
    [name]: value  
  }));  
};
```

```
return (  
  <div className="accueil-container">  
    { /* Header avec bouton admin */ }  
    <header className="accueil-header">  
      <div className="header-content">  
        <h1 className="accueil-title">Formations en Ligne</h1>  
        <button  
          className="admin-button"  
          onClick={() => setShowAdminLogin(true)}  

```

```

>
  <button>
    </button>
  </div>
</header>

{/* Contenu principal */}
<main className="accueil-main">
  <div className="formations-grid">
    {Object.entries(formations).map(([formationId, formation]) =>
    (
      <div
        key={formationId}
        className="formation-card"
        onClick={() => handleFormationClick(formationId)}
      >
        <div className="formation-image-container">
          <img
            src={formation.images[0]}
            alt={formation.titre}
            className="formation-image"
            onError={(e) => {
              e.target.src =
                `https://via.placeholder.com/300x200/667eea/ffffff?
                text=${encodeURIComponent(formation.titre)}`;
            }}
          />
        </div>
        <div className="formation-info">
          <h3 className="formation-titre">{formation.titre}</h3>
          <div className="formation-overlay">
            <span className="voir-details">Voir les détails
→</span>
          </div>
        </div>
      </div>
    )
    )
  </div>

  {Object.keys(formations).length === 0 && (

```

```
<div className="no-formations">
  <p>Aucune formation disponible pour le moment.</p>
</div>
)}
</main>
```

```
{/* Modal de connexion admin */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">
        <h2>Connexion Administrateur</h2>
        <button
          className="close-button"
          onClick={() => setShowAdminLogin(false)}
        >
          ×
        </button>
      </div>

      <form className="admin-form" onSubmit=
{handleAdminLogin}>
        <div className="form-group">
          <label htmlFor="username">Nom d'utilisateur</label>
          <input
            type="text"
            id="username"
            name="username"
            value={adminCredentials.username}
            onChange={handleAdminInputChange}
            placeholder="Entrez votre nom d'utilisateur"
            required
          />
        </div>

        <div className="form-group">
          <label htmlFor="password">Mot de passe</label>
          <input
            type="password"
            id="password"
```

```

        name="password"
        value={adminCredentials.password}
        onChange={handleAdminInputChange}
        placeholder="Entrez votre mot de passe"
        required
      />
    </div>

    <div className="admin-form-buttons">
      <button type="submit" className="login-button">
        Se connecter
      </button>
      <button
        type="button"
        className="cancel-button"
        onClick={() => setShowAdminLogin(false)}
      >
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
);
};

export default AccueilFormations;

```

Parfait ! Avec votre `App.jsx` modifié et votre `AccueilFormations.jsx` actuel, tout devrait fonctionner. Voici ce qui se passe :

Vérification du fonctionnement :

1. Quand vous cliquez sur une formation dans `AccueilFormations.jsx` :

- `handleFormationClick(formationId)` est appelé
- Qui appelle `onFormationSelect(formationId)`

- Qui dans App.jsx déclenche handleFormationSelect(formationId)

2. Dans App.jsx :

javascript

```
const handleFormationSelect = (formationId) => {  
  console.log('Formation sélectionnée:', formationId);  
  setSelectedFormation(formationId);  
  setCurrentPage('description'); // ← C'est cette ligne qui change la page  
};
```

3. Si ça ne marche pas, vérifiez :

- Que App.jsx a bien les états currentPage et selectedFormation
- Que handleFormationSelect est bien passé à AccueilFormations
- La console du navigateur pour les erreurs

Test de débogage :

Ajoutez temporairement dans AccueilFormations.jsx :

javascript

```
const handleFormationClick = (formationId) => {  
  console.log('🖱️ Clic sur formation:', formationId);  
  if (onFormationSelect) {  
    console.log('📞 Appel de onFormationSelect');  
    onFormationSelect(formationId);  
  } else {  
    console.log('❌ onFormationSelect non défini');  
  }  
};
```

Si vous voyez "🖱️ Clic sur formation:" dans la console mais pas "📞 Appel de onFormationSelect", alors le problème est que onFormationSelect n'est pas passé correctement depuis App.jsx .

Qu'est-ce que vous voyez dans la console quand vous cliquez sur une formation ?

Bon parfait tout fonctionne. Maintenant concentrons-nous sur la page de description de la formation. Je veux qu'on se concentre tout particulièrement sur l'anglais qui peut apparaître pour s'inscrire à la formation. pour l'instant elle ressemble à ceci :

```
// DescriptionFormation.jsx
import React, { useState } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React, incluant les hooks personnalisés, le contexte avancé, les performances et les bonnes pratiques de développement.",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1, // 0=lundi, 6=dimanche
  images = [], // Tableau d'images importées
  onRetour = () => {
    // Fonction par défaut qui utilise l'historique du navigateur
    window.history.back();
  }
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: "",
    prenom: "",
    email: "",
    telephone: "",
    dateSelectionnee: ""
  });
  const [errors, setErrors] = useState({});

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche'];

  // Fonction pour générer les prochaines dates correspondant au jour de la semaine
  const genererDatesDisponibles = () => {
    const dates = [];
```

```
const aujourdHui = new Date();

// Générer les 8 prochaines occurrences du jour de la semaine
for (let i = 0; i < 56; i++) { // 8 semaines pour être sûr d'avoir 8
dates
  const date = new Date(aujourdHui);
  date.setDate(aujourdHui.getDate() + i);

  // Vérifier si c'est le bon jour de la semaine
  if (date.getDay() === jourSemaine) {
    dates.push(new Date(date));
    if (dates.length >= 8) break; // On veut 8 dates maximum
  }
}

return dates;
};

const datesDisponibles = genererDatesDisponibles();

const formatDatePourSelect = (date) => {
  return date.toISOString().split('T')[0]; // Format YYYY-MM-DD
pour la valeur
};

const formatDateAffichage = (date) => {
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
};

const nextImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev + 1) % images.length);
  }
};
```

```
const prevImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev - 1 + images.length) %
images.length);
  }
};

const validateName = (name) => {
  return /^[A-Za-zÀ-ÿ\ - ]+$/ .test(name);
};

const validateEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email);
};

const validatePhone = (phone) => {
  if (!phone) return true; // Optionnel
  return /^[\+]?([]?[0-9]{1,4}[ ])?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/ .test(phone);
};

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
  // Effacer l'erreur du champ quand l'utilisateur tape
  if (errors[name]) {
    setErrors(prev => ({
      ...prev,
      [name]: ''
    }));
  }
};

const handleSubmit = (e) => {
  e.preventDefault();
  const newErrors = {};
```



```
// Validation
if (!formData.nom.trim()) {
  newErrors.nom = 'Le nom est requis';
} else if (!validateName(formData.nom)) {
  newErrors.nom = 'Le nom ne doit contenir que des lettres et des
traits d\'union';
}

if (!formData.prenom.trim()) {
  newErrors.prenom = 'Le prénom est requis';
} else if (!validateName(formData.prenom)) {
  newErrors.prenom = 'Le prénom ne doit contenir que des lettres
et des traits d\'union';
}

if (!formData.email.trim()) {
  newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length === 0) {
  // Récupérer la date formatée pour l'affichage
  const dateSelectionnee = datesDisponibles.find(date =>
    formatDatePourSelect(date) === formData.dateSelectionnee
  );
  const dateFormatee = dateSelectionnee ?
formatDateAffichage(dateSelectionnee) :
formData.dateSelectionnee;

  // Simulation d'envoi des données
  console.log('Données soumises:', formData);
```

```
    alert(`Inscription réussie pour le ${dateFormatee} ! Un email de confirmation vous a été envoyé.`);
```

```
    setFormData({
      nom: "",
      prenom: "",
      email: "",
      telephone: "",
      dateSelectionnee: ""
    });
    setShowForm(false);
    setErrors({});
  } else {
    setErrors(newErrors);
  }
};
```

```
const handleBack = () => {
  // Utilise la fonction onRetour passée en props
  if (onRetour) {
    onRetour();
  }
};
```

```
const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({
    nom: "",
    prenom: "",
    email: "",
    telephone: "",
    dateSelectionnee: ""
  });
};
```

```
return (
  <div className="formation-container">
    <div className="formation-content">
      {/* BOUTON MODIFIÉ POUR RETOURNER À L'ACCUEIL */}
      <button className="back-button" onClick={handleBack}>
```

← Retour aux formations

</button>

```

<div className="formation-grid">
  /* Section Galerie d'images */
  <div className="gallery-section">
    <div className="gallery-container">
      {images.length > 0 ? (
        <>
          <div className="image-main-container">
            <img
              src={images[imageIndex]}
              alt={`${titreEvenement} - Image ${imageIndex + 1}`}
              className="gallery-image"
            />

            {images.length > 1 && (
              <>
                <button className="nav-button prev" onClick=
{prevImage}>
                  <
                </button>
                <button className="nav-button next" onClick=
{nextImage}>
                  >
                </button>

                <div className="image-counter">
                  {imageIndex + 1} / {images.length}
                </div>
              </>
            )}
          </div>

          {images.length > 1 && (
            <div className="thumbnail-container">
              {images.map((img, index) => (
                <img
                  key={index}
                  src={img}

```

```

        alt={`Miniature ${index + 1}`}
        className={`thumbnail ${index === imageIndex ?
'active' : ''}`}
        onClick={() => setImageIndex(index)}
      />
    ))}
  </div>
)}
</>
): (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

{/* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>
  <div className="description-section">
    <h2>Description</h2>
    <p className="description-text">{descriptionEvenement}
  </p>
  </div>

  <button

```

```

        className="inscription-button"
        onClick={() => setShowForm(true)}
      >
        S'inscrire à la formation
      </button>
    </div>
  </div>

```

```

{ /* Formulaire d'inscription en overlay */
  {showForm && (

```

```

    <div className="form-fullscreen-overlay">
      <div className="form-container">
        <div className="form-header">
          <h3>Inscription à la formation</h3>
        </div>

```

```

          <form className="inscription-form" onSubmit=
{handleSubmit}>
            <div className="form-group">
              <label htmlFor="prenom">Prénom *</label>
              <input
                type="text"
                id="prenom"
                name="prenom"
                value={formData.prenom}
                onChange={handleInputChange}
                className={errors.prenom ? 'error' : ''}
                placeholder="Entrez votre prénom"
              />
              {errors.prenom && <span className="error-message">
{errors.prenom}</span>}
            </div>

```

```

            <div className="form-group">
              <label htmlFor="nom">Nom *</label>
              <input
                type="text"
                id="nom"
                name="nom"
                value={formData.nom}

```

```

        onChange={handleInputChange}
        className={errors.nom ? 'error' : ''}
        placeholder="Entrez votre nom"
      />
      {errors.nom && <span className="error-message">
{errors.nom}</span>}
    </div>

<div className="form-group">
  <label htmlFor="email">Email *</label>
  <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
  />
  {errors.email && <span className="error-message">
{errors.email}</span>}
</div>

{/* SÉLECTEUR DE DATE AMÉLIORÉ */}
<div className="form-group">
  <label htmlFor="dateSelectionnee">Date de formation *
</label>
  <select
    id="dateSelectionnee"
    name="dateSelectionnee"
    value={formData.dateSelectionnee}
    onChange={handleInputChange}
    className={errors.dateSelectionnee ? 'error' : ''}
  >
    <option value="">Sélectionnez une date</option>
    {datesDisponibles.map((date, index) => (
      <option
        key={index}
        value={formatDatePourSelect(date)}
      >

```

```

        {formatDateAffichage(date)}
      </option>
    )}}
  </select>
  {/* AFFICHAGE DE LA DATE SÉLECTIONNÉE */}
  {formData.dateSelectionnee && (
    <div className="selected-date-display">
      Date sélectionnée : <strong>
        {formatDateAffichage(
          datesDisponibles.find(date =>
            formatDatePourSelect(date) ===
formData.dateSelectionnee
          )
        )}
      </strong>
    </div>
  )}
  {errors.dateSelectionnee && <span className="error-
message">{errors.dateSelectionnee}</span>}
</div>

<div className="form-group">
  <label htmlFor="telephone">Téléphone (optionnel)
</label>
  <input
    type="tel"
    id="telephone"
    name="telephone"
    value={formData.telephone}
    onChange={handleInputChange}
    className={errors.telephone ? 'error' : ''}
    placeholder="+33 1 23 45 67 89"
  />
  {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
</div>

<div className="form-buttons">
  <button type="submit" className="submit-button">
    Confirmer l'inscription

```

```

        </button>
        <button type="button" className="cancel-button"
onClick={closeForm}>
            Annuler
        </button>
    </div>
</form>
</div>
</div>
    )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

lorsque l'utilisateur veut s'inscrire à la formation je veux qu'il ait uniquement les dates de formation_instance dans la base de données qui correspondent à cette formation dans les dates de maximum un mois à l'avance. Il faut aussi montrer les dates qui ne sont pas pleines c'est-à-dire maximum 30 réservation.

La base de données est structurée de la manière suivante :

tu connais déjà les colonnes de formation

tu connais déjà la table des formations instances

maintenant envoie ta faire explorer la table table_reservation et la table reservation :

reservation :

id_reservation (une chaîne de caractères sous forme de 'reservation' + 1)

prenom (chaîne de caractères qui désigne le prénom de la réservation)

nom (chaîne de caractères qui désigne le nom de la réservation)

courriel (chaîne de caractère qui désigne le courriel de la réservation)

telephone (nombre entre 0000000000 et 9999999999)

table_reservation (cette table sert principalement à relier plusieurs réservations à une formation instance) :

id_reservation (clé étrangère de la table réservation)

id_formation_instance (clé étrangère de la table des formations instances)

important : dans cette dernière il n'y a pas plus que 30 réservations par formation instance et une réservation ne peut pas être liée à plusieurs formation instance.

C'est pour cela qu'il faut upgrader le petit onglet qui désigne la date à laquelle l'utilisateur voudrait réserver sa place pour la formation. Encore une fois il faut qu'il voie toutes les dates jusqu'à un mois à l'avance et qu'il ne puisse pas choisir les séances de formation instance qui sont déjà pleines.

Parfait ! Je comprends la structure. Voici les modifications pour

DescriptionFormation.jsx qui va récupérer les instances de formation disponibles depuis l'API :

1. Modifier DescriptionFormation.jsx

javascript

```
// DescriptionFormation.jsx
import React, { useState, useEffect } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React...",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1,
```

```

images = [],
formationId, // ✅ AJOUT: ID de la formation pour récupérer les instances
onRetour = () => {
  window.history.back();
}
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
  const [errors, setErrors] = useState({});
  const [instancesDisponibles, setInstancesDisponibles] = useState([]);
  const [loading, setLoading] = useState(false);

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche'];

  // ✅ NOUVEAU: Récupérer les instances disponibles depuis l'API
  useEffect(() => {
    const fetchInstancesDisponibles = async () => {
      if (!formationId) return;

      try {
        setLoading(true);
        const response = await fetch(`http://localhost:3000/api/formations-disponibles?formation=${formationId}`);

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('📋 Instances disponibles:', data);
        setInstancesDisponibles(data);
      } catch (error) {
        console.error('❌ Erreur chargement des instances:', error);
        // En cas d'erreur, on génère des dates par défaut
        setInstancesDisponibles(genererDatesParDefaut());
      } finally {

```

```
        setLoading(false);
    }
};

    fetchInstancesDisponibles();
}, [formationId]);

// ✅ NOUVEAU: Générer des dates par défaut si l'API échoue
const genererDatesParDefaut = () => {
    const dates = [];
    const aujourdHui = new Date();
    const dansUnMois = new Date();
    dansUnMois.setMonth(aujourdHui.getMonth() + 1);

    for (let i = 0; i < 56; i++) {
        const date = new Date(aujourdHui);
        date.setDate(aujourdHui.getDate() + i);

        if (date.getDay() === jourSemaine && date <= dansUnMois) {
            dates.push({
                id_instance: `instance-${i}`,
                date: date.toISOString().split('T')[0],
                places_restantes: 30, // Par défaut
                complet: false
            });
            if (dates.length >= 8) break;
        }
    }

    return dates;
};

// ✅ MODIFIÉ: Formater la date pour l'affichage
const formatDateAffichage = (dateString) => {
    const date = new Date(dateString + 'T00:00:00');
    return date.toLocaleDateString('fr-FR', {
        weekday: 'long',
        year: 'numeric',
        month: 'long',
        day: 'numeric'
    });
};

const nextImage = () => {
```

```

    if (images.length > 0) {
      setImageIndex((prev) => (prev + 1) % images.length);
    }
  };

  const prevImage = () => {
    if (images.length > 0) {
      setImageIndex((prev) => (prev - 1 + images.length) % images.length);
    }
  };

  const validateName = (name) => {
    return /^[A-Za-zÀ-ÿ\- ]+$/ .test(name);
  };

  const validateEmail = (email) => {
    return /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/ .test(email);
  };

  const validatePhone = (phone) => {
    if (!phone) return true;
    return /^[\\+]?([\\(]?[0-9]{1,4}[\\)]?[-\\s\\.]?[0-9]{1,4}[-\\s\\.]?[0-9]{1,9})$/ .test(phone);
  };

  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));

    if (errors[name]) {
      setErrors(prev => ({
        ...prev,
        [name]: ''
      }));
    }
  };

  // ✅ MODIFIÉ: Soumission avec envoi à L'API
  const handleSubmit = async (e) => {
    e.preventDefault();
    const newErrors = {};

```

```
// Validation
if (!formData.nom.trim()) {
  newErrors.nom = 'Le nom est requis';
} else if (!validateName(formData.nom)) {
  newErrors.nom = 'Le nom ne doit contenir que des lettres et des traits d\'union';
}

if (!formData.prenom.trim()) {
  newErrors.prenom = 'Le prénom est requis';
} else if (!validateName(formData.prenom)) {
  newErrors.prenom = 'Le prénom ne doit contenir que des lettres et des traits d\'union';
}

if (!formData.email.trim()) {
  newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

try {
  // Trouver l\'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => instance.date === formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    alert('Erreur: instance non trouvée');
    return;
  }
}
```

```
}

// Envoyer la réservation à L'API
const response = await fetch('http://localhost:3000/api/reservations', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    prenom: formData.prenom,
    nom: formData.nom,
    courriel: formData.email,
    telephone: formData.telephone,
    id_formation_instance: instanceSelectionnee.id_instance
  })
});

if (!response.ok) {
  const errorData = await response.json();
  throw new Error(errorData.error || 'Erreur lors de la réservation');
}

const result = await response.json();

// Succès
const dateFormatee = formatDateAffichage(formData.dateSelectionnee);
alert(`Inscription réussie pour le ${dateFormatee} ! Un email de confirmation v
ous a été envoyé.`);

// Réinitialiser Le formulaire
setFormData({
  nom: '',
  prenom: '',
  email: '',
  telephone: '',
  dateSelectionnee: ''
});
setShowForm(false);
setErrors({});

// Recharger Les instances pour mettre à jour Les places
const refreshResponse = await fetch(`http://localhost:3000/api/formations-dispo
nibles?formation=${formationId}`);
if (refreshResponse.ok) {
```

```

        const newInstances = await refreshResponse.json();
        setInstancesDisponibles(newInstances);
    }

    } catch (error) {
        console.error('Erreur réservation:', error);
        alert(`Erreur lors de l'inscription: ${error.message}`);
    }
};

const handleBack = () => {
    if (onRetour) {
        onRetour();
    }
};

const closeForm = () => {
    setShowForm(false);
    setErrors({});
    setFormData({
        nom: '',
        prenom: '',
        email: '',
        telephone: '',
        dateSelectionnee: ''
    });
};

return (
    <div className="formation-container">
        <div className="formation-content">
            <button className="back-button" onClick={handleBack}>
                ← Retour aux formations
            </button>

            <div className="formation-grid">
                { /* Section Galerie d'images (inchangée) */ }
                <div className="gallery-section">
                    <div className="gallery-container">
                        {images.length > 0 ? (
                            <>
                                <div className="image-main-container">
                                    <img
                                        src={images[imageIndex]}

```

```

        alt={` ${titreEvenement} - Image ${imageIndex + 1}`}
        className="gallery-image"
      />

      {images.length > 1 && (
        <>
          <button className="nav-button prev" onClick={prevImage}></bu
tton>

          <button className="nav-button next" onClick={nextImage}>></bu
tton>

          <div className="image-counter">{imageIndex + 1} / {images.len
gth}</div>

        </>
      )}
    </div>

    {images.length > 1 && (
      <div className="thumbnail-container">
        {images.map((img, index) => (
          <img
            key={index}
            src={img}
            alt={` Miniature ${index + 1}`}
            className={`thumbnail ${index === imageIndex ? 'active' :

''}`}

            onClick={() => setImageIndex(index)}
          />
        ))}
      </div>
    )}
  </>
) : (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

{/* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">

```



```

    <div className="meta-item">
      <span className="meta-icon"> 📅 </span>
      <span>Tous les {jours[jourSemaine]}</span>
    </div>
    <div className="meta-item">
      <span className="meta-icon"> 🕒 </span>
      <span>{heureEvenement}</span>
    </div>
  </div>
</div>

<div className="description-section">
  <h2>Description</h2>
  <p className="description-text">{descriptionEvenement}</p>
</div>

<button
  className="inscription-button"
  onClick={() => setShowForm(true)}
  disabled={loading}
>
  {loading ? 'Chargement...' : 'S\'inscrire à la formation'}
</button>

{/* ✅ AJOUT: Affichage du nombre d'instances disponibles */}
{instancesDisponibles.length > 0 && (
  <div className="disponibilite-info">
    <p>
      <strong>{instancesDisponibles.filter(i => !i.complet).length}</stro
ng>
      session(s) disponible(s) dans le prochain mois
    </p>
  </div>
)}
</div>
</div>

{/* Formulaire d'inscription */}
{showForm && (
  <div className="form-fullscreen-overlay">
    <div className="form-container">
      <div className="form-header">
        <h3>Inscription à la formation</h3>
        {loading && <p>Chargement des dates disponibles...</p>}
      </div>
    </div>
  </div>
)}

```

```

</div>

<form className="inscription-form" onSubmit={handleSubmit}>
  <div className="form-group">
    <label htmlFor="prenom">Prénom *</label>
    <input
      type="text"
      id="prenom"
      name="prenom"
      value={formData.prenom}
      onChange={handleInputChange}
      className={errors.prenom ? 'error' : ''}
      placeholder="Entrez votre prénom"
    />
    {errors.prenom && <span className="error-message">{errors.prenom}</span>}
  </div>

  <div className="form-group">
    <label htmlFor="nom">Nom *</label>
    <input
      type="text"
      id="nom"
      name="nom"
      value={formData.nom}
      onChange={handleInputChange}
      className={errors.nom ? 'error' : ''}
      placeholder="Entrez votre nom"
    />
    {errors.nom && <span className="error-message">{errors.nom}</span>}
  </div>

  <div className="form-group">
    <label htmlFor="email">Email *</label>
    <input
      type="email"
      id="email"
      name="email"
      value={formData.email}
      onChange={handleInputChange}
      className={errors.email ? 'error' : ''}
      placeholder="entrez@votre.email"
    />
    {errors.email && <span className="error-message">{errors.email}</span>}
  </div>

```

```
an>}
```

```
</div>
```

```
{/*  MODIFIÉ: Sélecteur de dates avec places restantes */}
```

```
<div className="form-group">
```

```
<label htmlFor="dateSelectionnee">Date de formation *</label>
```

```
<select
```

```
  id="dateSelectionnee"
```

```
  name="dateSelectionnee"
```

```
  value={formData.dateSelectionnee}
```

```
  onChange={handleInputChange}
```

```
  className={errors.dateSelectionnee ? 'error' : ''}
```

```
  disabled={loading || instancesDisponibles.length === 0}
```

```
>
```

```
<option value="">
```

```
  {loading ? 'Chargement...' : 'Sélectionnez une date'}
```

```
</option>
```

```
{instancesDisponibles
```

```
  .filter(instance => !instance.complet)
```

```
  .map((instance, index) => (
```

```
    <option
```

```
      key={index}
```

```
      value={instance.date}
```

```
    >
```

```
      {formatDateAffichage(instance.date)}
```

```
      {instance.places_restantes !== undefined &&
```

```
        ` (${instance.places_restantes} place(s) restante(s))`}
```

```
    </option>
```

```
  ))
```

```
}
```

```
</select>
```

```
{/* Affichage si aucune date disponible */}
```

```
{!loading && instancesDisponibles.filter(i => !i.complet).length ==
```

```
= 0 && (
```

```
  <div className="warning-message">
```

```
    Aucune session disponible dans le prochain mois
```

```
</div>
```

```
)}>
```

```
{errors.dateSelectionnee && <span className="error-message">{error
```

```
s.dateSelectionnee}</span>}
```

```
</div>
```

```

    <div className="form-group">
      <label htmlFor="telephone">Téléphone (optionnel)</label>
      <input
        type="tel"
        id="telephone"
        name="telephone"
        value={formData.telephone}
        onChange={handleInputChange}
        className={errors.telephone ? 'error' : ''}
        placeholder="+33 1 23 45 67 89"
      />
      {errors.telephone && <span className="error-message">{errors.teleph
one}</span>}
    </div>

    <div className="form-buttons">
      <button
        type="submit"
        className="submit-button"
        disabled={loading || instancesDisponibles.filter(i => !i.comple
t).length === 0}
      >
        {loading ? 'Traitement...' : 'Confirmer l\'inscription'}
      </button>
      <button type="button" className="cancel-button" onClick={closeForm}
>
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
</div>
);
};

export default DescriptionFormation;

```

2. Modifier App.jsx pour passer formationId

Dans votre `App.jsx`, assurez-vous de passer `formationId` à `DescriptionFormation` :

javascript

```
// Dans App.jsx - dans la partie qui affiche DescriptionFormation
return (
  <DescriptionFormation
    titreEvenement={formation.titre}
    descriptionEvenement={formation.description}
    heureEvenement={` ${formation.heure_debut?.substring(0, 5)} - ${formation.heure_fin?.substring(0, 5)} `}
    jourSemaine={formation.jour_semaine}
    images={formation.images}
    formationId={selectedFormation} // ✅ AJOUT: Passer l'ID de la formation
    onRetour={handleRetourAccueil}
  />
);
```

3. Routes API nécessaires

Vous aurez besoin de ces routes dans votre backend :

- GET `/api/formations-disponibles?formation=formation1` - Retourne les instances disponibles
- POST `/api/reservations` - Crée une nouvelle réservation

Maintenant le formulaire affichera uniquement les dates disponibles avec le nombre de places restantes !

Parfait cela fonctionne mais est-ce que cela cherche dans la base de données pour vérifier le nombre de places?

Non, actuellement le code frontend fait des appels API mais vous devez implémenter les routes backend qui vont réellement interroger la base de données. Voici les routes manquantes :

1. Route pour récupérer les instances disponibles

Ajoutez cette route dans `Backend/routes/formations.js` :

javascript

```
// Route pour récupérer les instances disponibles d'une formation
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;

    if (!formation) {
      return res.status(400).json({ error: 'ID de formation requis' });
    }

    const query = `
      SELECT
        fi.id_instance,
        fi.date_instance,
        fi.heure_debut,
        fi.heure_fin,
        fi.id_formation,
        COALESCE(tr.reservation_count, 0) as reservations_count,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.id_formation = $1
        AND fi.date_instance >= CURRENT_DATE
        AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
        AND (COALESCE(tr.reservation_count, 0) < 30)
      ORDER BY fi.date_instance
    `;

    const result = await pool.query(query, [formation]);

    const instances = result.rows.map(row => ({
      id_instance: row.id_instance,
      date: row.date_instance,
      heure_debut: row.heure_debut,
      heure_fin: row.heure_fin,
      places_restantes: row.places_restantes,
      complet: row.complet,
      reservations_count: row.reservations_count
    }));
  }
});
```

```

    }));

    res.json(instances);
  } catch (error) {
    console.error('Erreur récupération instances:', error);
    res.status(500).json({ error: error.message });
  }
});

```

2. Route pour créer une réservation

Ajoutez cette route dans Backend/routes/formations.js :

javascript

```

// Route pour créer une réservation
router.post('/reservations', async (req, res) => {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = req.body;

    // Validation des données requises
    if (!prenom || !nom || !courriel || !id_formation_instance) {
      return res.status(400).json({ error: 'Tous les champs obligatoires doivent être remplis' });
    }

    // Vérifier que l'instance existe et a des places disponibles
    const checkInstanceQuery = `
      SELECT
        fi.id_instance,
        COALESCE(tr.reservation_count, 0) as reservations_count
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.id_instance = $1
        AND (COALESCE(tr.reservation_count, 0) < 30)
    `;

    const instanceResult = await pool.query(checkInstanceQuery, [id_formation_instance]);
  }
});

```

```
if (instanceResult.rows.length === 0) {
  return res.status(400).json({ error: 'Cette session est complète ou n\'existe p
as' });
}

// Commencer une transaction
await pool.query('BEGIN');

// 1. Créer la réservation
const idReservation = 'reservation' + Date.now(); // Générer un ID unique

const insertReservationQuery = `
  INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
  VALUES ($1, $2, $3, $4, $5)
  RETURNING id_reservation
`;

const reservationResult = await pool.query(insertReservationQuery, [
  idReservation, prenom, nom, courriel, telephone || null
]);

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `
  INSERT INTO table_reservation (id_reservation, id_formation_instance)
  VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');

res.json({
  success: true,
  message: 'Réservation créée avec succès',
  id_reservation: idReservation
});

} catch (error) {
  // Annuler la transaction en cas d'erreur
  await pool.query('ROLLBACK');
  console.error('Erreur création réservation:', error);
}
```



```
    res.status(500).json({ error: error.message });  
  }  
});
```

3. Vérification de la structure des tables

Assurez-vous que vos tables ont bien ces colonnes :

formation_instance :

- id_instance (clé primaire)
- id_formation (clé étrangère vers formation)
- date_instance (DATE)
- heure_debut (TIME)
- heure_fin (TIME)

reservation :

- id_reservation (clé primaire)
- prenom (VARCHAR)
- nom (VARCHAR)
- courriel (VARCHAR)
- telephone (BIGINT ou VARCHAR)

table_reservation :

- id_reservation (clé étrangère vers reservation)
- id_formation_instance (clé étrangère vers formation_instance)

4. Test des routes

Testez maintenant :

1. Récupération des instances :

GET <http://localhost:3000/api/formations-disponibles?formation=formation1>

2. Création réservation : POST `http://localhost:3000/api/reservations` avec le body JSON

Maintenant oui, votre application va réellement vérifier dans la base de données le nombre de places disponibles !

Lorsque je fais `http://localhost:3000/api/formations-disponibles?formation=formation1` dans mon navigateur web cela me dit
`{"error":"reservationController is not defined"}`

L'erreur vient du fait que votre route `/formations-disponibles` essaie d'utiliser `reservationController` qui n'est pas défini. Voici la correction :

1. Modifier la route dans `Backend/routes/formations.js`

Remplacez cette route :

javascript

```
// Route pour Les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const formations = await reservationController.listerFormationsDisponibles();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Par cette version qui utilise directement la base de données :

javascript

```
// Route pour récupérer Les instances disponibles d'une formation
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;

    if (!formation) {
      return res.status(400).json({ error: 'ID de formation requis' });
    }
  }
});
```

```
console.log('🔍 Recherche instances pour formation:', formation);

const query = `
  SELECT
    fi.id_instance,
    fi.date_instance,
    fi.heure_debut,
    fi.heure_fin,
    fi.id_formation,
    COALESCE(tr.reservation_count, 0) as reservations_count,
    (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
    (COALESCE(tr.reservation_count, 0) >= 30) as complet
  FROM formation_instance fi
  LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
  ) tr ON fi.id_instance = tr.id_formation_instance
  WHERE fi.id_formation = $1
    AND fi.date_instance >= CURRENT_DATE
    AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
  ORDER BY fi.date_instance
`;

const result = await pool.query(query, [formation]);

console.log('📊 Instances trouvées:', result.rows.length);

const instances = result.rows.map(row => ({
  id_instance: row.id_instance,
  date: row.date_instance,
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
  places_restantes: row.places_restantes,
  complet: row.complet,
  reservations_count: row.reservations_count
}));

res.json(instances);
} catch (error) {
  console.error('❌ Erreur récupération instances:', error);
  res.status(500).json({ error: error.message });
}
```

```
}  
});
```

2. Assurez-vous d'avoir le pool de base de données

En haut de votre `routes/formations.js`, ajoutez :

javascript

```
import { Pool } from 'pg';  
  
const pool = new Pool({  
  user: process.env.DB_USER || 'postgres',  
  host: process.env.DB_HOST || 'localhost',  
  database: process.env.DB_NAME || 'softghost',  
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',  
  port: process.env.DB_PORT || 5431,  
});
```

3. Vérifiez la structure complète de `routes/formations.js`

Votre fichier devrait ressembler à ceci :

javascript

```
// routes/formations.js  
import express from 'express';  
import { Pool } from 'pg';  
  
const router = express.Router();  
  
const pool = new Pool({  
  user: process.env.DB_USER || 'postgres',  
  host: process.env.DB_HOST || 'localhost',  
  database: process.env.DB_NAME || 'softghost',  
  password: process.env.DB_PASSWORD || 'votre_mot_de_passe',  
  port: process.env.DB_PORT || 5431,  
});  
  
// Route pour récupérer toutes les formations  
router.get('/formations', async (req, res) => {
```

```

    try {
      const formations = await formationController.obtenirToutesFormations();
      res.json(formations);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

```

// Route pour récupérer Les instances disponibles d'une formation

```

router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;

    if (!formation) {
      return res.status(400).json({ error: 'ID de formation requis' });
    }

```

```

    console.log('🔍 Recherche instances pour formation:', formation);

```

```

    const query = `
      SELECT
        fi.id_instance,
        fi.date_instance,
        fi.heure_debut,
        fi.heure_fin,
        fi.id_formation,
        COALESCE(tr.reservation_count, 0) as reservations_count,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.id_formation = $1
        AND fi.date_instance >= CURRENT_DATE
        AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
      ORDER BY fi.date_instance
    `;

```

```

    const result = await pool.query(query, [formation]);

```

```

    console.log('📊 Instances trouvées:', result.rows.length);

```

```

const instances = result.rows.map(row => ({
  id_instance: row.id_instance,
  date: row.date_instance,
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
  places_restantes: row.places_restantes,
  complet: row.complet,
  reservations_count: row.reservations_count
}));

res.json(instances);
} catch (error) {
  console.error('✖ Erreur récupération instances:', error);
  res.status(500).json({ error: error.message });
}
});

// Route pour créer une réservation
router.post('/reservations', async (req, res) => {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = req.body;

    // Validation des données requises
    if (!prenom || !nom || !courriel || !id_formation_instance) {
      return res.status(400).json({ error: 'Tous les champs obligatoires doivent être remplis' });
    }

    console.log('📄 Nouvelle réservation:', { prenom, nom, courriel, id_formation_instance });

    // Vérifier que l'instance existe et a des places disponibles
    const checkInstanceQuery = `
      SELECT
        fi.id_instance,
        COALESCE(tr.reservation_count, 0) as reservations_count
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.id_instance = $1
    `

```

```

        AND (COALESCE(tr.reservation_count, 0) < 30)
    `;

    const instanceResult = await pool.query(checkInstanceQuery, [id_formation_instance]);

    if (instanceResult.rows.length === 0) {
        return res.status(400).json({ error: 'Cette session est complète ou n\'existe pas' });
    }

    // Commencer une transaction
    await pool.query('BEGIN');

    // 1. Créer la réservation
    const idReservation = 'reservation' + Date.now(); // Générer un ID unique

    const insertReservationQuery = `
        INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
        VALUES ($1, $2, $3, $4, $5)
        RETURNING id_reservation
    `;

    const reservationResult = await pool.query(insertReservationQuery, [
        idReservation, prenom, nom, courriel, telephone || null
    ]);

    // 2. Lier la réservation à l'instance
    const insertTableReservationQuery = `
        INSERT INTO table_reservation (id_reservation, id_formation_instance)
        VALUES ($1, $2)
    `;

    await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

    // Valider la transaction
    await pool.query('COMMIT');

    console.log('✅ Réservation créée:', idReservation);

    res.json({
        success: true,
        message: 'Réservation créée avec succès',
    });

```

```

        id_reservation: idReservation
    });

    } catch (error) {
        // Annuler la transaction en cas d'erreur
        await pool.query('ROLLBACK');
        console.error('✖ Erreur création réservation:', error);
        res.status(500).json({ error: error.message });
    }
});

export default router;

```

Redémarrez votre serveur backend et testez à nouveau :

`http://localhost:3000/api/formations-disponibles?formation=formation1`

Je pense plutôt que c'est parce que le fichier réservation Controller est mal importé

Vous avez raison ! Le problème vient de l'import du `reservationController` . Voici les corrections :

1. Vérifier l'import dans `Backend/routes/formations.js`

Regardez en haut du fichier, vous devez avoir :

javascript

```

// Backend/routes/formations.js
import express from 'express';
import formationController from '../controllers/formationController.js';
import formationInstanceController from '../controllers/formationInstanceController.js';
import reservationController from '../controllers/reservationController.js'; // ✅ Cette ligne doit exister

const router = express.Router();

```

2. Vérifier que `reservationController.js` existe et exporte correctement

Ouvrez Backend/controllers/reservationController.js et assurez-vous qu'il exporte la fonction :

javascript

```
// Backend/controllers/reservationController.js
import pool from '../db.js';

// Fonction pour lister les formations disponibles
async function listerFormationsDisponibles(formationId = null) {
  try {
    let query = `
      SELECT
        fi.id_instance,
        fi.date_instance,
        fi.heure_debut,
        fi.heure_fin,
        fi.id_formation,
        f.nom as formation_nom,
        COALESCE(tr.reservation_count, 0) as reservations_count,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation_instance fi
      JOIN formation f ON fi.id_formation = f.id_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.date_instance >= CURRENT_DATE
        AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
    `;

    const params = [];

    if (formationId) {
      query += ' AND fi.id_formation = $1';
      params.push(formationId);
    }

    query += ' ORDER BY fi.date_instance, fi.heure_debut';

    const result = await pool.query(query, params);
  } catch (error) {
    console.error('Erreur lors de la requête SQL:', error);
  }
}
```

```

    return result.rows.map(row => ({
      id_instance: row.id_instance,
      date: row.date_instance,
      heure_debut: row.heure_debut,
      heure_fin: row.heure_fin,
      formation_nom: row.formation_nom,
      places_restantes: row.places_restantes,
      complet: row.complet,
      reservations_count: row.reservations_count
    }));

  } catch (error) {
    console.error('Erreur lors de la récupération des formations disponibles:', error);
    throw error;
  }
}

// Exporter les fonctions
export default {
  listerFormationsDisponibles
};

```

3. Solution alternative : Utiliser directement le pool

Si le problème persiste, modifiez temporairement la route pour utiliser directement le pool :

javascript

```

// Dans Backend/routes/formations.js - Solution temporaire
import pool from '../db.js';

// Route pour les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;

    if (!formation) {
      return res.status(400).json({ error: 'ID de formation requis' });
    }
  }
}

```

```
const query = `
  SELECT
    fi.id_instance,
    fi.date_instance,
    fi.heure_debut,
    fi.heure_fin,
    fi.id_formation,
    COALESCE(tr.reservation_count, 0) as reservations_count,
    (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
    (COALESCE(tr.reservation_count, 0) >= 30) as complet
  FROM formation_instance fi
  LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
  ) tr ON fi.id_instance = tr.id_formation_instance
  WHERE fi.id_formation = $1
    AND fi.date_instance >= CURRENT_DATE
    AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
  ORDER BY fi.date_instance
`;

const result = await pool.query(query, [formation]);

const instances = result.rows.map(row => ({
  id_instance: row.id_instance,
  date: row.date_instance,
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
  places_restantes: row.places_restantes,
  complet: row.complet
})));

res.json(instances);
} catch (error) {
  res.status(500).json({ error: error.message });
}
});
```

4. Vérifier la structure des dossiers

Assurez-vous que votre structure est correcte :

text

```
Backend/
├─ controllers/
│   ├─ formationController.js
│   ├─ formationInstanceController.js
│   └─ reservationController.js    ← Doit exister
├─ routes/
│   └─ formations.js
└─ db.js
```

Quelle est l'erreur exacte que vous voyez dans les logs du serveur backend ? Cela nous dira si c'est un problème d'import ou si le fichier n'existe pas.

Je pense que tu t'es trompé dans la fonction,
listerFormationsDisponibles dans le fichier reservationController.js

cela me dit qu'il y a certaines colonnes qui n'existent pas

je pense que tu ne fais pas bien la différence entre
formation_instance et formation

la table des formations est la table qu'il ne faut absolument pas modifier car c'est la table qui contient les sortes de formation pour les différents logiciels. La formation instance est simplement un moment où on donne la formation du dit logiciel c'est pour ça que l'on dit que c'est une instance. Voici les colonnes associées

formation_instance :

id_formation_instance (chaîne de caractères sous forme de 'formation_instance' + nombre. C'est la clé primaire)
jour (date précise à laquelle aura lieu l'instance de la formation)
id_reference_formation (clé étrangère de la table formation)

formation :

id_formation (chaîne de caractères qui constitue la clé primaire sous le format 'formation' + nombre)
nom (nom de la formation)
jour_semaine (a un entier qui représente le jour de la semaine comme je te l'ai déjà expliqué)
heure_debut (heure de début de la formation)

heure_fin (heure de fin de la formation)
chemins_images (chemin vers les images, comme je te l'ai déjà expliqué)
description (description de l'événement sous forme de chaîne)

est-ce que tu peux corriger la fonction en question s'il te plaît?

Vous avez raison ! Je me suis trompé dans les noms de colonnes. Voici la fonction corrigée pour reservationController.js :

Backend/controllers/reservationController.js corrigé

javascript

```
// Backend/controllers/reservationController.js
import pool from '../db.js';

// Fonction pour lister les instances disponibles d'une formation
async function listerFormationsDisponibles(formationId = null) {
  try {
    let query = `
      SELECT
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        fi.id_reference_formation,
        f.nom as formation_nom,
        COALESCE(tr.reservation_count, 0) as reservations_count,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation_instance fi
      JOIN formation f ON fi.id_reference_formation = f.id_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.jour >= CURRENT_DATE
        AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
    `;
  }
}
```

```
const params = [];  
  
if (formationId) {  
  query += ' AND fi.id_reference_formation = $1';  
  params.push(formationId);  
}  
  
query += ' ORDER BY fi.jour, f.heure_debut';  
  
const result = await pool.query(query, params);  
  
return result.rows.map(row => ({  
  id_instance: row.id_formation_instance, // ✅ Corrigé  
  date: row.date_instance, // ✅ Corrigé  
  heure_debut: row.heure_debut,  
  heure_fin: row.heure_fin,  
  formation_nom: row.formation_nom,  
  places_restantes: row.places_restantes,  
  complet: row.complet,  
  reservations_count: row.reservations_count  
}));  
  
} catch (error) {  
  console.error('Erreur lors de la récupération des instances disponibles:', error);  
  throw error;  
}  
}  
  
// Fonction pour inscrire un utilisateur à une formation  
async function inscrireFormation(reservationData) {  
  try {  
    const { prenom, nom, courriel, telephone, id_formation_instance } = reservationData;  
  
    // Vérifier que l'instance a des places disponibles  
    const checkQuery = `  
      SELECT  
        fi.id_formation_instance,  
        COALESCE(tr.reservation_count, 0) as reservations_count  
      FROM formation_instance fi  
      LEFT JOIN (  
        SELECT id_formation_instance, COUNT(*) as reservation_count  
        FROM table_reservation
```

```
        GROUP BY id_formation_instance
    ) tr ON fi.id_formation_instance = tr.id_formation_instance
    WHERE fi.id_formation_instance = $1
        AND (COALESCE(tr.reservation_count, 0) < 30)
`;

const checkResult = await pool.query(checkQuery, [id_formation_instance]);

if (checkResult.rows.length === 0) {
    throw new Error('Cette session est complète ou n\'existe pas');
}

// Commencer une transaction
await pool.query('BEGIN');

// 1. Créer la réservation
const idReservation = 'reservation' + Date.now();

const insertReservationQuery = `
    INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
    VALUES ($1, $2, $3, $4, $5)
    RETURNING id_reservation
`;

const reservationResult = await pool.query(insertReservationQuery, [
    idReservation, prenom, nom, courriel, telephone || null
]);

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `
    INSERT INTO table_reservation (id_reservation, id_formation_instance)
    VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');

return idReservation;

} catch (error) {
    // Annuler la transaction en cas d'erreur
```

```

    await pool.query('ROLLBACK');
    console.error('Erreur lors de l\'inscription:', error);
    throw error;
  }
}

// Exporter les fonctions
export default {
  listerFormationsDisponibles,
  inscrireFormation
};

```

Corrections apportées :

1. `fi.id_instance` → `fi.id_formation_instance` (bon nom de colonne)
2. `fi.date_instance` → `fi.jour` (bon nom de colonne)
3. `fi.id_formation` → `fi.id_reference_formation` (clé étrangère correcte)
4. **Jointure corrigée** : `fi.id_reference_formation = f.id_formation`

Vérification dans Backend/routes/formations.js

Assurez-vous que la route utilise bien le controller :

javascript

```

// Dans Backend/routes/formations.js
import reservationController from '../controllers/reservationController.js';

// Route pour les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;
    const instances = await reservationController.listerFormationsDisponibles(formation);
    res.json(instances);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```



```
// Route pour Les inscriptions
router.post('/reservations', async (req, res) => {
  try {
    const idReservation = await reservationController.inscrireFormation(req.body);
    res.json({
      success: true,
      message: 'Inscription réussie',
      id_reservation: idReservation
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

Maintenant les noms de colonnes devraient correspondre à votre base de données !

Parfait maintenant revenons à notre problème initial je voudrais que lorsque j'appuie sur sélectionner une date dans l'inscription de la page description de la formation que l'on m'affiche les vraies places restantes au vraie date au maximum un mois à l'avance.

Voici à quoi ressemble le fichier formation.js

```
// Backend/routes/ formations.js
import express from 'express';
import formationController from '../formationController.js';
import reservationController from '../reservationController.js';

const router = express.Router();

// Routes pour les formations
router.get('/formations', async (req, res) => {
  try {
    const formations = await
    formationController.obtenirToutesFormations();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
router.get('/formations/:id', async (req, res) => {
  try {
    const formation = await
formationController.obtenirFormationParId(req.params.id);
    if (!formation) {
      return res.status(404).json({ error: 'Formation non trouvée' });
    }
    res.json(formation);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

// Routes pour les instances

```
router.get('/instances', async (req, res) => {
  try {
    const instances = await
formationInstanceController.obtenirInstancesDetaillées();
    res.json(instances);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
router.get('/instances/:id', async (req, res) => {
  try {
    const instance = await
formationInstanceController.obtenirInstanceParId(req.params.id);
    if (!instance) {
      return res.status(404).json({ error: 'Instance non trouvée' });
    }
    res.json(instance);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Route pour les inscriptions
router.post('/reservations', async (req, res) => {
  try {
    const idReservation = await
reservationController.inscrireFormation(req.body);
    res.json({
      success: true,
      message: 'Inscription réussie',
      id_reservation: idReservation
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// Route pour les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const formations = await
reservationController.listerFormationsDisponibles();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Route pour récupérer les instances disponibles d'une formation
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;

    if (!formation) {
      return res.status(400).json({ error: 'ID de formation requis' });
    }

    const query = `
      SELECT
        fi.id_instance,
        fi.date_instance,
```

```

    fi.heure_debut,
    fi.heure_fin,
    fi.id_formation,
    COALESCE(tr.reservation_count, 0) as reservations_count,
    (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
    (COALESCE(tr.reservation_count, 0) >= 30) as complet
FROM formation_instance fi
LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
) tr ON fi.id_instance = tr.id_formation_instance
WHERE fi.id_formation = $1
    AND fi.date_instance >= CURRENT_DATE
    AND fi.date_instance <= CURRENT_DATE + INTERVAL '1 month'
    AND (COALESCE(tr.reservation_count, 0) < 30)
ORDER BY fi.date_instance
`;

```

```
const result = await pool.query(query, [formation]);
```

```

const instances = result.rows.map(row => ({
  id_instance: row.id_instance,
  date: row.date_instance,
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
  places_restantes: row.places_restantes,
  complet: row.complet,
  reservations_count: row.reservations_count
}));

```

```

    res.json(instances);
  } catch (error) {
    console.error('Erreur récupération instances:', error);
    res.status(500).json({ error: error.message });
  }
});

```

```
// Route pour créer une réservation
```

```
router.post('/reservations', async (req, res) => {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance }
    = req.body;

    // Validation des données requises
    if (!prenom || !nom || !courriel || !id_formation_instance) {
      return res.status(400).json({ error: 'Tous les champs obligatoires
doivent être remplis' });
    }

    // Vérifier que l'instance existe et a des places disponibles
    const checkInstanceQuery = `
      SELECT
        fi.id_instance,
        COALESCE(tr.reservation_count, 0) as reservations_count
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_instance = tr.id_formation_instance
      WHERE fi.id_instance = $1
      AND (COALESCE(tr.reservation_count, 0) < 30)
    `;

    const instanceResult = await pool.query(checkInstanceQuery,
[id_formation_instance]);

    if (instanceResult.rows.length === 0) {
      return res.status(400).json({ error: 'Cette session est complète
ou n\'existe pas' });
    }

    // Commencer une transaction
    await pool.query('BEGIN');

    // 1. Créer la réservation
    const idReservation = 'reservation' + Date.now(); // Générer un ID
unique
```

```
const insertReservationQuery = `
  INSERT INTO reservation (id_reservation, prenom, nom, courriel,
telephone)
  VALUES ($1, $2, $3, $4, $5)
  RETURNING id_reservation
`;

const reservationResult = await
pool.query(insertReservationQuery, [
  idReservation, prenom, nom, courriel, telephone || null
]);

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `
  INSERT INTO table_reservation (id_reservation,
id_formation_instance)
  VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation,
id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');

res.json({
  success: true,
  message: 'Réservation créée avec succès',
  id_reservation: idReservation
});

} catch (error) {
  // Annuler la transaction en cas d'erreur
  await pool.query('ROLLBACK');
  console.error('Erreur création réservation:', error);
  res.status(500).json({ error: error.message });
}
});
```

```
export default router;
```

voici à quoi ressemble le fichier reservationController.js :

```
// Backend/controllers/reservationController.js
import pool from './db.js';

// Fonction pour lister les instances disponibles d'une formation
async function listerFormationsDisponibles(formationId = null) {
  try {
    let query = `
      SELECT
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        fi.id_reference_formation,
        f.nom as formation_nom,
        COALESCE(tr.reservation_count, 0) as reservations_count,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation_instance fi
      JOIN formation f ON fi.id_reference_formation = f.id_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.jour >= CURRENT_DATE
        AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
    `;

    const params = [];

    if (formationId) {
      query += ' AND fi.id_reference_formation = $1';
      params.push(formationId);
    }
  }
}
```

```
query += ' ORDER BY fi.jour, f.heure_debut';

const result = await pool.query(query, params);

return result.rows.map(row => ({
  id_instance: row.id_formation_instance, // ✅ Corrigé
  date: row.date_instance, // ✅ Corrigé
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
  formation_nom: row.formation_nom,
  places_restantes: row.places_restantes,
  complet: row.complet,
  reservations_count: row.reservations_count
}));

} catch (error) {
  console.error('Erreur lors de la récupération des instances disponibles:', error);
  throw error;
}
}

// Fonction pour inscrire un utilisateur à une formation
async function inscrireFormation(reservationData) {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = reservationData;

    // Vérifier que l'instance a des places disponibles
    const checkQuery = `
    SELECT
      fi.id_formation_instance,
      COALESCE(tr.reservation_count, 0) as reservations_count
    FROM formation_instance fi
    LEFT JOIN (
      SELECT id_formation_instance, COUNT(*) as reservation_count
      FROM table_reservation
      GROUP BY id_formation_instance
    ) tr ON fi.id_formation_instance = tr.id_formation_instance
    WHERE fi.id_formation_instance = $1
```



```
    AND (COALESCE(tr.reservation_count, 0) < 30)
`;

const checkResult = await pool.query(checkQuery,
[id_formation_instance]);

if (checkResult.rows.length === 0) {
    throw new Error('Cette session est complète ou n\'existe pas');
}

// Commencer une transaction
await pool.query('BEGIN');

// 1. Créer la réservation
const idReservation = 'reservation' + Date.now();

const insertReservationQuery = `
    INSERT INTO reservation (id_reservation, prenom, nom, courriel,
telephone)
    VALUES ($1, $2, $3, $4, $5)
    RETURNING id_reservation
`;

const reservationResult = await
pool.query(insertReservationQuery, [
    idReservation, prenom, nom, courriel, telephone || null
]);

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `
    INSERT INTO table_reservation (id_reservation,
id_formation_instance)
    VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation,
id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');
```

```

    return idReservation;

  } catch (error) {
    // Annuler la transaction en cas d'erreur
    await pool.query('ROLLBACK');
    console.error('Erreur lors de l\'inscription:', error);
    throw error;
  }
}

```

```

// Exporter les fonctions
export default {
  listerFormationsDisponibles,
  inscrireFormation
};

```

et finalement voici à quoi ressemble le fichier de la DescriptionFormation.jsx :

```

// DescriptionFormation.jsx
import React, { useState, useEffect } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React...",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1,
  images = [],
  formationId, // ✅ AJOUT: ID de la formation pour récupérer les instances
  onRetour = () => {
    window.history.back();
  }
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);

```

```
const [formData, setFormData] = useState({
  nom: '',
  prenom: '',
  email: '',
  telephone: '',
  dateSelectionnee: ''
});
const [errors, setErrors] = useState({});
const [instancesDisponibles, setInstancesDisponibles] =
useState([]);
const [loading, setLoading] = useState(false);

const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];

// ✅ NOUVEAU: Récupérer les instances disponibles depuis l'API
useEffect(() => {
  const fetchInstancesDisponibles = async () => {
    if (!formationId) return;

    try {
      setLoading(true);
      const response = await
fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);

      if (!response.ok) {
        throw new Error(`Erreur HTTP: ${response.status}`);
      }

      const data = await response.json();
      console.log('📅 Instances disponibles:', data);
      setInstancesDisponibles(data);
    } catch (error) {
      console.error('❌ Erreur chargement des instances:', error);
      // En cas d'erreur, on génère des dates par défaut
      setInstancesDisponibles(genererDatesParDefaut());
    } finally {
      setLoading(false);
    }
  }
});
```

```
};

fetchInstancesDisponibles();
}, [formationId]);

// ✅ NOUVEAU: Générer des dates par défaut si l'API échoue
const genererDatesParDefaut = () => {
  const dates = [];
  const aujourdHui = new Date();
  const dansUnMois = new Date();
  dansUnMois.setMonth(aujourdHui.getMonth() + 1);

  for (let i = 0; i < 56; i++) {
    const date = new Date(aujourdHui);
    date.setDate(aujourdHui.getDate() + i);

    if (date.getDay() === jourSemaine && date <= dansUnMois) {
      dates.push({
        id_instance: `instance-${i}`,
        date: date.toISOString().split('T')[0],
        places_restantes: 30, // Par défaut
        complet: false
      });
      if (dates.length >= 8) break;
    }
  }

  return dates;
};

// ✅ MODIFIÉ: Formater la date pour l'affichage
const formatDateAffichage = (dateString) => {
  const date = new Date(dateString + 'T00:00:00');
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
};
```

```
const nextImage = () => {  
  if (images.length > 0) {  
    setImageIndex((prev) => (prev + 1) % images.length);  
  }  
};
```

```
const prevImage = () => {  
  if (images.length > 0) {  
    setImageIndex((prev) => (prev - 1 + images.length) %  
images.length);  
  }  
};
```

```
const validateName = (name) => {  
  return /^[A-Za-zÀ-ÿ\ -]+$/.test(name);  
};
```

```
const validateEmail = (email) => {  
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);  
};
```

```
const validatePhone = (phone) => {  
  if (!phone) return true;  
  return /^[+]?([0-9]{1,4})?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/.test(phone);  
};
```

```
const handleInputChange = (e) => {  
  const { name, value } = e.target;  
  setFormData(prev => ({  
    ...prev,  
    [name]: value  
  }));  
};
```

```
if (errors[name]) {  
  setErrors(prev => ({  
    ...prev,  
    [name]: ''  
  }));  
};
```

```
    }  
};  
  
// ✅ MODIFIÉ: Soumission avec envoi à l'API  
const handleSubmit = async (e) => {  
  e.preventDefault();  
  const newErrors = {};  
  
  // Validation  
  if (!formData.nom.trim()) {  
    newErrors.nom = 'Le nom est requis';  
  } else if (!validateName(formData.nom)) {  
    newErrors.nom = 'Le nom ne doit contenir que des lettres et des  
traits d\'union';  
  }  
  
  if (!formData.prenom.trim()) {  
    newErrors.prenom = 'Le prénom est requis';  
  } else if (!validateName(formData.prenom)) {  
    newErrors.prenom = 'Le prénom ne doit contenir que des lettres  
et des traits d\'union';  
  }  
  
  if (!formData.email.trim()) {  
    newErrors.email = 'L\'email est requis';  
  } else if (!validateEmail(formData.email)) {  
    newErrors.email = 'Format d\'email invalide';  
  }  
  
  if (!formData.dateSelectionnee) {  
    newErrors.dateSelectionnee = 'Veuillez sélectionner une date';  
  }  
  
  if (formData.telephone && !validatePhone(formData.telephone)) {  
    newErrors.telephone = 'Format de téléphone invalide';  
  }  
  
  if (Object.keys(newErrors).length > 0) {  
    setErrors(newErrors);  
    return;  
  }  
}
```

```
}

try {
  // Trouver l'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => instance.date === formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    alert('Erreur: instance non trouvée');
    return;
  }

  // Envoyer la réservation à l'API
  const response = await
fetch('http://localhost:3000/api/reservations', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    prenom: formData.prenom,
    nom: formData.nom,
    courriel: formData.email,
    telephone: formData.telephone,
    id_formation_instance: instanceSelectionnee.id_instance
  })
});

  if (!response.ok) {
    const errorData = await response.json();
    throw new Error(errorData.error || 'Erreur lors de la
réservation');
  }

  const result = await response.json();

  // Succès
  const dateFormatee =
formatDateAffichage(formData.dateSelectionnee);
```

```
alert(`Inscription réussie pour le ${dateFormatee} ! Un email de confirmation vous a été envoyé.`);
```

```
// Réinitialiser le formulaire
setFormData({
  nom: '',
  prenom: '',
  email: '',
  telephone: '',
  dateSelectionnee: ''
});
setShowForm(false);
setErrors({});

// Recharger les instances pour mettre à jour les places
const refreshResponse = await
fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);
if (refreshResponse.ok) {
  const newInstancees = await refreshResponse.json();
  setInstancesDisponibles(newInstancees);
}

} catch (error) {
  console.error('Erreur réservation:', error);
  alert(`Erreur lors de l'inscription: ${error.message}`);
}
};

const handleBack = () => {
  if (onRetour) {
    onRetour();
  }
};

const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({
    nom: '',
```



```

    prenom: "",
    email: "",
    telephone: "",
    dateSelectionnee: ""
  });
};

return (
  <div className="formation-container">
    <div className="formation-content">
      <button className="back-button" onClick={handleBack}>
        ← Retour aux formations
      </button>

      <div className="formation-grid">
        { /* Section Galerie d'images (inchangée) */ }
        <div className="gallery-section">
          <div className="gallery-container">
            {images.length > 0 ? (
              <>
                <div className="image-main-container">
                  <img
                    src={images[imageIndex]}
                    alt={` ${titreEvenement} - Image ${imageIndex + 1} `}
                    className="gallery-image"
                  />

                  {images.length > 1 && (
                    <>
                      <button className="nav-button prev" onClick=
{prevImage}><</button>
                      <button className="nav-button next" onClick=
{nextImage}>></button>
                      <div className="image-counter">{imageIndex + 1} /
{images.length}</div>
                    </>
                  )}
                </div>

                {images.length > 1 && (

```

```

<div className="thumbnail-container">
  {images.map((img, index) => (
    <img
      key={index}
      src={img}
      alt={`Miniature ${index + 1}`}
      className={`thumbnail ${index === imageIndex ?
'active' : ''}`}
      onClick={() => setImageIndex(index)}
    />
  ))}
</div>
)}
</>
): (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

```

```

{/* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>
</div>

```

```

<div className="description-section">
  <h2>Description</h2>

```

```

    <p className="description-text">{descriptionEvenement}
  </p>
</div>

<button
  className="inscription-button"
  onClick={() => setShowForm(true)}
  disabled={loading}
>
  {loading ? 'Chargement...' : 'S\'inscrire à la formation'}
</button>

{/* ✅ AJOUT: Affichage du nombre d'instances disponibles
*/}
{instancesDisponibles.length > 0 && (
  <div className="disponibilite-info">
    <p>
      <strong>{instancesDisponibles.filter(i =>
!i.complet).length}</strong>
      session(s) disponible(s) dans le prochain mois
    </p>
    </div>
  )}
</div>
</div>

{/* Formulaire d'inscription */}
{showForm && (
  <div className="form-fullscreen-overlay">
    <div className="form-container">
      <div className="form-header">
        <h3>Inscription à la formation</h3>
        {loading && <p>Chargement des dates disponibles...</p>}
      </div>

      <form className="inscription-form" onSubmit=
{handleSubmit}>
        <div className="form-group">
          <label htmlFor="prenom">Prénom *</label>
          <input

```

```
    type="text"
    id="prenom"
    name="prenom"
    value={formData.prenom}
    onChange={handleInputChange}
    className={errors.prenom ? 'error' : ''}
    placeholder="Entrez votre prénom"
  />
  {errors.prenom && <span className="error-message">
{errors.prenom}</span>}
</div>
```

```
<div className="form-group">
  <label htmlFor="nom">Nom *</label>
  <input
    type="text"
    id="nom"
    name="nom"
    value={formData.nom}
    onChange={handleInputChange}
    className={errors.nom ? 'error' : ''}
    placeholder="Entrez votre nom"
  />
  {errors.nom && <span className="error-message">
{errors.nom}</span>}
</div>
```

```
<div className="form-group">
  <label htmlFor="email">Email *</label>
  <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
  />
  {errors.email && <span className="error-message">
{errors.email}</span>}
</div>
```

```

</div>

{/*  MODIFIÉ: Sélecteur de dates avec places restantes
*/}

<div className="form-group">
  <label htmlFor="dateSelectionnee">Date de formation *
</label>

  <select
    id="dateSelectionnee"
    name="dateSelectionnee"
    value={formData.dateSelectionnee}
    onChange={handleInputChange}
    className={errors.dateSelectionnee ? 'error' : ''}
    disabled={loading || instancesDisponibles.length === 0}
  >
    <option value="">
      {loading ? 'Chargement...' : 'Sélectionnez une date'}
    </option>
    {instancesDisponibles
      .filter(instance => !instance.complet)
      .map((instance, index) => (
        <option
          key={index}
          value={instance.date}
        >
          {formatDateAffichage(instance.date)}
          {instance.places_restantes !== undefined &&
            ` (${instance.places_restantes} place(s)
restante(s))`}
        </option>
      ))
    }
  </select>

  {/* Affichage si aucune date disponible */}
  {!loading && instancesDisponibles.filter(i =>
!i.complet).length === 0 && (
    <div className="warning-message">
      Aucune session disponible dans le prochain mois
    </div>
  )}

```

```

    })

    {errors.dateSelectionnee && <span className="error-
message">{errors.dateSelectionnee}</span>}
    </div>

    <div className="form-group">
      <label htmlFor="telephone">Téléphone (optionnel)
    </label>
      <input
        type="tel"
        id="telephone"
        name="telephone"
        value={formData.telephone}
        onChange={handleInputChange}
        className={errors.telephone ? 'error' : ''}
        placeholder="+33 1 23 45 67 89"
      />
      {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
    </div>

    <div className="form-buttons">
      <button
        type="submit"
        className="submit-button"
        disabled={loading || instancesDisponibles.filter(i =>
!i.complet).length === 0}
      >
        {loading ? 'Traitement...' : 'Confirmer l\'inscription'}
      </button>
      <button type="button" className="cancel-button"
onClick={closeForm}>
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}

```

```

    </div>
  </div>
);
};

```

```
export default DescriptionFormation;
```

Je vois plusieurs problèmes dans vos fichiers. Voici les corrections nécessaires :

1. Problème dans Backend/routes/formations.js

Vous avez **deux routes avec le même nom** /formations-disponibles ! Supprimez la première et gardez seulement celle qui utilise le controller :

javascript

```

// Backend/routes/formations.js
import express from 'express';
import formationController from '../controllers/formationController.js';
import formationInstanceController from '../controllers/formationInstanceController.js';
import reservationController from '../controllers/reservationController.js';
import pool from '../db.js'; // ✅ AJOUT: Import du pool

const router = express.Router();

// Routes pour Les formations
router.get('/formations', async (req, res) => {
  try {
    const formations = await formationController.obtenirToutesFormations();
    res.json(formations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/formations/:id', async (req, res) => {
  try {
    const formation = await formationController.obtenirFormationParId(req.params.id);
    if (!formation) {
      return res.status(404).json({ error: 'Formation non trouvée' });
    }
  }
});

```

```
    res.json(formation);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Routes pour les instances
router.get('/instances', async (req, res) => {
  try {
    const instances = await formationInstanceController.obtenirInstancesDetaillees();
    res.json(instances);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/instances/:id', async (req, res) => {
  try {
    const instance = await formationInstanceController.obtenirInstanceParId(req.params.id);
    if (!instance) {
      return res.status(404).json({ error: 'Instance non trouvée' });
    }
    res.json(instance);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// ✅ CORRIGÉ: Une seule route pour les formations disponibles
router.get('/formations-disponibles', async (req, res) => {
  try {
    const { formation } = req.query;
    console.log('🔍 Recherche instances pour formation:', formation);

    const instances = await reservationController.listerFormationsDisponibles(formation);
    res.json(instances);
  } catch (error) {
    console.error('❌ Erreur récupération instances:', error);
    res.status(500).json({ error: error.message });
  }
});
```



```
//  CORRIGÉ: Une seule route pour Les réservations
router.post('/reservations', async (req, res) => {
  try {
    const idReservation = await reservationController.inscrireFormation(req.body);
    res.json({
      success: true,
      message: 'Inscription réussie',
      id_reservation: idReservation
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

//  SUPPRIMER: Les routes dupliquées en bas du fichier

export default router;
```

2. Vérifier l'import dans reservationController.js

```
javascript

// Backend/controllers/reservationController.js
import pool from '../db.js'; //  Corrigé Le chemin

// ... Le reste du code reste identique ...
```

3. Test de l'API

Maintenant testez l'API directement :

text

<http://localhost:3000/api/formations-disponibles?formation=formation1>

Si ça ne marche toujours pas, ajoutez ce debug dans reservationController.js :

```
javascript

async function listerFormationsDisponibles(formationId = null) {
  try {
```

```
console.log('🔗 Formation ID reçu:', formationId);

let query = `
  SELECT
    fi.id_formation_instance,
    fi.jour as date_instance,
    f.heure_debut,
    f.heure_fin,
    fi.id_reference_formation,
    f.nom as formation_nom,
    COALESCE(tr.reservation_count, 0) as reservations_count,
    (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
    (COALESCE(tr.reservation_count, 0) >= 30) as complet
  FROM formation_instance fi
  JOIN formation f ON fi.id_reference_formation = f.id_formation
  LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
  ) tr ON fi.id_formation_instance = tr.id_formation_instance
  WHERE fi.jour >= CURRENT_DATE
    AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
`;

const params = [];

if (formationId) {
  query += ' AND fi.id_reference_formation = $1';
  params.push(formationId);
}

query += ' ORDER BY fi.jour, f.heure_debut';

console.log('📄 Requête SQL:', query);
console.log('🔧 Paramètres:', params);

const result = await pool.query(query, params);
console.log('✅ Résultats:', result.rows);

return result.rows.map(row => ({
  id_instance: row.id_formation_instance,
  date: row.date_instance,
  heure_debut: row.heure_debut,
  heure_fin: row.heure_fin,
```

```

        formation_nom: row.formation_nom,
        places_restantes: row.places_restantes,
        complet: row.complet,
        reservations_count: row.reservations_count
    }));

} catch (error) {
    console.error('✖ Erreur SQL:', error);
    throw error;
}
}

```

4. Vérifier que vous avez des données dans formation_instance

Connectez-vous à PostgreSQL et vérifiez :

```

sql

-- Vérifier les instances de formation
SELECT * FROM formation_instance
WHERE id_reference_formation = 'formation1'
    AND jour >= CURRENT_DATE
    AND jour <= CURRENT_DATE + INTERVAL '1 month';

-- Vérifier les réservations
SELECT id_formation_instance, COUNT(*)
FROM table_reservation
GROUP BY id_formation_instance;

```

Qu'est-ce que vous voyez dans les logs du backend quand vous testez l'API ? Cela nous dira exactement où est le problème.

Lorsque je tape le lien `http://localhost:3000/api/formations-disponibles?formation=formation1`

je vois ceci ce qui veut dire que cela fonctionne je pense et cela veut dire que l'on peut passer à l'étape suivante

```
[{"id_instance": "formation_instance1", "date": "2025-11-
```

```
04T05:00:00.000Z","heure_debut":"14:00:00","heure_fin":"17:00:00",
,"formation_nom":"Formation Blender
3D","places_restantes":"30","complet":false,"reservations_count":"0
"},{"id_instance":"formation_instance2","date":"2025-11-
11T05:00:00.000Z","heure_debut":"14:00:00","heure_fin":"17:00:00",
,"formation_nom":"Formation Blender
3D","places_restantes":"30","complet":false,"reservations_count":"0
"},{"id_instance":"formation_instance3","date":"2025-11-
18T05:00:00.000Z","heure_debut":"14:00:00","heure_fin":"17:00:00",
,"formation_nom":"Formation Blender
3D","places_restantes":"30","complet":false,"reservations_count":"0
"},{"id_instance":"formation_instance4","date":"2025-11-
25T05:00:00.000Z","heure_debut":"14:00:00","heure_fin":"17:00:00"
,"formation_nom":"Formation Blender
3D","places_restantes":"30","complet":false,"reservations_count":"0
"},{"id_instance":"formation_instance5","date":"2025-12-
02T05:00:00.000Z","heure_debut":"14:00:00","heure_fin":"17:00:00"
,"formation_nom":"Formation Blender
3D","places_restantes":"30","complet":false,"reservations_count":"0
"}]
```

Parfait ! L'API fonctionne correctement. Maintenant testons l'inscription :

Test d'inscription à une formation

1. Allez sur votre site : <http://localhost:5173>
2. Cliquez sur "Formation Blender 3D"
3. Cliquez sur "S'inscrire à la formation"
4. Remplissez le formulaire avec :
 - Prénom : Test
 - Nom : Utilisateur
 - Email : test@example.com
 - Date : Sélectionnez une des dates disponibles
 - Téléphone : (optionnel)

Ce qui devrait se passer :

1. **Dans le formulaire** : Vous devriez voir les vraies dates avec "30 place(s) restante(s)"
2. **Après soumission** : Un message de confirmation devrait apparaître
3. **Dans la base de données** : Une nouvelle réservation devrait être créée

Vérification dans la base de données :

Après une inscription réussie, vérifiez avec ces commandes SQL :

```
sql

-- Vérifier Les réservations créées
SELECT * FROM reservation;

-- Vérifier Les Liens table_reservation
SELECT * FROM table_reservation;

-- Vérifier que Le compteur de places a diminué
SELECT
    fi.id_formation_instance,
    fi.jour,
    COUNT(tr.id_reservation) as reservations_count,
    (30 - COUNT(tr.id_reservation)) as places_restantes
FROM formation_instance fi
LEFT JOIN table_reservation tr ON fi.id_formation_instance = tr.id_formation_instance
WHERE fi.id_reference_formation = 'formation1'
GROUP BY fi.id_formation_instance, fi.jour;
```

Si ça ne marche pas :

1. Ouvrez la console du navigateur (F12) et regardez les erreurs
2. Vérifiez les logs du backend pour voir les requêtes
3. Testez la route POST directement avec un outil comme Postman ou curl :

```
bash

curl -X POST http://localhost:3000/api/reservations \
  -H "Content-Type: application/json" \
  -d '{
```

```
"prenom": "Test",
"nom": "Utilisateur",
"courriel": "test@example.com",
"id_formation_instance": "formation_instance1"
}'
```

Testez l'inscription et dites-moi ce qui se passe ! Si ça fonctionne, vous devriez voir le compteur de places diminuer après chaque inscription.

Cela semble fonctionner car je vois dans les logs du back end le fait que le serveur cherche vraiment les instances de la formation

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend\$ npm start

> start

> node server.js

[dotenv@17.2.3] injecting env (6) from .env -- tip:  add secrets lifecycle management: <https://dotenvx.com/ops>

Serveur démarré sur le port 3000

 Recherche instances pour formation: formation1

 Recherche instances pour formation: formation1

 Recherche instances pour formation: formation4

 Recherche instances pour formation: formation4

^C

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend\$

cependant à l'affichage cela me dit des choses très bizarres, cela me dirait in valide et qu'il reste 30 places disponibles dans le menu déroulant. C'est comme si il disait qu'il restait 30 places à une plage indisponible ou invalide. De plus lorsque j'en sélectionne un cela ne m'affiche pas clairement la date que j'ai sélectionnée.

Pour l'instant la page de description de la formation ressemble à cela :

```
// DescriptionFormation.jsx
```

```
import React, { useState, useEffect } from 'react';
```

```
import './DescriptionFormation.css';
```

```
const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de
maîtriser les concepts avancés de React...",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1,
  images = [],
  formationId, // ✅ AJOUT: ID de la formation pour récupérer les
instances
  onRetour = () => {
    window.history.back();
  }
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: "",
    prenom: "",
    email: "",
    telephone: "",
    dateSelectionnee: ""
  });
  const [errors, setErrors] = useState({});
  const [instancesDisponibles, setInstancesDisponibles] =
useState([]);
  const [loading, setLoading] = useState(false);

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];

  // ✅ NOUVEAU: Récupérer les instances disponibles depuis l'API
  useEffect(() => {
    const fetchInstancesDisponibles = async () => {
      if (!formationId) return;

      try {
        setLoading(true);
        const response = await
fetch('http://localhost:3000/api/formations-disponibles?'
```

```
formation=${formationId}`);
```

```
    if (!response.ok) {
      throw new Error(`Erreur HTTP: ${response.status}`);
    }

```

```

    const data = await response.json();
    console.log('📅 Instances disponibles:', data);
    setInstancesDisponibles(data);
  } catch (error) {
    console.error('❌ Erreur chargement des instances:', error);
    // En cas d'erreur, on génère des dates par défaut
    setInstancesDisponibles(genererDatesParDefaut());
  } finally {
    setLoading(false);
  }
};

```

```

    fetchInstancesDisponibles();
  }, [formationId]);

```

// ✅ NOUVEAU: Générer des dates par défaut si l'API échoue

```

const genererDatesParDefaut = () => {
  const dates = [];
  const aujourdHui = new Date();
  const dansUnMois = new Date();
  dansUnMois.setMonth(aujourdHui.getMonth() + 1);

```

```

  for (let i = 0; i < 56; i++) {
    const date = new Date(aujourdHui);
    date.setDate(aujourdHui.getDate() + i);

```

```

    if (date.getDay() === jourSemaine && date <= dansUnMois) {
      dates.push({
        id_instance: `instance-${i}`,
        date: date.toISOString().split('T')[0],
        places_restantes: 30, // Par défaut
        complet: false
      });
      if (dates.length >= 8) break;
    }

```



```
    }  
  }  
  
  return dates;  
};  
  
//  MODIFIÉ: Formater la date pour l'affichage  
const formatDateAffichage = (dateString) => {  
  const date = new Date(dateString + 'T00:00:00');  
  return date.toLocaleDateString('fr-FR', {  
    weekday: 'long',  
    year: 'numeric',  
    month: 'long',  
    day: 'numeric'  
  });  
};  
  
const nextImage = () => {  
  if (images.length > 0) {  
    setIndex((prev) => (prev + 1) % images.length);  
  }  
};  
  
const prevImage = () => {  
  if (images.length > 0) {  
    setIndex((prev) => (prev - 1 + images.length) %  
images.length);  
  }  
};  
  
const validateName = (name) => {  
  return /^[A-Za-zÀ-ÿ\-\ ]+$/.test(name);  
};  
  
const validateEmail = (email) => {  
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);  
};  
  
const validatePhone = (phone) => {  
  if (!phone) return true;
```

```
    return /^[+]?([0-9]{1,4})?[-\s.]?[0-9]{1,4}[-\s.]?[0-9]{1,9}$/.test(phone);
  };

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));

  if (errors[name]) {
    setErrors(prev => ({
      ...prev,
      [name]: ''
    }));
  }
};

// ✅ MODIFIÉ: Soumission avec envoi à l'API
const handleSubmit = async (e) => {
  e.preventDefault();
  const newErrors = {};

  // Validation
  if (!formData.nom.trim()) {
    newErrors.nom = 'Le nom est requis';
  } else if (!validateName(formData.nom)) {
    newErrors.nom = 'Le nom ne doit contenir que des lettres et des traits d\'union';
  }

  if (!formData.prenom.trim()) {
    newErrors.prenom = 'Le prénom est requis';
  } else if (!validateName(formData.prenom)) {
    newErrors.prenom = 'Le prénom ne doit contenir que des lettres et des traits d\'union';
  }

  if (!formData.email.trim()) {
```

```
newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

try {
  // Trouver l'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => instance.date === formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    alert('Erreur: instance non trouvée');
    return;
  }

  // Envoyer la réservation à l'API
  const response = await
fetch('http://localhost:3000/api/reservations', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    prenom: formData.prenom,
    nom: formData.nom,
    courriel: formData.email,
```

```
        telephone: formData.telephone,
        id_formation_instance: instanceSelectionnee.id_instance
    })
});

if (!response.ok) {
    const errorData = await response.json();
    throw new Error(errorData.error || 'Erreur lors de la
réservation');
}

const result = await response.json();

// Succès
const dateFormatee =
formatDateAffichage(formData.dateSelectionnee);
    alert(`Inscription réussie pour le ${dateFormatee} ! Un email de
confirmation vous a été envoyé.`);

// Réinitialiser le formulaire
setFormData({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
});
setShowForm(false);
setErrors({});

// Recharger les instances pour mettre à jour les places
const refreshResponse = await
fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);
    if (refreshResponse.ok) {
        const newInstancees = await refreshResponse.json();
        setInstancesDisponibles(newInstancees);
    }

} catch (error) {
```

```
console.error('Erreur réservation:', error);
alert(`Erreur lors de l'inscription: ${error.message}`);
}
};

const handleBack = () => {
  if (onRetour) {
    onRetour();
  }
};

const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
};

return (
  <div className="formation-container">
    <div className="formation-content">
      <button className="back-button" onClick={handleBack}>
        ← Retour aux formations
      </button>

      <div className="formation-grid">
        { /* Section Galerie d'images (inchangée) */ }
        <div className="gallery-section">
          <div className="gallery-container">
            {images.length > 0 ? (
              <>
                <div className="image-main-container">
                  <img
                    src={images[imageIndex]}
                    alt={` ${titreEvenement} - Image ${imageIndex + 1} `}

```

```

        className="gallery-image"
      />

      {images.length > 1 && (
        <>
        <button className="nav-button prev" onClick=
{prevImage}></button>
        <button className="nav-button next" onClick=
{nextImage}></button>
        <div className="image-counter">{imageIndex + 1} /
{images.length}</div>
      </>
    )}
  </div>

  {images.length > 1 && (
    <div className="thumbnail-container">
      {images.map((img, index) => (
        <img
          key={index}
          src={img}
          alt={`Miniature ${index + 1}`}
          className={`thumbnail ${index === imageIndex ?
'active' : ''}`}
          onClick={() => setImageIndex(index)}
        />
      ))}
    </div>
  )}
  </>
) : (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

{/* Section Informations */}
<div className="info-section">

```

```

<div className="formation-header">
  <h1 className="formation-title">{titreEvenement}</h1>
  <div className="formation-meta">
    <div className="meta-item">
      <span className="meta-icon"> 📅 </span>
      <span>Tous les {jours[jourSemaine]}</span>
    </div>
    <div className="meta-item">
      <span className="meta-icon"> ⌚ </span>
      <span>{heureEvenement}</span>
    </div>
  </div>
</div>

<div className="description-section">
  <h2>Description</h2>
  <p className="description-text">{descriptionEvenement}
</p>
</div>

<button
  className="inscription-button"
  onClick={() => setShowForm(true)}
  disabled={loading}
>
  {loading ? 'Chargement...' : 'S\'inscrire à la formation'}
</button>

{/* ✅ AJOUT: Affichage du nombre d'instances disponibles
*/}

{instancesDisponibles.length > 0 && (
  <div className="disponibilite-info">
    <p>
      <strong>{instancesDisponibles.filter(i =>
!i.complet).length}</strong>
      session(s) disponible(s) dans le prochain mois
    </p>
  </div>
)}
</div>

```

</div>

{/* Formulaire d'inscription */}

{showForm && (

<div className="form-fullscreen-overlay">

<div className="form-container">

<div className="form-header">

<h3>Inscription à la formation</h3>

{loading && <p>Chargement des dates disponibles...</p>}

</div>

<form className="inscription-form" onSubmit={handleSubmit}>

<div className="form-group">

<label htmlFor="prenom">Prénom *</label>

<input

type="text"

id="prenom"

name="prenom"

value={formData.prenom}

onChange={handleInputChange}

className={errors.prenom ? 'error' : ''}

placeholder="Entrez votre prénom"

/>

{errors.prenom &&

{errors.prenom}}

</div>

<div className="form-group">

<label htmlFor="nom">Nom *</label>

<input

type="text"

id="nom"

name="nom"

value={formData.nom}

onChange={handleInputChange}

className={errors.nom ? 'error' : ''}

placeholder="Entrez votre nom"

/>

{errors.nom &&


```
{errors.nom}</span>}
```

```
</div>
```

```
<div className="form-group">
```

```
<label htmlFor="email">Email *</label>
```

```
<input
```

```
  type="email"
```

```
  id="email"
```

```
  name="email"
```

```
  value={formData.email}
```

```
  onChange={handleInputChange}
```

```
  className={errors.email ? 'error' : ''}
```

```
  placeholder="entrez@votre.email"
```



```
{errors.email && <span className="error-message">
```

```
{errors.email}</span>}
```

```
</div>
```

```
/*  MODIFIÉ: Sélecteur de dates avec places restantes
```

```
*/}
```

```
<div className="form-group">
```

```
<label htmlFor="dateSelectionnee">Date de formation *
```

```
</label>
```

```
<select
```

```
  id="dateSelectionnee"
```

```
  name="dateSelectionnee"
```

```
  value={formData.dateSelectionnee}
```

```
  onChange={handleInputChange}
```

```
  className={errors.dateSelectionnee ? 'error' : ''}
```

```
  disabled={loading || instancesDisponibles.length === 0}
```

```
>
```

```
<option value="">
```

```
  {loading ? 'Chargement...' : 'Sélectionnez une date'}
```

```
</option>
```

```
{instancesDisponibles
```

```
  .filter(instance => !instance.complet)
```

```
  .map((instance, index) => (
```

```
    <option
```

```
      key={index}
```

```
      value={instance.date}
```

```

    >
    {formatDateAffichage(instance.date)}
    {instance.places_restantes !== undefined &&
      ` (${instance.places_restantes} place(s)
restante(s))`}
    </option>
  ))
}
</select>

{ /* Affichage si aucune date disponible */ }
{ !loading && instancesDisponibles.filter(i =>
!i.complet).length === 0 && (
  <div className="warning-message">
    Aucune session disponible dans le prochain mois
  </div>
)}

{errors.dateSelectionnee && <span className="error-
message">{errors.dateSelectionnee}</span>}
</div>

<div className="form-group">
  <label htmlFor="telephone">Téléphone (optionnel)
</label>
  <input
    type="tel"
    id="telephone"
    name="telephone"
    value={formData.telephone}
    onChange={handleInputChange}
    className={errors.telephone ? 'error' : ''}
    placeholder="+33 1 23 45 67 89"
  />
  {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
</div>

<div className="form-buttons">
  <button

```

```

        type="submit"
        className="submit-button"
        disabled={loading || instancesDisponibles.filter(i =>
!i.complet).length === 0}
      >
        {loading ? 'Traitement...' : 'Confirmer l\'inscription'}
      </button>
      <button type="button" className="cancel-button"
onClick={closeForm}>
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

Je vois le problème ! Les dates de l'API sont au format ISO avec le timezone (2025-11-04T05:00:00.000Z) mais le code les traite comme des dates simples. Voici les corrections :

1. Modifier DescriptionFormation.jsx - Correction du format de date

```
javascript
```

```
// DescriptionFormation.jsx
```

```
//  CORRIGÉ: Formater la date pour l'affichage
```

```
const formatDateAffichage = (dateString) => {
  try {
    // Gérer les dates ISO avec timezone
    const date = new Date(dateString);
    return date.toLocaleDateString('fr-FR', {
```

```

        weekday: 'long',
        year: 'numeric',
        month: 'long',
        day: 'numeric'
    });
} catch (error) {
    console.error('Erreur formatage date:', dateString, error);
    return 'Date invalide';
}
};

// ✅ CORRIGÉ: Formater la date pour la valeur du select
const formatDatePourSelect = (dateString) => {
    try {
        const date = new Date(dateString);
        return date.toISOString().split('T')[0]; // Format YYYY-MM-DD
    } catch (error) {
        console.error('Erreur formatage date select:', dateString, error);
        return dateString;
    }
};

// ✅ CORRIGÉ: Dans le JSX du select
<div className="form-group">
    <label htmlFor="dateSelectionnee">Date de formation *</label>
    <select
        id="dateSelectionnee"
        name="dateSelectionnee"
        value={formData.dateSelectionnee}
        onChange={handleInputChange}
        className={errors.dateSelectionnee ? 'error' : ''}
        disabled={loading || instancesDisponibles.length === 0}
    >
        <option value="">
            {loading ? 'Chargement...' : 'Sélectionnez une date'}
        </option>
        {instancesDisponibles
            .filter(instance => !instance.complet)
            .map((instance, index) => (
                <option
                    key={index}
                    value={formatDatePourSelect(instance.date)} // ✅ Utiliser formatDatePourS
elect
                >

```

```

        {formatDateAffichage(instance.date)}
        {instance.places_restantes !== undefined &&
          ` (${instance.places_restantes} place(s) restante(s))`}
      </option>
    ))
  }
</select>

{/*  AJOUT: Affichage de La date sélectionnée */}
{formData.dateSelectionnee && (
  <div className="selected-date-display">
     Date sélectionnée : <strong>{formatDateAffichage(formData.dateSelectionnee
+ 'T00:00:00')}</strong>
  </div>
)}

{/* Affichage si aucune date disponible */}
{!loading && instancesDisponibles.filter(i => !i.complet).length === 0 && (
  <div className="warning-message">
    Aucune session disponible dans le prochain mois
  </div>
)}

{errors.dateSelectionnee && <span className="error-message">{errors.dateSelectionnee}</span>}
</div>

```

2. Ajouter du CSS pour l'affichage de la date sélectionnée

Ajoutez dans DescriptionFormation.css :

css

```

.selected-date-display {
  margin-top: 10px;
  padding: 10px;
  background: #f8f9fa;
  border-radius: 8px;
  border-left: 4px solid #4CAF50;
  font-size: 0.9em;
}

.warning-message {

```

```

margin-top: 10px;
padding: 10px;
background: #fff3cd;
border: 1px solid #ffeaa7;
border-radius: 8px;
color: #856404;
font-size: 0.9em;
}

```

3. Debug pour voir les données reçues

Ajoutez temporairement ce console.log pour vérifier les données :

javascript

```

// Dans le useEffect qui fetch les instances
useEffect(() => {
  const fetchInstancesDisponibles = async () => {
    if (!formationId) return;

    try {
      setLoading(true);
      const response = await fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);

      if (!response.ok) {
        throw new Error(`Erreur HTTP: ${response.status}`);
      }

      const data = await response.json();
      console.log('📋 Données brutes de l API:', data);
      console.log('🔍 Premier élément:', data[0]);
      console.log('📅 Date du premier élément:', data[0]?.date, 'Type:', typeof data
[0]?.date);

      setInstancesDisponibles(data);
    } catch (error) {
      console.error('❌ Erreur chargement des instances:', error);
      setInstancesDisponibles(genererDatesParDefaut());
    } finally {
      setLoading(false);
    }
  };
}

```

```
fetchInstancesDisponibles();  
}, [formationId]);
```

4. Vérification dans la console

Ouvrez la console du navigateur (F12) et regardez ce qui s'affiche quand vous ouvrez le formulaire. Vous devriez voir :

- Les données brutes de l'API
- Le format exact des dates
- Si les fonctions de formatage fonctionnent correctement

Maintenant testez à nouveau - les dates devraient s'afficher correctement en français avec le nombre de places restantes, et la date sélectionnée devrait être clairement affichée !

Maintenant je veux que tu face en sorte que l'on puisse faire une réservation pour une formation instance. La façon de faire une réservation est extrêmement simple. On doit d'abord créer une instance réservation dans la table des réservations. Puis on doit simplement la lier à une ligne de la table formation_instance par l'entremise de la table table_reservation. Si la réservation est effectuée avec succès je veux que l'utilisateur soit redirigé vers la page src/components/ConfirmationReservation.jsx. Sinon je veux qu'il soit redirigé vers la page d'erreur src/components/ErreurReservation.jsx.

Je te rappelle que la base de données est structurée de la manière suivante :

formation_instance :

id_formation_instance (chaîne de caractères qui représente la clé primaire sous format de "fomation_instance" + nombre)

jour (date à laquelle va se dérouler la formation)

id_reference_formation (clé étrangère qui pointe vers une ligne de la table formation (et non formation_instance))

reservation :

id_reservation (ID d'une réservation qui représente la clé primaire d'une réservation sous le format de "reservation" + nombre)
 prenom (prénom de la réservation)
 nom (nom relié à la formation)
 courriel (courriel relié à la formation sous le format d'une adresse courriel)
 telephone (nombre entre 0000000000 et 9999999999 qui représente le numéro de téléphone)

table_reservation :

id_reservation (clé étrangère qui pointe vers une ligne de la table réservation)
 id_fomration_instance (et une autre clé étrangère qui pointe vers une ligne de la table formation_instance)

voici à quoi ressemble la page

src/components/ConfirmationReservation.jsx :

```
// DescriptionFormation.js
import React, { useState, useEffect } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React, incluant les hooks personnalisés, le contexte avancé, les performances et les bonnes pratiques de développement.",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1, // 0=lundi, 6=dimanche
  dossierImages = "/images/formation" // Chemin vers le dossier d'images
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: '',
    prenom: '',
```



```
email: '',
telephone: '',
date: ''
});
const [errors, setErrors] = useState({});
const [images, setImages] = useState([]);
const [datesDisponibles, setDatesDisponibles] = useState([]);
const [showDateDropdown, setShowDateDropdown] =
useState(false);

const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];

// Générer les dates disponibles pour les 3 prochains mois
useEffect(() => {
  const genererDatesDisponibles = () => {
    const dates = [];
    const aujourd'hui = new Date();
    const finPeriode = new Date();
    finPeriode.setMonth(aujourd'hui.getMonth() + 3);

    let dateCourante = new Date(aujourd'hui);

    while (dateCourante <= finPeriode) {
      if (dateCourante.getDay() === jourSemaine) {
        dates.push(new Date(dateCourante));
      }
      dateCourante.setDate(dateCourante.getDate() + 1);
    }

    setDatesDisponibles(dates);
  };

  genererDatesDisponibles();
}, [jourSemaine]);

// Simulation du chargement des images depuis un dossier
useEffect(() => {
  const chargerImages = () => {
    // En environnement de développement, on simule des images
```

```

if (process.env.NODE_ENV === 'development') {
  const imagesSimulees = [
    `${dossierImages}/image1.jpg`,
    `${dossierImages}/image2.jpg`,
    `${dossierImages}/image3.jpg`,
    `${dossierImages}/image4.jpg`
  ];
  setImages(imagesSimulees);
} else {
  // En production, on charge les images depuis le dossier
  try {
    const context = require.context(' ../../public/images/formation',
false, /\. (jpg|jpeg|png|gif)$/);
    const imagesPaths = context.keys().map(context);
    setImages(imagesPaths);
  } catch (error) {
    console.warn('Erreur lors du chargement des images,
utilisation des images par défaut');
    setImages([
      "https://via.placeholder.com/800x400/667eea/ffffff?
text=Image+1",
      "https://via.placeholder.com/800x400/764ba2/ffffff?
text=Image+2",
      "https://via.placeholder.com/800x400/f093fb/ffffff?
text=Image+3",
      "https://via.placeholder.com/800x400/4facfe/ffffff?
text=Image+4"
    ]);
  }
}

chargerImages();
}, [dossierImages]);

const nextImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev + 1) % images.length);
  }
};

```

```
const prevImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev - 1 + images.length) %
images.length);
  }
};

const validateName = (name) => {
  return /^[A-Za-zÀ-ÿ\ - ]+$/ .test(name);
};

const validateEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email);
};

const validatePhone = (phone) => {
  if (!phone) return true; // Optionnel
  return /^[+]?([0-9]{1,4})?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/ .test(phone);
};

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};

const selectDate = (date) => {
  const dateFormatee = formatDateForDisplay(date);
  setFormData(prev => ({
    ...prev,
    date: dateFormatee
  }));
  setShowDateDropdown(false);
};

const formatDateForDisplay = (date) => {
```

```
return date.toLocaleDateString('fr-FR', {
  weekday: 'long',
  year: 'numeric',
  month: 'long',
  day: 'numeric'
});
};
```

```
const formatDateForDropdown = (date) => {
  return date.toLocaleDateString('fr-FR', {
    weekday: 'short',
    day: 'numeric',
    month: 'long'
  });
};
```

```
const handleSubmit = (e) => {
  e.preventDefault();
  const newErrors = {};
```

```
  // Validation
  if (!formData.nom.trim()) {
    newErrors.nom = 'Le nom est requis';
  } else if (!validateName(formData.nom)) {
    newErrors.nom = 'Le nom ne doit contenir que des lettres et des
traits d\'union';
  }
```

```
  if (!formData.prenom.trim()) {
    newErrors.prenom = 'Le prénom est requis';
  } else if (!validateName(formData.prenom)) {
    newErrors.prenom = 'Le prénom ne doit contenir que des lettres
et des traits d\'union';
  }
```

```
  if (!formData.email.trim()) {
    newErrors.email = 'L\'email est requis';
  } else if (!validateEmail(formData.email)) {
    newErrors.email = 'Format d\'email invalide';
  }
```

```
if (!formData.date.trim()) {
  newErrors.date = 'La date est requise';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length === 0) {
  // Simulation d'envoi des données
  console.log('Données soumises:', formData);
  alert(`Inscription réussie pour le ${formData.date} !`);
  setFormData({ nom: '', prenom: '', email: '', telephone: '', date: ''
});
  setShowForm(false);
} else {
  setErrors(newErrors);
}
};

const handleBack = () => {
  window.history.back();
};

const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({ nom: '', prenom: '', email: '', telephone: '', date: '' });
  setShowDateDropdown(false);
};

// Si pas d'images chargées, afficher un message de chargement
if (images.length === 0) {
  return (
    <div className="formation-container">
      <div className="formation-content">
        <button className="back-button" onClick={handleBack}>
          ← Retour à la page précédente
        </button>
      </div>
    </div>
  );
}
```

```

    <div className="loading-message">
      <div className="loading-spinner"></div>
      <p>Chargement des images...</p>
    </div>
  </div>
</div>
);
}

return (
  <div className="formation-container">
    <div className="formation-content">
      <button className="back-button" onClick={handleBack}>
        ← Retour à la page précédente
      </button>

      <div className="formation-grid">
        {/* Section Galerie d'images */}
        <div className="gallery-section">
          <div className="gallery-container">
            <img
              src={images[imageIndex]}
              alt={` ${titreEvenement} - Image ${imageIndex + 1}`}
              className="gallery-image"
              onError={(e) => {
                e.target.src =
`https://via.placeholder.com/800x400/667eea/ffffff?
text=Image+${imageIndex + 1}`;
              }}
            />

            {images.length > 1 && (
              <>
                <button className="nav-button prev" onClick=
{prevImage}>
                  <
                </button>
                <button className="nav-button next" onClick=
{nextImage}>
                  >
            )}
          )}
        </div>
      </div>
    </div>
  </div>
);
}

```

```

</button>

<div className="image-counter">
  {imageIndex + 1} / {images.length}
</div>

<div className="thumbnail-container">
  {images.map((img, index) => (
    <img
      key={index}
      src={img}
      alt={`Miniature ${index + 1}`}
      className={`thumbnail ${index === imageIndex ?
'active' : ''}`}
      onClick={() => setImageIndex(index)}
      onError={(e) => {
        e.target.src =
`https://via.placeholder.com/60x40/667eea/ffffff?text=${index + 1}`;
      }}
    />
  ))}
</div>
</>
)}
</div>
</div>

{/* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}s</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>

```

```

    </div>
  </div>

  <div className="description-section">
    <h2>Description</h2>
    <p className="description-text">{descriptionEvenement}
  </p>
  </div>

  <button
    className="inscription-button"
    onClick={() => setShowForm(true)}
  >
    S'inscrire à la formation
  </button>
</div>
</div>

{/* Formulaire d'inscription en overlay */}
{showForm && (
  <div className="form-fullscreen-overlay">
    <div className="form-container">
      <form className="inscription-form" onSubmit=
{handleSubmit}>
        <h3>Inscription à la formation</h3>

        <div className="form-group">
          <label htmlFor="prenom">Prénom *</label>
          <input
            type="text"
            id="prenom"
            name="prenom"
            value={formData.prenom}
            onChange={handleInputChange}
            className={errors.prenom ? 'error' : ''}
            placeholder="Entrez votre prénom"
          />
          {errors.prenom && <span className="error-message">
{errors.prenom}</span>}
        </div>

```



```
<div className="form-group">
  <label htmlFor="nom">Nom *</label>
  <input
    type="text"
    id="nom"
    name="nom"
    value={formData.nom}
    onChange={handleInputChange}
    className={errors.nom ? 'error' : ''}
    placeholder="Entrez votre nom"
  />
  {errors.nom && <span className="error-message">
{errors.nom}</span>}
</div>
```

```
<div className="form-group">
  <label htmlFor="email">Email *</label>
  <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
  />
  {errors.email && <span className="error-message">
{errors.email}</span>}
</div>
```

```
<div className="form-group date-group">
  <label htmlFor="date">Date de formation *</label>
  <div className="date-input-container">
    <input
      type="text"
      id="date"
      name="date"
      value={formData.date}
      readOnly
```

```

        className={`date-input ${errors.date ? 'error' : ''}
    ${showDateDropdown ? 'active' : ''}}
        placeholder="Sélectionnez une date"
        onClick={() =>
    setShowDateDropdown(!showDateDropdown)}
    />
    <span className="date-dropdown-arrow">▼</span>

    {showDateDropdown && (
    <div className="date-dropdown">
        {datesDisponibles.map((date, index) => (
        <div
            key={index}
            className="date-option"
            onClick={() => selectDate(date)}
        >
            {formatDateForDropdown(date)}
        </div>
        )))
    </div>
    )}
    {errors.date && <span className="error-message">
    {errors.date}</span>}
    </div>

    <div className="form-group">
    <label htmlFor="telephone">Téléphone (optionnel)
    </label>
    <input
        type="tel"
        id="telephone"
        name="telephone"
        value={formData.telephone}
        onChange={handleInputChange}
        className={errors.telephone ? 'error' : ''}
        placeholder="+33 1 23 45 67 89"
    />
    {errors.telephone && <span className="error-
    message">{errors.telephone}</span>}

```

```

    </div>

    <div className="form-buttons">
      <button type="submit" className="submit-button">
        Confirmer l'inscription
      </button>
      <button type="button" className="cancel-button"
onClick={closeForm}>
        Retour à la description
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

voici à quoi ressemble la page
src/components/ErreurReservation.jsx :

```

// ErreurReservation.js
import React from 'react';
import './ErreurReservation.css';

const ErreurReservation = () => {
  return (
    <div className="erreur-container">
      <div className="erreur-content">
        <div className="erreur-card">
          <div className="erreur-header">
            <div className="error-icon">X</div>
            <h1>Erreur de Réservation</h1>
            <p className="subtitle">Nous n'avons pas pu traiter votre
demande</p>

```

</div>

<div className="erreur-details">

<div className="error-message-section">

<div className="error-alert">

<h3>Une erreur technique est survenue</h3>

<p className="error-text">

Votre réservation n'a pas pu être finalisée en raison d'un problème technique.

Veuillez réessayer ultérieurement ou contacter notre support technique.

</p>

<div className="error-code">

Référence : ERR-
{Math.random().toString(36).substr(2, 8).toUpperCase()}

</div>

</div>

</div>

<div className="support-section">

<div className="info-box">

<h3>Assistance technique</h3>

<p>

Notre équipe de support est disponible pour vous aider à résoudre ce problème.

</p>

<div className="contact-info">

<div className="contact-item">

 📞

Téléphone : +33 1 84 88 88 88

</div>

<div className="contact-item">

 ✉️

Email : support@formations-tech.fr

</div>

<div className="contact-item">

```

        <span className="contact-icon">📞</span>
        <span className="contact-details">
          <strong>Horaires :</strong> Lun-Ven 9h-18h
        </span>
      </div>
    </div>
    <p className="note">
      ⚠️ Veuillez conserver la référence d'erreur ci-dessus
      lorsque vous contactez le support.
    </p>
  </div>
</div>

  <div className="erreur-footer">
    <p>Nous nous excusons pour la gêne occasionnée.</p>
  </div>
</div>
</div>
);
};

```

```
export default ErreurReservation;
```

et finalement voici à quoi ressemble la page
src/components/DescriptionFormation.jsx :

```

// DescriptionFormation.jsx
import React, { useState, useEffect } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de
maîtriser les concepts avancés de React...",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1,
  images = [],

```

formationId, //  AJOUT: ID de la formation pour récupérer les instances

```
onRetour = () => {
  window.history.back();
}
}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
  const [errors, setErrors] = useState({});
  const [instancesDisponibles, setInstancesDisponibles] =
  useState([]);
  const [loading, setLoading] = useState(false);

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];
```

//  NOUVEAU: Récupérer les instances disponibles depuis l'API

```
useEffect(() => {
  const fetchInstancesDisponibles = async () => {
    if (!formationId) return;

    try {
      setLoading(true);
      const response = await
fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);

      if (!response.ok) {
        throw new Error(`Erreur HTTP: ${response.status}`);
      }

      const data = await response.json();
      console.log('📋 Instances disponibles:', data);
```

```
    setInstancesDisponibles(data);
  } catch (error) {
    console.error(' ✖ Erreur chargement des instances:', error);
    // En cas d'erreur, on génère des dates par défaut
    setInstancesDisponibles(genererDatesParDefaut());
  } finally {
    setLoading(false);
  }
};

fetchInstancesDisponibles();
}, [formationId]);

// ✔ NOUVEAU: Générer des dates par défaut si l'API échoue
const genererDatesParDefaut = () => {
  const dates = [];
  const aujourdHui = new Date();
  const dansUnMois = new Date();
  dansUnMois.setMonth(aujourdHui.getMonth() + 1);

  for (let i = 0; i < 56; i++) {
    const date = new Date(aujourdHui);
    date.setDate(aujourdHui.getDate() + i);

    if (date.getDay() === jourSemaine && date <= dansUnMois) {
      dates.push({
        id_instance: `instance-${i}`,
        date: date.toISOString().split('T')[0],
        places_restantes: 30, // Par défaut
        complet: false
      });
      if (dates.length >= 8) break;
    }
  }

  return dates;
};

// ✔ CORRIGÉ: Formater la date pour l'affichage
const formatDateAffichage = (dateString) => {
```

```
try {
  // Gérer les dates ISO avec timezone
  const date = new Date(dateString);
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
} catch (error) {
  console.error('Erreur formatage date:', dateString, error);
  return 'Date invalide';
}
};

// ✅ CORRIGÉ: Formater la date pour la valeur du select
const formatDatePourSelect = (dateString) => {
  try {
    const date = new Date(dateString);
    return date.toISOString().split('T')[0]; // Format YYYY-MM-DD
  } catch (error) {
    console.error('Erreur formatage date select:', dateString, error);
    return dateString;
  }
};

const nextImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev + 1) % images.length);
  }
};

const prevImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev - 1 + images.length) %
images.length);
  }
};

const validateName = (name) => {
```



```
    return /^[A-Za-zÀ-ÿ- ]+$/.test(name);
  };

  const validateEmail = (email) => {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  };

  const validatePhone = (phone) => {
    if (!phone) return true;
    return /^[\+]?([]?[0-9]{1,4}[ ])?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/.test(phone);
  };

  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));

    if (errors[name]) {
      setErrors(prev => ({
        ...prev,
        [name]: ''
      }));
    }
  };

  //  MODIFIÉ: Soumission avec envoi à l'API
  const handleSubmit = async (e) => {
    e.preventDefault();
    const newErrors = {};

    // Validation
    if (!formData.nom.trim()) {
      newErrors.nom = 'Le nom est requis';
    } else if (!validateName(formData.nom)) {
      newErrors.nom = 'Le nom ne doit contenir que des lettres et des traits d\'union';
    }
  }
}
```

```
if (!formData.prenom.trim()) {
  newErrors.prenom = 'Le prénom est requis';
} else if (!validateName(formData.prenom)) {
  newErrors.prenom = 'Le prénom ne doit contenir que des lettres
et des traits d\'union';
}

if (!formData.email.trim()) {
  newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

try {
  // Trouver l'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => instance.date === formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    alert('Erreur: instance non trouvée');
    return;
  }

  // Envoyer la réservation à l'API
  const response = await
```

```
fetch('http://localhost:3000/api/reservations', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    prenom: formData.prenom,
    nom: formData.nom,
    courriel: formData.email,
    telephone: formData.telephone,
    id_formation_instance: instanceSelectionnee.id_instance
  })
});

if (!response.ok) {
  const errorData = await response.json();
  throw new Error(errorData.error || 'Erreur lors de la
réservation');
}

const result = await response.json();

// Succès
const dateFormatee =
formatDateAffichage(formData.dateSelectionnee);
alert(`Inscription réussie pour le ${dateFormatee} ! Un email de
confirmation vous a été envoyé.`);

// Réinitialiser le formulaire
setFormData({
  nom: '',
  prenom: '',
  email: '',
  telephone: '',
  dateSelectionnee: ''
});
setShowForm(false);
setErrors({});

// Recharger les instances pour mettre à jour les places
```

```
const refreshResponse = await
fetch(`http://localhost:3000/api/formations-disponibles?
formation=${formationId}`);
  if (refreshResponse.ok) {
    const newInstances = await refreshResponse.json();
    setInstancesDisponibles(newInstances);
  }

} catch (error) {
  console.error('Erreur réservation:', error);
  alert('Erreur lors de l'inscription: ${error.message}');
}
};

const handleBack = () => {
  if (onRetour) {
    onRetour();
  }
};

const closeForm = () => {
  setShowForm(false);
  setErrors({});
  setFormData({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
};

return (
  <div className="formation-container">
    <div className="formation-content">
      <button className="back-button" onClick={handleBack}>
        ← Retour aux formations
      </button>

      <div className="formation-grid">
```

```

    { /* Section Galerie d'images (inchangée) */ }
    <div className="gallery-section">
      <div className="gallery-container">
        {images.length > 0 ? (
          <>
            <div className="image-main-container">
              <img
                src={images[imageIndex]}
                alt={` ${titreEvenement} - Image ${imageIndex + 1} `}
                className="gallery-image"
              />

              {images.length > 1 && (
                <>
                  <button className="nav-button prev" onClick=
{prevImage}><</button>
                  <button className="nav-button next" onClick=
{nextImage}>></button>
                  <div className="image-counter">{imageIndex + 1} /
{images.length}</div>
                </>
              )}
            </div>

            {images.length > 1 && (
              <div className="thumbnail-container">
                {images.map((img, index) => (
                  <img
                    key={index}
                    src={img}
                    alt={` Miniature ${index + 1} `}
                    className={`thumbnail ${index === imageIndex ?
'active' : ''} `}
                    onClick={() => setImageIndex(index)}
                  />
                ))}
              </div>
            )}
          </>
        ) : (

```

```

    <div className="no-images">
      <p>Aucune image disponible</p>
    </div>
  )}
</div>
</div>

```

```

{ /* Section Informations */ }

```

```

<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>
</div>

```

```

<div className="description-section">
  <h2>Description</h2>
  <p className="description-text">{descriptionEvenement}
</p>
</div>

```

```

<button
  className="inscription-button"
  onClick={() => setShowForm(true)}
  disabled={loading}
>
  {loading ? 'Chargement...' : 'S\'inscrire à la formation'}
</button>

```

```

{ /* ✅ AJOUT: Affichage du nombre d'instances disponibles
*/ }

{instancesDisponibles.length > 0 && (

```

```

<div className="disponibilite-info">
  <p>
    <strong>{instancesDisponibles.filter(i =>
!i.complet).length}</strong>
    session(s) disponible(s) dans le prochain mois
  </p>
</div>
)}
</div>
</div>

{/* Formulaire d'inscription */}
{showForm && (
  <div className="form-fullscreen-overlay">
    <div className="form-container">
      <div className="form-header">
        <h3>Inscription à la formation</h3>
        {loading && <p>Chargement des dates disponibles...</p>}
      </div>

      <form className="inscription-form" onSubmit=
{handleSubmit}>
        <div className="form-group">
          <label htmlFor="prenom">Prénom *</label>
          <input
            type="text"
            id="prenom"
            name="prenom"
            value={formData.prenom}
            onChange={handleInputChange}
            className={errors.prenom ? 'error' : ''}
            placeholder="Entrez votre prénom"
          />
          {errors.prenom && <span className="error-message">
{errors.prenom}</span>}
        </div>

        <div className="form-group">
          <label htmlFor="nom">Nom *</label>
          <input

```

```

        type="text"
        id="nom"
        name="nom"
        value={formData.nom}
        onChange={handleInputChange}
        className={errors.nom ? 'error' : ''}
        placeholder="Entrez votre nom"
      />
      {errors.nom && <span className="error-message">
{errors.nom}</span>}
    </div>

<div className="form-group">
  <label htmlFor="email">Email *</label>
  <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
  />
  {errors.email && <span className="error-message">
{errors.email}</span>}
</div>

{/*  MODIFIÉ: Sélecteur de dates avec places restantes
*/}

<div className="form-group">
  <label htmlFor="dateSelectionnee">Date de formation *
</label>
  <select
    id="dateSelectionnee"
    name="dateSelectionnee"
    value={formData.dateSelectionnee}
    onChange={handleInputChange}
    className={errors.dateSelectionnee ? 'error' : ''}
    disabled={loading || instancesDisponibles.length === 0}
  >

```



```

<option value="">
  {loading ? 'Chargement...' : 'Sélectionnez une date'}
</option>
{instancesDisponibles
  .filter(instance => !instance.complet)
  .map((instance, index) => (
    <option
      key={index}
      value={formatDatePourSelect(instance.date)} // ✅ Utiliser
formatDatePourSelect
      >
        {formatDateAffichage(instance.date)}
        {instance.places_restantes !== undefined &&
          ` (${instance.places_restantes} place(s) restante(s))`
        </option>
      ))
  })
}
</select>

{/* ✅ AJOUT: Affichage de la date sélectionnée */}
{formData.dateSelectionnee && (
  <div className="selected-date-display">
    📅 Date sélectionnée : <strong>
{formatDateAffichage(formData.dateSelectionnee + 'T00:00:00')}
</strong>
    </div>
  )}

{/* Affichage si aucune date disponible */}
{!loading && instancesDisponibles.filter(i => !i.complet).length ===
0 && (
  <div className="warning-message">
    Aucune session disponible dans le prochain mois
  </div>
  )}

{errors.dateSelectionnee && <span className="error-message">
{errors.dateSelectionnee}</span>
  </div>

```

```

    <div className="form-group">
      <label htmlFor="telephone">Téléphone (optionnel)
    </label>
      <input
        type="tel"
        id="telephone"
        name="telephone"
        value={formData.telephone}
        onChange={handleInputChange}
        className={errors.telephone ? 'error' : ''}
        placeholder="+33 1 23 45 67 89"
      />
      {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
    </div>

    <div className="form-buttons">
      <button
        type="submit"
        className="submit-button"
        disabled={loading || instancesDisponibles.filter(i =>
!i.complet).length === 0}
      >
        {loading ? 'Traitement...' : 'Confirmer l\'inscription'}
      </button>
      <button type="button" className="cancel-button"
onClick={closeForm}>
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

Parfait ! Je vais modifier `DescriptionFormation.jsx` pour qu'il redirige vers les pages de confirmation ou d'erreur après la réservation. Voici les modifications :

1. Modifier `src/App.jsx` pour gérer la navigation

javascript

```
// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import DescriptionFormation from './components/DescriptionFormation';
import ConfirmationReservation from './components/ConfirmationReservation';
import ErreurReservation from './components/ErreurReservation';
import './App.css';

function App() {
  const [currentPage, setCurrentPage] = useState('accueil'); // 'accueil', 'description', 'confirmation', 'erreur'
  const [selectedFormation, setSelectedFormation] = useState(null);
  const [reservationData, setReservationData] = useState(null);
  const [formations, setFormations] = useState({});

  useEffect(() => {
    const fetchFormations = async () => {
      try {
        console.log('🔄 Chargement des formations depuis l API...');
        const response = await fetch('http://localhost:3000/api/formations');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('✅ Données brutes reçues:', data);

        const formationsFormatees = {};
        data.forEach(formation => {
          formationsFormatees[formation.id_formation] = {
            titre: formation.nom,
            images: formation.chemins_images.map(img => `http://localhost:3000/images/${img}`),
            description: formation.description,
            jour_semaine: formation.jour_semaine,
```

```

        heure_debut: formation.heure_debut,
        heure_fin: formation.heure_fin
    };
});

setFormations(formationsFormatees);

} catch (error) {
    console.error('✖ Erreur chargement formations:', error);
    setFormations({
        "formation-test": {
            titre: "Formation Test",
            images: ["https://via.placeholder.com/300x200/667eea/ffffff?text=Formatio
n+Test"],
            description: "Description de test",
            jour_semaine: 1,
            heure_debut: "14:00:00",
            heure_fin: "17:00:00"
        }
    });
}
};

fetchFormations();
}, []);

const handleFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
    setSelectedFormation(formationId);
    setCurrentPage('description');
};

const handleReservationSuccess = (reservationInfo) => {
    setReservationData(reservationInfo);
    setCurrentPage('confirmation');
};

const handleReservationError = (errorInfo) => {
    setReservationData(errorInfo);
    setCurrentPage('erreur');
};

const handleRetourAccueil = () => {
    setCurrentPage('accueil');
};

```

```

        setSelectedFormation(null);
        setReservationData(null);
    };

    // Afficher la page de confirmation
    if (currentPage === 'confirmation') {
        return <ConfirmationReservation onRetourAccueil={handleRetourAccueil} />;
    }

    // Afficher la page d'erreur
    if (currentPage === 'erreur') {
        return <ErreurReservation onRetourAccueil={handleRetourAccueil} />;
    }

    // Afficher la page de description
    if (currentPage === 'description' && selectedFormation && formations[selectedFormation]) {
        const formation = formations[selectedFormation];
        return (
            <DescriptionFormation
                titreEvenement={formation.titre}
                descriptionEvenement={formation.description}
                heureEvenement={` ${formation.heure_debut?.substring(0, 5)} - ${formation.heure_fin?.substring(0, 5)} `}
                jourSemaine={formation.jour_semaine}
                images={formation.images}
                formationId={selectedFormation}
                onRetour={handleRetourAccueil}
                onReservationSuccess={handleReservationSuccess}
                onReservationError={handleReservationError}
            />
        );
    }

    // Sinon afficher la page d'accueil
    return (
        <AccueilFormations
            formations={formations}
            onFormationSelect={handleFormationSelect}
        />
    );
}

```

```
export default App;
```

2. Modifier src/components/DescriptionFormation.jsx pour la redirection

javascript

```
// DescriptionFormation.jsx - MODIFICATIONS DANS LA FONCTION handleSubmit
```

```
//  MODIFIÉ: Soumission avec redirection
```

```
const handleSubmit = async (e) => {
```

```
  e.preventDefault();
```

```
  const newErrors = {};
```

```
  // Validation
```

```
  if (!formData.nom.trim()) {
```

```
    newErrors.nom = 'Le nom est requis';
```

```
  } else if (!validateName(formData.nom)) {
```

```
    newErrors.nom = 'Le nom ne doit contenir que des lettres et des traits d\'union';
```

```
  }
```

```
  if (!formData.prenom.trim()) {
```

```
    newErrors.prenom = 'Le prénom est requis';
```

```
  } else if (!validateName(formData.prenom)) {
```

```
    newErrors.prenom = 'Le prénom ne doit contenir que des lettres et des traits d\'union';
```

```
  }
```

```
  if (!formData.email.trim()) {
```

```
    newErrors.email = 'L\'email est requis';
```

```
  } else if (!validateEmail(formData.email)) {
```

```
    newErrors.email = 'Format d\'email invalide';
```

```
  }
```

```
  if (!formData.dateSelectionnee) {
```

```
    newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
```

```
  }
```

```
  if (formData.telephone && !validatePhone(formData.telephone)) {
```

```
    newErrors.telephone = 'Format de téléphone invalide';
```

```
  }
```

```
if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

try {
  // Trouver L'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => formatDatePourSelect(instance.date) === formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    throw new Error('Instance non trouvée');
  }

  // Envoyer La réservation à L'API
  const response = await fetch('http://localhost:3000/api/reservations', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      prenom: formData.prenom,
      nom: formData.nom,
      courriel: formData.email,
      telephone: formData.telephone,
      id_formation_instance: instanceSelectionnee.id_instance
    })
  });

  if (!response.ok) {
    const errorData = await response.json();
    throw new Error(errorData.error || 'Erreur lors de la réservation');
  }

  const result = await response.json();

  // ✅ SUCCÈS: Rediriger vers La page de confirmation
  if (onReservationSuccess) {
    const dateFormatee = formatDateAffichage(formData.dateSelectionnee + 'T00:00:00');
    onReservationSuccess({
      id_reservation: result.id_reservation,
    });
  }
}
```

```

        date: dateFormatee,
        formation: titreEvenement,
        prenom: formData.prenom,
        nom: formData.nom,
        email: formData.email
    });
}

} catch (error) {
    console.error('Erreur réservation:', error);

    // ✅ ERREUR: Rediriger vers la page d'erreur
    if (onReservationError) {
        onReservationError({
            message: error.message,
            formation: titreEvenement,
            date: formData.dateSelectionnee
        });
    }
}
};

```

3. Ajouter les props à DescriptionFormation

javascript

```

// Dans les props de DescriptionFormation
const DescriptionFormation = ({
    titreEvenement = "Formation React Avancé",
    descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React...",
    heureEvenement = "14:00 - 17:00",
    jourSemaine = 1,
    images = [],
    formationId,
    onRetour = () => {
        window.history.back();
    },
    onReservationSuccess = null, // ✅ NOUVEAU: Callback pour succès
    onReservationError = null   // ✅ NOUVEAU: Callback pour erreur
}) => {
    // ... Le reste du code

```


4. Modifier src/components/ConfirmationReservation.jsx pour accepter les props

javascript

```
// ConfirmationReservation.jsx
import React from 'react';
import './ConfirmationReservation.css';

const ConfirmationReservation = ({
  reservationData = {},
  onRetourAccueil = () => {}
}) => {
  const {
    id_reservation = 'RES-' + Math.random().toString(36).substr(2, 8).toUpperCase(),
    date = 'Date non spécifiée',
    formation = 'Formation non spécifiée',
    prenom = 'Prénom',
    nom = 'Nom',
    email = 'email@exemple.com'
  } = reservationData;

  return (
    <div className="confirmation-container">
      <div className="confirmation-content">
        <div className="confirmation-card">
          <div className="confirmation-header">
            <div className="success-icon">✓</div>
            <h1>Réservation Confirmée !</h1>
            <p className="subtitle">Votre inscription a été enregistrée avec succès</p>
          </div>

          <div className="confirmation-details">
            <div className="reservation-info">
              <h3>Détails de votre réservation</h3>
              <div className="info-grid">
                <div className="info-item">
                  <span className="info-label">Numéro de réservation :</span>
                  <span className="info-value reservation-id">{id_reservation}</span>
                </div>
                <div className="info-item">
                  <span className="info-label">Formation :</span>

```

```
<span className="info-value">{formation}</span>
</div>
<div className="info-item">
  <span className="info-label">Date :</span>
  <span className="info-value">{date}</span>
</div>
<div className="info-item">
  <span className="info-label">Participant :</span>
  <span className="info-value">{prenom} {nom}</span>
</div>
<div className="info-item">
  <span className="info-label">Email :</span>
  <span className="info-value">{email}</span>
</div>
</div>
</div>

<div className="next-steps">
  <div className="steps-card">
    <h3>Prochaines étapes</h3>
    <div className="steps-list">
      <div className="step">
        <span className="step-number">1</span>
        <span className="step-text">
          <strong>Email de confirmation</strong> - Vous recevrez un email
de confirmation dans les prochaines minutes
        </span>
      </div>
      <div className="step">
        <span className="step-number">2</span>
        <span className="step-text">
          <strong>Préparation</strong> - Aucun matériel spécifique n'est
requis pour cette formation
        </span>
      </div>
      <div className="step">
        <span className="step-number">3</span>
        <span className="step-text">
          <strong>Rappel</strong> - Un rappel vous sera envoyé 24h avant
la formation
        </span>
      </div>
    </div>
  </div>
</div>
</div>
```

```

    </div>
  </div>

  <div className="confirmation-footer">
    <button className="return-button" onClick={onRetourAccueil}>
      ← Retour à l'accueil
    </button>
    <p className="support-note">
      Besoin d'aide ? Contactez-nous à <strong>support@formations.fr</strong>
    </p>
  </div>
</div>
</div>
</div>
);
};

export default ConfirmationReservation;

```

5. Modifier src/components/ErreurReservation.jsx pour accepter les props

```

javascript

// ErreurReservation.jsx
import React from 'react';
import './ErreurReservation.css';

const ErreurReservation = ({
  errorData = {},
  onRetourAccueil = () => {}
}) => {
  const {
    message = 'Une erreur technique est survenue',
    formation = 'Formation non spécifiée',
    date = 'Date non spécifiée'
  } = errorData;

  return (
    <div className="erreur-container">
      <div className="erreur-content">
        <div className="erreur-card">

```

```

<div className="erreur-header">
  <div className="error-icon">X</div>
  <h1>Erreur de Réservation</h1>
  <p className="subtitle">Nous n'avons pas pu traiter votre demande</p>
</div>

<div className="erreur-details">
  <div className="error-message-section">
    <div className="error-alert">
      <h3>Détails de l'erreur</h3>
      <p className="error-text">{message}</p>
      <div className="reservation-context">
        <p><strong>Formation :</strong> {formation}</p>
        <p><strong>Date sélectionnée :</strong> {date}</p>
      </div>
      <div className="error-code">
        Référence : <span>ERR-{Math.random().toString(36).substr(2, 8).toUp
perCase()}</span>
      </div>
    </div>
  </div>
</div>

<div className="support-section">
  <div className="info-box">
    <h3>Assistance technique</h3>
    <p>
      Notre équipe de support est disponible pour vous aider à résoudre c
e problème.
    </p>
    <div className="contact-info">
      <div className="contact-item">
        <span className="contact-icon">📞</span>
        <span className="contact-details">
          <strong>Téléphone :</strong> +33 1 84 88 88 88
        </span>
      </div>
      <div className="contact-item">
        <span className="contact-icon">✉️</span>
        <span className="contact-details">
          <strong>Email :</strong> support@formations-tech.fr
        </span>
      </div>
      <div className="contact-item">
        <span className="contact-icon">🕒</span>

```



```
sql
```

```
ALTER TABLE reservation
DROP CONSTRAINT IF EXISTS telephone_check;
```

2. Ensuite, créer la nouvelle contrainte

```
sql
```

```
ALTER TABLE reservation
ADD CONSTRAINT telephone_check
CHECK (telephone BETWEEN 0000000000 AND 9999999999);
```

3. Version complète en une seule transaction (recommandée)

```
sql
```

```
BEGIN;

-- Supprimer l'ancienne contrainte si elle existe
ALTER TABLE reservation
DROP CONSTRAINT IF EXISTS telephone_check;

-- Ajouter la nouvelle contrainte avec la plage étendue
ALTER TABLE reservation
ADD CONSTRAINT telephone_check
CHECK (telephone BETWEEN 0000000000 AND 9999999999);

COMMIT;
```

4. Vérification après modification

```
sql
```

```
-- Vérifier que la contrainte a été modifiée
SELECT
    conname as constraint_name,
    pg_get_constraintdef(oid) as constraint_definition
```

```
FROM pg_constraint
WHERE conname = 'telephone_check'
AND conrelid = 'reservation'::regclass;
```

5. Test avec des numéros valides

sql

```
-- Ces insertions devraient maintenant fonctionner :
INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
VALUES
    ('reservation-test1', 'Jean', 'Dupont', 'jean@test.com', 99999999999), -- 11 chiffres
    ('reservation-test2', 'Marie', 'Martin', 'marie@test.com', 12345678901); -- 11 chiffres
```

Explication des changements :

- **Ancienne plage** : 0000000000 à 9999999999 (10 chiffres)
- **Nouvelle plage** : 0000000000 à 99999999999 (11 chiffres)

Cela permettra maintenant d'accepter les numéros de téléphone avec 11 chiffres, ce qui couvre les formats internationaux comme :

- 33123456789 (France avec indicatif)
- 14161234567 (Canada/États-Unis)
- Et tous les autres numéros jusqu'à 11 chiffres

Note : Si vous avez des numéros existants dans votre base qui étaient rejetés par l'ancienne contrainte, vous devrez les mettre à jour après cette modification.

J'ai oublié de te mentionner que dans la table de réservation j'ai une contrainte ou plutôt un Trigger qui choisit automatiquement le ID de la réservation lorsqu'une réservation est update ou insérer. Je pense que c'est la cause de l'erreur suivante lorsque j'essaie de réserver et que je me retrouve sur la page d'erreur de réservation,

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend\$ npm start

```
> start
> node server.js
```

[dotenv@17.2.3] injecting env (6) from .env -- tip:  audit secrets and track compliance: <https://dotenvx.com/ops>

Serveur démarré sur le port 3000

 Recherche instances pour formation: formation9

 Recherche instances pour formation: formation9

Erreur lors de l'inscription: error: insert or update on table

"table_reservation" violates foreign key constraint

"association_reservation"

at

/mnt/c/Users/nicol/Desktop/softghost/Backend/node_modules/pg-pool/index.js:45:11

at process.processTicksAndRejections

(node:internal/process/task_queues:105:5)

at async Object.inscrireFormation

(file:///mnt/c/Users/nicol/Desktop/softghost/Backend/reservationController.js:105:5)

at async

file:///mnt/c/Users/nicol/Desktop/softghost/Backend/routes/formations.js:71:27 {

length: 312,

severity: 'ERROR',

code: '23503',

detail: 'Key (id_reservation)=(reservation1762393804597) is not present in table "reservation".',

hint: undefined,

position: undefined,

internalPosition: undefined,

internalQuery: undefined,

where: undefined,

schema: 'public',

table: 'table_reservation',

column: undefined,

dataType: undefined,

constraint: 'association_reservation',

file: 'ri_triggers.c',

line: '2772',


```

    routine: 'ri_ReportViolation'
  }
^C
wizcolas@Purple-
Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend$

```

Vous avez raison ! L'erreur vient du fait que vous avez un trigger qui génère automatiquement l'ID de réservation, mais notre code essaie de l'insérer manuellement. Voici la correction :

1. Modifier reservationController.js - Ne pas insérer l'ID manuellement

javascript

```

// Backend/controllers/reservationController.js

// Fonction pour inscrire un utilisateur à une formation
async function inscrireFormation(reservationData) {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = reservationData;

    // Vérifier que l'instance a des places disponibles
    const checkQuery = `
      SELECT
        fi.id_formation_instance,
        COALESCE(tr.reservation_count, 0) as reservations_count
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.id_formation_instance = $1
      AND (COALESCE(tr.reservation_count, 0) < 30)
    `;

    const checkResult = await pool.query(checkQuery, [id_formation_instance]);

    if (checkResult.rows.length === 0) {

```

```
        throw new Error('Cette session est complète ou n\'existe pas');
    }

    // Commencer une transaction
    await pool.query('BEGIN');

    // 1. Créer la réservation (sans spécifier l'ID - Le trigger le génère automatiquement)
    const insertReservationQuery = `
        INSERT INTO reservation (prenom, nom, courriel, telephone)
        VALUES ($1, $2, $3, $4)
        RETURNING id_reservation
    `;

    const reservationResult = await pool.query(insertReservationQuery, [
        prenom, nom, courriel, telephone || null
    ]);

    const idReservation = reservationResult.rows[0].id_reservation;

    // 2. Lier la réservation à l'instance
    const insertTableReservationQuery = `
        INSERT INTO table_reservation (id_reservation, id_formation_instance)
        VALUES ($1, $2)
    `;

    await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

    // Valider la transaction
    await pool.query('COMMIT');

    return idReservation;
} catch (error) {
    // Annuler la transaction en cas d'erreur
    await pool.query('ROLLBACK');
    console.error('Erreur lors de l\'inscription:', error);
    throw error;
}
```

2. Vérifier la structure de votre trigger

Pour comprendre comment votre trigger fonctionne, vous pouvez vérifier sa définition :

```
sql

-- Voir les triggers sur la table reservation
SELECT
    tgrname as trigger_name,
    pg_get_triggerdef(oid) as trigger_definition
FROM pg_trigger
WHERE tgrelid = 'reservation'::regclass;

-- Ou voir les fonctions de trigger
SELECT
    proname as function_name,
    prosrc as function_source
FROM pg_proc
WHERE proname LIKE '%reservation%';
```

3. Si vous voulez désactiver temporairement le trigger pour tester

```
sql

-- Désactiver le trigger
ALTER TABLE reservation DISABLE TRIGGER nom_du_trigger;

-- Réactiver le trigger
ALTER TABLE reservation ENABLE TRIGGER nom_du_trigger;
```

4. Alternative : Si le trigger attend des valeurs spécifiques

Si votre trigger s'attend à ce que l'ID soit NULL ou une valeur spécifique, modifiez l'insertion :

```
javascript
```

```
// Alternative si le trigger a besoin de NULL
const insertReservationQuery = `
  INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
  VALUES (DEFAULT, $1, $2, $3, $4) -- Utiliser DEFAULT pour laisser le trigger faire
  son travail
  RETURNING id_reservation
`;
```

5. Test de la réservation corrigée

Avec la correction, le processus sera :

1. **Insertion dans** `reservation` : Sans spécifier l'ID, le trigger génère automatiquement l'ID
2. **Récupération de l'ID** : Via `RETURNING id_reservation`
3. **Insertion dans** `table_reservation` : Avec le bon ID récupéré

Redémarrez votre serveur backend et testez à nouveau la réservation. L'erreur de clé étrangère ne devrait plus se produire puisque l'ID sera correctement généré par le trigger et récupéré pour l'insertion dans `table_reservation`.

Voici ce que j'obtiens lorsque je regarde les triggers avec la commande :

```
-- Voir les triggers sur la table reservation
SELECT
  tgname as trigger_name,
  pg_get_triggerdef(oid) as trigger_definition
FROM pg_trigger
WHERE tgrelid = 'reservation'::regclass;
```

sous forme de Json :

```
[
  {
    "trigger_name": "RI_ConstraintTrigger_a_16503",
    "trigger_definition": "CREATE CONSTRAINT TRIGGER
  \"RI_ConstraintTrigger_a_16503\" AFTER DELETE ON
  public.reservation FROM table_reservation NOT DEFERRABLE
  INITIALLY IMMEDIATE FOR EACH ROW EXECUTE FUNCTION
```

```

\ "RI_FKey_noaction_del\()"
},
{
  "trigger_name": "RI_ConstraintTrigger_a_16504",
  "trigger_definition": "CREATE CONSTRAINT TRIGGER
\ "RI_ConstraintTrigger_a_16504\ " AFTER UPDATE ON
public.reservation FROM table_reservation NOT DEFERRABLE
INITIALLY IMMEDIATE FOR EACH ROW EXECUTE FUNCTION
\ "RI_FKey_noaction_upd\()"
},
{
  "trigger_name": "trg_generate_reservation_id",
  "trigger_definition": "CREATE TRIGGER
trg_generate_reservation_id BEFORE INSERT ON public.reservation
FOR EACH ROW WHEN ((new.id_reservation IS NULL)) EXECUTE
FUNCTION generate_reservation_id()"
},
{
  "trigger_name": "trg_set_reservation_id",
  "trigger_definition": "CREATE TRIGGER trg_set_reservation_id
BEFORE INSERT ON public.reservation FOR EACH ROW EXECUTE
FUNCTION set_reservation_id()"
}
]

```

Parfait ! Je vois que vous avez **deux triggers** qui s'exécutent sur l'insertion dans la table reservation :

1. trg_generate_reservation_id - Se déclenche quand id_reservation IS NULL
2. trg_set_reservation_id - Se déclenche sur TOUTE insertion

Le problème est que notre code essaie d'insérer un ID manuellement, mais les triggers veulent le générer automatiquement. Voici la correction :

1. Modifier reservationController.js - Laisser les triggers générer l'ID

javascript

```
// Backend/controllers/reservationController.js

// Fonction pour inscrire un utilisateur à une formation
async function inscrireFormation(reservationData) {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = reservationData;

    // Vérifier que l'instance a des places disponibles
    const checkQuery = `
      SELECT
        fi.id_formation_instance,
        COALESCE(tr.reservation_count, 0) as reservations_count
      FROM formation_instance fi
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.id_formation_instance = $1
      AND (COALESCE(tr.reservation_count, 0) < 30)
    `;

    const checkResult = await pool.query(checkQuery, [id_formation_instance]);

    if (checkResult.rows.length === 0) {
      throw new Error('Cette session est complète ou n\'existe pas');
    }

    // Commencer une transaction
    await pool.query('BEGIN');

    // 1. Créer la réservation - NE PAS fournir l'ID, laisser les triggers le générer
    const insertReservationQuery = `
      INSERT INTO reservation (prenom, nom, courriel, telephone)
      VALUES ($1, $2, $3, $4)
      RETURNING id_reservation
    `;

    const reservationResult = await pool.query(insertReservationQuery, [
      prenom, nom, courriel, telephone || null
    ]);
  }
}
```

```

const idReservation = reservationResult.rows[0].id_reservation;
console.log('✅ ID de réservation généré:', idReservation);

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `
  INSERT INTO table_reservation (id_reservation, id_formation_instance)
  VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');

return idReservation;

} catch (error) {
  // Annuler la transaction en cas d'erreur
  await pool.query('ROLLBACK');
  console.error('❌ Erreur lors de l'inscription:', error);
  throw error;
}
}

```

2. Vérifier les fonctions de trigger

Pour mieux comprendre, regardons les fonctions de trigger :

```

sql

-- Voir la fonction generate_reservation_id
SELECT proname, prosrc
FROM pg_proc
WHERE proname = 'generate_reservation_id';

-- Voir la fonction set_reservation_id
SELECT proname, prosrc
FROM pg_proc
WHERE proname = 'set_reservation_id';

```

3. Si les triggers créent un conflit

Si vous avez toujours des problèmes à cause des deux triggers, vous pouvez temporairement désactiver un des deux :

sql

```
-- Désactiver temporairement un trigger (choisir un des deux)
ALTER TABLE reservation DISABLE TRIGGER trg_set_reservation_id;
-- OU
ALTER TABLE reservation DISABLE TRIGGER trg_generate_reservation_id;

-- Pour réactiver plus tard :
ALTER TABLE reservation ENABLE TRIGGER trg_set_reservation_id;
ALTER TABLE reservation ENABLE TRIGGER trg_generate_reservation_id;
```

4. Test de debug

Ajoutez des logs pour voir ce qui se passe :

javascript

```
// Dans reservationController.js - version avec debug
console.log('🚧 Tentative d insertion dans reservation avec:', { prenom, nom, courriel, telephone: telephone || 'non fourni' });

const reservationResult = await pool.query(insertReservationQuery, [
  prenom, nom, courriel, telephone || null
]);

console.log('✅ Résultat de l insertion:', reservationResult.rows[0]);
```

5. Solution alternative si les triggers persistent

Si les triggers continuent de poser problème, vous pouvez essayer d'insérer avec DEFAULT :

javascript


```
// Alternative - utiliser DEFAULT pour l'ID
const insertReservationQuery = `
  INSERT INTO reservation (id_reservation, prenom, nom, courriel, telephone)
  VALUES (DEFAULT, $1, $2, $3, $4)
  RETURNING id_reservation
`;
```

La clé est de ne PAS fournir d'ID manuellement - laissez les triggers PostgreSQL le générer automatiquement, puis récupérez-le avec `RETURNING id_reservation`.

Testez maintenant - les réservations devraient fonctionner correctement !

Bon maintenant ce que je veux que tu fasses c'est que tu fasses en sorte que seul l'administrateur puisse accéder à la page d'administrateur. La page d'administrateur ressemble à ceci :

```
// PageAdministrateur.jsx
import React, { useState } from 'react';
import './PageAdministrateur.css';

const PageAdministrateur = ({
  formations = [
    {
      id: "formation-1",
      nom: "Formation React Avancé",
      date: "2024-02-15",
      heureDebut: "14:00",
      inscrits: 28
    },
    {
      id: "formation-2",
      nom: "Formation JavaScript Moderne",
      date: "2024-02-20",
      heureDebut: "10:00",
      inscrits: 15
    },
    {
      id: "formation-3",
      nom: "Formation Node.js & Express",
      date: "2024-02-10",
```

```
    heureDebut: "09:00",
    inscrits: 30
  },
  {
    id: "formation-4",
    nom: "Formation TypeScript",
    date: "2024-02-25",
    heureDebut: "16:00",
    inscrits: 22
  }
],
onRetourAccueil = () => {
  console.log('Retour à l\'accueil');
}
}) => {
  const [filtre, setFiltre] = useState('toutes'); // 'toutes', 'disponibles',
  'pleines'

  // Trier les formations par date (du plus récent au plus tard)
  const formationsTriees = [...formations].sort((a, b) => new
  Date(a.date) - new Date(b.date));

  // Filtrer les formations selon le filtre sélectionné
  const formationsFiltrees = formationsTriees.filter(formation => {
    if (filtre === 'pleines') {
      return formation.inscrits >= 30;
    } else if (filtre === 'disponibles') {
      return formation.inscrits < 30;
    }
    return true; // 'toutes'
  });

  const formatDate = (dateStr) => {
    const date = new Date(dateStr);
    return date.toLocaleDateString('fr-FR', {
      weekday: 'long',
      year: 'numeric',
      month: 'long',
      day: 'numeric'
    });
  });
```

```
};
```

```
const getStatutFormation = (inscrits) => {  
  if (inscrits >= 30) {  
    return { texte: 'Formation pleine', classe: 'pleine' };  
  } else if (inscrits >= 25) {  
    return { texte: 'Presque pleine', classe: 'presque-pleine' };  
  } else {  
    return { texte: 'Places disponibles', classe: 'disponible' };  
  }  
};
```

```
const getPourcentageInscription = (inscrits) => {  
  return Math.min((inscrits / 30) * 100, 100);  
};
```

```
return (  
  <div className="admin-container">  
    <div className="admin-content">  
      {/* Header */}  
      <header className="admin-header">  
        <div className="header-top">  
          <button className="back-button" onClick=  
{onRetourAccueil}>  
            ← Retour à l'accueil  
          </button>  
          <h1 className="admin-title">Espace Administrateur</h1>  
        </div>  
  
        {/* Statistiques */}  
        <div className="admin-stats">  
          <div className="stat-card">  
            <div className="stat-number">{formations.length}</div>  
            <div className="stat-label">Formations totales</div>  
          </div>  
          <div className="stat-card">  
            <div className="stat-number">  
              {formations.filter(f => f.inscrits >= 30).length}  
            </div>  
            <div className="stat-label">Formations pleines</div>
```

```

</div>
<div className="stat-card">
  <div className="stat-number">
    {formations.reduce((total, f) => total + f.inscrits, 0)}
  </div>
  <div className="stat-label">Total inscriptions</div>
</div>
</div>
</header>

{/* Filtres */}
<div className="filtres-section">
  <h2>Filtrer les formations</h2>
  <div className="filtres-buttons">
    <button
      className={`filtre-btn ${filtre === 'toutes' ? 'active' : ''}`}
      onClick={() => setFiltre('toutes')}
    >
      Toutes les formations
    </button>
    <button
      className={`filtre-btn ${filtre === 'disponibles' ? 'active' : ''}`}
      onClick={() => setFiltre('disponibles')}
    >
      Places disponibles
    </button>
    <button
      className={`filtre-btn ${filtre === 'pleines' ? 'active' : ''}`}
      onClick={() => setFiltre('pleines')}
    >
      Formations pleines
    </button>
  </div>
</div>

{/* Liste des formations */}
<main className="admin-main">
  <h2>Liste des formations</h2>

```

```

{formationsFiltrees.length === 0 ? (
  <div className="no-formations">
    <p>Aucune formation ne correspond aux critères
sélectionnés.</p>
  </div>
) : (
  <div className="formations-list">
    {formationsFiltrees.map((formation) => {
      const statut = getStatutFormation(formation.inscrits);
      const pourcentage =
getPourcentageInscription(formation.inscrits);

      return (
        <div key={formation.id} className="formation-admin-
card">
          <div className="formation-header">
            <h3 className="formation-nom">{formation.nom}
</h3>

            <span className={`statut-badge ${statut.classe}`}>
              {statut.texte}
            </span>
          </div>

          <div className="formation-details">
            <div className="detail-item">
              <span className="detail-label"> 📅 Date :</span>
              <span className="detail-value">
{formatDate(formation.date)}</span>
            </div>
            <div className="detail-item">
              <span className="detail-label"> 🕒 Heure :</span>
              <span className="detail-value">
{formation.heureDebut}</span>
            </div>
            <div className="detail-item">
              <span className="detail-label"> 👤 Inscrits :</span>
              <span className="detail-value">
                {formation.inscrits} / 30 personnes
              </span>
            </div>
          </div>
        </div>
      );
    })}
  </div>
)

```

```
</div>

    {/ * Barre de progression */}
    <div className="progression-section">
        <div className="progression-bar">
            <div
                className={`progression-remplissage
${statut.classe}`}
                style={{ width: `${pourcentage}%` }}
            ></div>
        </div>
        <div className="progression-text">
            {formation.inscrits} inscrits
            ({Math.round(pourcentage)}%)
        </div>
    </div>

    {/ * Actions */}
    <div className="formation-actions">
        <button className="action-btn details-btn">
            Voir les détails
        </button>
        <button className="action-btn export-btn">
            Exporter la liste
        </button>
    </div>
</div>
    );
    }}}
</div>
    })
</main>
</div>
</div>
);
};

export default PageAdministrateur;
```

et voici la page d'accueil duquel on peut accéder à la page administrateur :

```
// AccueilFormations.jsx
import React, { useState } from 'react';
import './AccueilFormations.css';

const AccueilFormations = ({
  formations = {
    "formation-1": {
      titre: "Formation React Avancé",
      images: ["/images/react1.jpg", "/images/react2.jpg"]
    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  }
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: "",
    password: ""
  });

  const handleFormationClick = (formationId) => {
    if (onFormationSelect) {
      onFormationSelect(formationId);
    }
  };

  const handleAdminLogin = (e) => {
```

```

e.preventDefault();
// Simulation de connexion admin
console.log('Tentative de connexion admin:', adminCredentials);
if (adminCredentials.username && adminCredentials.password) {
  alert('Connexion administrateur réussie !');
  setShowAdminLogin(false);
  setAdminCredentials({ username: '', password: '' });
} else {
  alert('Veuillez entrer un nom d'utilisateur et un mot de passe');
}
};

```

```

const handleAdminInputChange = (e) => {
  const { name, value } = e.target;
  setAdminCredentials(prev => ({
    ...prev,
    [name]: value
  }));
};

```

```

return (
  <div className="accueil-container">
    { /* Header avec bouton admin */ }
    <header className="accueil-header">
      <div className="header-content">
        <h1 className="accueil-title">Formations en Ligne</h1>
        <button
          className="admin-button"
          onClick={() => setShowAdminLogin(true)}
        >
           Administrateur
        </button>
      </div>
    </header>

    { /* Contenu principal */ }
    <main className="accueil-main">
      <div className="formations-grid">
        {Object.entries(formations).map(([formationId, formation]) =>
          (

```



```

<div
  key={formationId}
  className="formation-card"
  onClick={() => handleFormationClick(formationId)}
>
  <div className="formation-image-container">
    <img
      src={formation.images[0]}
      alt={formation.titre}
      className="formation-image"
      onError={(e) => {
        e.target.src =
`https://via.placeholder.com/300x200/667eea/ffffff?
text=${encodeURIComponent(formation.titre)}`;
      }}
    />
  </div>
  <div className="formation-info">
    <h3 className="formation-titre">{formation.titre}</h3>
    <div className="formation-overlay">
      <span className="voir-details">Voir les détails
→</span>
    </div>
  </div>
</div>
)))
</div>

{Object.keys(formations).length === 0 && (
  <div className="no-formations">
    <p>Aucune formation disponible pour le moment.</p>
  </div>
)}
</main>

{/* Modal de connexion admin */}
{showAdminLogin && (
  <div className="admin-modal-overlay">
    <div className="admin-modal">
      <div className="admin-modal-header">

```

```
<h2>Connexion Administrateur</h2>
<button
  className="close-button"
  onClick={() => setShowAdminLogin(false)}
>
  ×
</button>
</div>
```

```
<form className="admin-form" onSubmit=
{handleAdminLogin}>
  <div className="form-group">
    <label htmlFor="username">Nom d'utilisateur</label>
    <input
      type="text"
      id="username"
      name="username"
      value={adminCredentials.username}
      onChange={handleAdminInputChange}
      placeholder="Entrez votre nom d'utilisateur"
      required
    />
  </div>
```

```
<div className="form-group">
  <label htmlFor="password">Mot de passe</label>
  <input
    type="password"
    id="password"
    name="password"
    value={adminCredentials.password}
    onChange={handleAdminInputChange}
    placeholder="Entrez votre mot de passe"
    required
  />
</div>
```

```
<div className="admin-form-buttons">
  <button type="submit" className="login-button">
    Se connecter
```



```

    },
    "formation-2": {
      titre: "Formation JavaScript Moderne",
      images: ["/images/js1.jpg", "/images/js2.jpg"]
    },
    "formation-3": {
      titre: "Formation Node.js & Express",
      images: ["/images/node1.jpg", "/images/node2.jpg"]
    }
  },
  onFormationSelect = (formationId) => {
    console.log('Formation sélectionnée:', formationId);
  },
  onAdminLogin = () => {
    console.log('Connexion admin réussie');
  }
}) => {
  const [showAdminLogin, setShowAdminLogin] = useState(false);
  const [adminCredentials, setAdminCredentials] = useState({
    username: '',
    password: ''
  });
  const [loginError, setLoginError] = useState('');

  const handleFormationClick = (formationId) => {
    if (onFormationSelect) {
      onFormationSelect(formationId);
    }
  };

  const handleAdminLogin = async (e) => {
    e.preventDefault();
    setLoginError('');

    // Vérification des identifiants
    if (adminCredentials.username === 'tessiern782@gmail.com' && adminCredentials.password === 'Password1') {
      console.log('✅ Connexion admin réussie !');
      setShowAdminLogin(false);
      setAdminCredentials({ username: '', password: '' });

      // Appeler le callback parent pour naviguer vers la page admin
      if (onAdminLogin) {
        onAdminLogin();
      }
    }
  };
}

```

```

    }
  } else {
    setLoginError('Identifiants incorrects. Utilisez tessiern782@gmail.com / Passwo
rd1');
  }
};

const handleAdminInputChange = (e) => {
  const { name, value } = e.target;
  setAdminCredentials(prev => ({
    ...prev,
    [name]: value
  }));
  // Effacer l'erreur quand l'utilisateur tape
  if (loginError) setLoginError('');
};

return (
  <div className="accueil-container">
    {/* Header avec bouton admin */}
    <header className="accueil-header">
      <div className="header-content">
        <h1 className="accueil-title">Formations en Ligne</h1>
        <button
          className="admin-button"
          onClick={() => setShowAdminLogin(true)}
        >
           Administrateur
        </button>
      </div>
    </header>

    {/* Contenu principal */}
    <main className="accueil-main">
      <div className="formations-grid">
        {Object.entries(formations).map(([formationId, formation]) => (
          <div
            key={formationId}
            className="formation-card"
            onClick={() => handleFormationClick(formationId)}
          >
            <div className="formation-image-container">
              <img
                src={formation.images[0]}

```

```

        alt={formation.titre}
        className="formation-image"
        onError={(e) => {
            e.target.src = `https://via.placeholder.com/300x200/667eea/ffffff
f?text=${encodeURIComponent(formation.titre)}`;
        }}
    />
</div>
<div className="formation-info">
    <h3 className="formation-titre">{formation.titre}</h3>
    <div className="formation-overlay">
        <span className="voir-details">Voir les détails →</span>
    </div>
</div>
</div>
    )})
</div>

```

```

{Object.keys(formations).length === 0 && (
    <div className="no-formations">
        <p>Aucune formation disponible pour le moment.</p>
    </div>
)}
</main>

```

```

{/* Modal de connexion admin */}
{showAdminLogin && (
    <div className="admin-modal-overlay">
        <div className="admin-modal">
            <div className="admin-modal-header">
                <h2>Connexion Administrateur</h2>
                <button
                    className="close-button"
                    onClick={() => {
                        setShowAdminLogin(false);
                        setLoginError('');
                        setAdminCredentials({ username: '', password: '' });
                    }}
                >
                    x
                </button>
            </div>

            <form className="admin-form" onSubmit={handleAdminLogin}>

```

```
<div className="form-group">
  <label htmlFor="username">Nom d'utilisateur</label>
  <input
    type="email"
    id="username"
    name="username"
    value={adminCredentials.username}
    onChange={handleAdminInputChange}
    placeholder="tessiern782@gmail.com"
    required
  />
</div>

<div className="form-group">
  <label htmlFor="password">Mot de passe</label>
  <input
    type="password"
    id="password"
    name="password"
    value={adminCredentials.password}
    onChange={handleAdminInputChange}
    placeholder="Password1"
    required
  />
</div>

{loginError && (
  <div className="login-error">
    {loginError}
  </div>
)}

<div className="admin-form-buttons">
  <button type="submit" className="login-button">
    Se connecter
  </button>
  <button
    type="button"
    className="cancel-button"
    onClick={() => {
      setShowAdminLogin(false);
      setLoginError('');
      setAdminCredentials({ username: '', password: '' });
    }}
  >
```

```

        >
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
)}
</div>
);
};

export default AccueilFormations;

```

2. Modifier App.jsx pour gérer la navigation admin

javascript

```

// App.jsx - Ajouter ces états et fonctions
const [isAdmin, setIsAdmin] = useState(false);

const handleAdminLogin = () => {
  setIsAdmin(true);
  setCurrentPage('admin');
};

const handleRetourAccueil = () => {
  setCurrentPage('accueil');
  setSelectedFormation(null);
  setReservationData(null);
  setIsAdmin(false); // Déconnecter l'admin quand il retourne à l'accueil
};

// Afficher la page admin si connecté
if (currentPage === 'admin' && isAdmin) {
  return (
    <PageAdministrateur
      onRetourAccueil={handleRetourAccueil}
    />
  );
}

// Dans le return de AccueilFormations

```



```

return (
  <AccueilFormations
    formations={formations}
    onFormationSelect={handleFormationSelect}
    onAdminLogin={handleAdminLogin} // ✅ Nouveau callback
  />
);

```

3. Créer une route API pour les données admin

Ajoutez dans Backend/routes/formations.js :

javascript

```

// Route pour récupérer les statistiques admin
router.get('/admin/statistiques', async (req, res) => {
  try {
    const query = `
      SELECT
        f.id_formation,
        f.nom,
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        COALESCE(tr.reservation_count, 0) as inscrits,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation f
      JOIN formation_instance fi ON f.id_formation = fi.id_reference_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.jour >= CURRENT_DATE
        AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
      ORDER BY fi.jour, f.heure_debut
    `;

    const result = await pool.query(query);

    const formations = result.rows.map(row => ({

```

```

        id: row.id_formation_instance,
        nom: row.nom,
        date: row.date_instance,
        heureDebut: row.heure_debut?.substring(0, 5),
        heureFin: row.heure_fin?.substring(0, 5),
        inscrits: parseInt(row.inscrits),
        places_restantes: parseInt(row.places_restantes),
        complet: row.complet
    }));

    res.json(formations);
  } catch (error) {
    console.error('Erreur récupération statistiques admin:', error);
    res.status(500).json({ error: error.message });
  }
});

```

4. Modifier PageAdministrateur.jsx pour utiliser les vraies données

javascript

```

// PageAdministrateur.jsx
import React, { useState, useEffect } from 'react';
import './PageAdministrateur.css';

const PageAdministrateur = ({
  onRetourAccueil = () => {
    console.log('Retour à l\'accueil');
  }
}) => {
  const [filtre, setFiltre] = useState('toutes');
  const [formations, setFormations] = useState([]);
  const [loading, setLoading] = useState(true);

  // Récupérer les vraies données depuis l'API
  useEffect(() => {
    const fetchFormationsAdmin = async () => {
      try {
        setLoading(true);
        const response = await fetch('http://localhost:3000/api/admin/statistiques');

```

```
    if (!response.ok) {
      throw new Error('Erreur lors du chargement des données');
    }

    const data = await response.json();
    console.log('📊 Données admin:', data);
    setFormations(data);
  } catch (error) {
    console.error('Erreur chargement données admin:', error);
    // Données de test en cas d'erreur
    setFormations([]);
  } finally {
    setLoading(false);
  }
};

fetchFormationsAdmin();
}, []);

// Trier les formations par date
const formationsTriees = [...formations].sort((a, b) => new Date(a.date) - new Date(b.date));

// Filtrer les formations selon le filtre sélectionné
const formationsFiltrees = formationsTriees.filter(formation => {
  if (filtre === 'pleines') {
    return formation.inscrits >= 30;
  } else if (filtre === 'disponibles') {
    return formation.inscrits < 30;
  }
  return true;
});

const formatDate = (dateStr) => {
  const date = new Date(dateStr);
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
};

const getStatutFormation = (inscrits) => {
```

```
if (inscrits >= 30) {
  return { texte: 'Formation pleine', classe: 'pleine' };
} else if (inscrits >= 25) {
  return { texte: 'Presque pleine', classe: 'presque-pleine' };
} else {
  return { texte: 'Places disponibles', classe: 'disponible' };
}
};

const getPourcentageInscription = (inscrits) => {
  return Math.min((inscrits / 30) * 100, 100);
};

if (loading) {
  return (
    <div className="admin-container">
      <div className="loading-admin">
        <p>Chargement des données administrateur...</p>
      </div>
    </div>
  );
}

return (
  <div className="admin-container">
    <div className="admin-content">
      { /* Header */ }
      <header className="admin-header">
        <div className="header-top">
          <button className="back-button" onClick={onRetourAccueil}>
            ← Retour à l'accueil
          </button>
          <h1 className="admin-title">Espace Administrateur</h1>
        </div>

        { /* Statistiques */ }
        <div className="admin-stats">
          <div className="stat-card">
            <div className="stat-number">{formations.length}</div>
            <div className="stat-label">Sessions ce mois</div>
          </div>
          <div className="stat-card">
            <div className="stat-number">
              {formations.filter(f => f.inscrits >= 30).length}
            </div>
          </div>
        </div>
      </header>
    </div>
  </div>
);
```

```

    </div>
    <div className="stat-label">Sessions pleines</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.reduce((total, f) => total + f.inscrits, 0)}
    </div>
    <div className="stat-label">Total inscriptions</div>
  </div>
</div>
</header>

{/* Filtres */}
<div className="filtres-section">
  <h2>Filtrer les sessions</h2>
  <div className="filtres-buttons">
    <button
      className={`filtre-btn ${filtre === 'toutes' ? 'active' : ''}`}
      onClick={() => setFiltre('toutes')}
    >
      Toutes les sessions
    </button>
    <button
      className={`filtre-btn ${filtre === 'disponibles' ? 'active' : ''}`}
      onClick={() => setFiltre('disponibles')}
    >
      Places disponibles
    </button>
    <button
      className={`filtre-btn ${filtre === 'pleines' ? 'active' : ''}`}
      onClick={() => setFiltre('pleines')}
    >
      Sessions pleines
    </button>
  </div>
</div>

{/* Liste des formations */}
<main className="admin-main">
  <h2>Sessions du mois prochain</h2>

  {formationsFiltrees.length === 0 ? (
    <div className="no-formations">
      <p>Aucune session ne correspond aux critères sélectionnés.</p>

```

```

    </div>
  ) : (
    <div className="formations-list">
      {formationsFiltrees.map((formation) => {
        const statut = getStatutFormation(formation.inscrits);
        const pourcentage = getPourcentageInscription(formation.inscrits);

        return (
          <div key={formation.id} className="formation-admin-card">
            <div className="formation-header">
              <h3 className="formation-nom">{formation.nom}</h3>
              <span className={`statut-badge ${statut.classe}`}>
                {statut.texte}
              </span>
            </div>

            <div className="formation-details">
              <div className="detail-item">
                <span className="detail-label">📅 Date :</span>
                <span className="detail-value">{formatDate(formation.date)}</span>
              </div>
              <div className="detail-item">
                <span className="detail-label">🕒 Heure :</span>
                <span className="detail-value">{formation.heureDebut} - {formation.heureFin}</span>
              </div>
              <div className="detail-item">
                <span className="detail-label">👤 Inscrits :</span>
                <span className="detail-value">
                  {formation.inscrits} / 30 personnes
                </span>
              </div>
            </div>

            <div className="progression-section">
              <div className="progression-bar">
                <div
                  className={`progression-remplissage ${statut.classe}`}
                  style={{ width: `${pourcentage}%` }}
                ></div>
              </div>
              <div className="progression-text">

```

```

        {formation.inscrits} inscrits ({Math.round(pourcentage)}%)
      </div>
    </div>

    {/* Actions */}
    <div className="formation-actions">
      <button className="action-btn details-btn">
        Voir les détails
      </button>
      <button className="action-btn export-btn">
        Exporter la liste
      </button>
    </div>
  </div>
);
}}
</div>
)}
</main>
</div>
</div>
);
};

export default PageAdministrateur;

```

5. Ajouter du CSS pour les erreurs de login

Ajoutez dans `AccueilFormations.css` :

css

```

.login-error {
  background: #fee;
  border: 1px solid #fcc;
  border-radius: 8px;
  padding: 12px;
  margin: 15px 0;
  color: #c33;
  font-size: 0.9em;
  text-align: center;
}

```

```
.loading-admin {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 50vh;
  font-size: 1.2em;
  color: #666;
}
```

Maintenant :

- Seul tessiern782@gmail.com / Password1 peut accéder à l'admin
- La page admin affiche les vraies données des instances du mois prochain
- Les statistiques sont calculées en temps réel depuis la base de données

Je ne comprends pas très bien ce que je dois modifier dans App.jsx.
Voici à quoi ressemble le fichier pour l'instant :

```
// src/App.jsx
import React, { useState, useEffect } from 'react';
import AccueilFormations from './components/AccueilFormations';
import DescriptionFormation from
'./components/DescriptionFormation';
import ConfirmationReservation from
'./components/ConfirmationReservation';
import ErreurReservation from './components/ErreurReservation';
import './App.css';

function App() {
  const [currentPage, setCurrentPage] = useState('accueil'); //
'accueil', 'description', 'confirmation', 'erreur'
  const [selectedFormation, setSelectedFormation] = useState(null);
  const [reservationData, setReservationData] = useState(null);
  const [formations, setFormations] = useState({});

  useEffect(() => {
    const fetchFormations = async () => {
```



```
try {
  console.log('🔄 Chargement des formations depuis l API...');
  const response = await
fetch('http://localhost:3000/api/formations');

  if (!response.ok) {
    throw new Error(`Erreur HTTP: ${response.status}`);
  }

  const data = await response.json();
  console.log('✅ Données brutes reçues:', data);

  const formationsFormatees = {};
  data.forEach(formation => {
    formationsFormatees[formation.id_formation] = {
      titre: formation.nom,
      images: formation.chemins_images.map(img =>
`http://localhost:3000/images/${img}`),
      description: formation.description,
      jour_semaine: formation.jour_semaine,
      heure_debut: formation.heure_debut,
      heure_fin: formation.heure_fin
    };
  });

  setFormations(formationsFormatees);

} catch (error) {
  console.error('❌ Erreur chargement formations:', error);
  setFormations({
    "formation-test": {
      titre: "Formation Test",
      images: ["https://via.placeholder.com/300x200/667eea/ffffff?
text=Formation+Test"],
      description: "Description de test",
      jour_semaine: 1,
      heure_debut: "14:00:00",
      heure_fin: "17:00:00"
    }
  });
}
```

```
    }  
  };  
  
  fetchFormations();  
}, []);  
  
const handleFormationSelect = (formationId) => {  
  console.log('Formation sélectionnée:', formationId);  
  setSelectedFormation(formationId);  
  setCurrentPage('description');  
};  
  
const handleReservationSuccess = (reservationInfo) => {  
  setReservationData(reservationInfo);  
  setCurrentPage('confirmation');  
};  
  
const handleReservationError = (errorInfo) => {  
  setReservationData(errorInfo);  
  setCurrentPage('erreur');  
};  
  
const handleRetourAccueil = () => {  
  setCurrentPage('accueil');  
  setSelectedFormation(null);  
  setReservationData(null);  
};  
  
// Afficher la page de confirmation  
if (currentPage === 'confirmation') {  
  return <ConfirmationReservation onRetourAccueil=  
{handleRetourAccueil} />;  
}  
  
// Afficher la page d'erreur  
if (currentPage === 'erreur') {  
  return <ErreurReservation onRetourAccueil=  
{handleRetourAccueil} />;  
}
```

```

// Afficher la page de description
if (currentPage === 'description' && selectedFormation &&
formations[selectedFormation]) {
  const formation = formations[selectedFormation];
  return (
    <DescriptionFormation
      titreEvenement={formation.titre}
      descriptionEvenement={formation.description}
      heureEvenement={`${formation.heure_debut?.substring(0, 5)} -
${formation.heure_fin?.substring(0, 5)}`}
      jourSemaine={formation.jour_semaine}
      images={formation.images}
      formationId={selectedFormation}
      onRetour={handleRetourAccueil}
      onReservationSuccess={handleReservationSuccess}
      onReservationError={handleReservationError}
    />
  );
}

// Sinon afficher la page d'accueil
return (
  <AccueilFormations
    formations={formations}
    onFormationSelect={handleFormationSelect}
  />
);
}

export default App;

```

Je vais vous montrer exactement comment modifier votre `App.jsx` pour ajouter la gestion de la page administrateur. Voici le fichier complet modifié :

src/App.jsx modifié

javascript

```

// src/App.jsx
import React, { useState, useEffect } from 'react';

```

```

import AccueilFormations from './components/AccueilFormations';
import DescriptionFormation from './components/DescriptionFormation';
import ConfirmationReservation from './components/ConfirmationReservation';
import ErreurReservation from './components/ErreurReservation';
import PageAdministrateur from './components/PageAdministrateur'; // ✅ AJOUT: Import de la page admin
import './App.css';

function App() {
  const [currentPage, setCurrentPage] = useState('accueil'); // 'accueil', 'description', 'confirmation', 'erreur', 'admin'
  const [selectedFormation, setSelectedFormation] = useState(null);
  const [reservationData, setReservationData] = useState(null);
  const [formations, setFormations] = useState({});
  const [isAdmin, setIsAdmin] = useState(false); // ✅ AJOUT: État pour vérifier si admin connecté

  useEffect(() => {
    const fetchFormations = async () => {
      try {
        console.log('🔄 Chargement des formations depuis l API...');
        const response = await fetch('http://localhost:3000/api/formations');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('✅ Données brutes reçues:', data);

        const formationsFormatees = {};
        data.forEach(formation => {
          formationsFormatees[formation.id_formation] = {
            titre: formation.nom,
            images: formation.chemins_images.map(img => `http://localhost:3000/images/${img}`),
            description: formation.description,
            jour_semaine: formation.jour_semaine,
            heure_debut: formation.heure_debut,
            heure_fin: formation.heure_fin
          };
        });

        setFormations(formationsFormatees);
      } catch (error) {
        console.error('Erreur lors du chargement des formations:', error);
      }
    };

    fetchFormations();
  });
}

```

```

    } catch (error) {
      console.error('✖ Erreur chargement formations:', error);
      setFormations({
        "formation-test": {
          titre: "Formation Test",
          images: ["https://via.placeholder.com/300x200/667eea/ffffff?text=Formatio
n+Test"],
          description: "Description de test",
          jour_semaine: 1,
          heure_debut: "14:00:00",
          heure_fin: "17:00:00"
        }
      });
    }
  };

  fetchFormations();
}, []);

const handleFormationSelect = (formationId) => {
  console.log('Formation sélectionnée:', formationId);
  setSelectedFormation(formationId);
  setCurrentPage('description');
};

// ✅ AJOUT: Gestion de la connexion admin
const handleAdminLogin = () => {
  console.log('🔑 Connexion admin réussie, navigation vers page admin');
  setIsAdmin(true);
  setCurrentPage('admin');
};

const handleReservationSuccess = (reservationInfo) => {
  setReservationData(reservationInfo);
  setCurrentPage('confirmation');
};

const handleReservationError = (errorInfo) => {
  setReservationData(errorInfo);
  setCurrentPage('erreur');
};

const handleRetourAccueil = () => {

```

```

    setCurrentPage('accueil');
    setSelectedFormation(null);
    setReservationData(null);
    setIsAdmin(false); // ✅ AJOUT: Déconnecter l'admin quand il retourne à l'accueil
  };

  // ✅ AJOUT: Afficher la page admin si connecté
  if (currentPage === 'admin' && isAdmin) {
    return (
      <PageAdministrateur
        onRetourAccueil={handleRetourAccueil}
      />
    );
  }

  // Afficher la page de confirmation
  if (currentPage === 'confirmation') {
    return <ConfirmationReservation onRetourAccueil={handleRetourAccueil} />;
  }

  // Afficher la page d'erreur
  if (currentPage === 'erreur') {
    return <ErreurReservation onRetourAccueil={handleRetourAccueil} />;
  }

  // Afficher la page de description
  if (currentPage === 'description' && selectedFormation && formations[selectedFormation]) {
    const formation = formations[selectedFormation];
    return (
      <DescriptionFormation
        titreEvenement={formation.titre}
        descriptionEvenement={formation.description}
        heureEvenement={` ${formation.heure_debut?.substring(0, 5)} - ${formation.heure_fin?.substring(0, 5)} `}
        jourSemaine={formation.jour_semaine}
        images={formation.images}
        formationId={selectedFormation}
        onRetour={handleRetourAccueil}
        onReservationSuccess={handleReservationSuccess}
        onReservationError={handleReservationError}
      />
    );
  }

```

```

    }

    // Sinon afficher la page d'accueil
    return (
      <AccueilFormations
        formations={formations}
        onFormationSelect={handleFormationSelect}
        onAdminLogin={handleAdminLogin} // ✅ AJOUT: Passer le callback pour l'admin
      />
    );
  }

  export default App;

```

Résumé des modifications :

1. Import ajouté :

```
import PageAdministrateur from './components/PageAdministrateur';
```

2. Nouvel état : `const [isAdmin, setIsAdmin] = useState(false);`

3. Nouvelle fonction : `handleAdminLogin()` pour gérer la connexion admin

4. Nouvelle condition : Affichage de la page admin si

```
currentPage === 'admin' && isAdmin
```

5. Modification de `handleRetourAccueil` : Déconnecte l'admin avec

```
setIsAdmin(false)
```

6. Callback ajouté : `onAdminLogin={handleAdminLogin}` passé à `AccueilFormations`

Fonctionnement :

- **Accueil** → Cliquez sur "🔒 Administrateur"
- **Modal de connexion** → Entrez `tessiern782@gmail.com` / `Password1`
- **Si identifiants corrects** → Redirection vers `PageAdministrateur`
- **Bouton "Retour à l'accueil"** dans l'admin → Retour à l'accueil et déconnexion

Testez maintenant - vous devriez pouvoir vous connecter en tant qu'admin et voir la page d'administration avec les vraies données !

Ok parfait cela fonctionne. Maintenant je voudrais juste que tu ajoutes à la page d'administrateur un filtre pour filtrer les formation_instance par rapport au type de formation dans la table formation. C'est à dire par rapport au logiciel qui est enseigner dans la formation

Je vais ajouter un filtre par type de formation (logiciel) à la page administrateur.

Voici les modifications :

1. Modifier PageAdministrateur.jsx

javascript

```
// PageAdministrateur.jsx
import React, { useState, useEffect } from 'react';
import './PageAdministrateur.css';

const PageAdministrateur = ({
  onRetourAccueil = () => {
    console.log('Retour à l\'accueil');
  }
}) => {
  const [filtreStatut, setFiltreStatut] = useState('toutes'); // 'toutes', 'disponibl
es', 'pleines'
  const [filtreFormation, setFiltreFormation] = useState('toutes'); // 'toutes' ou id
_formation spécifique
  const [formations, setFormations] = useState([]);
  const [typesFormation, setTypesFormation] = useState([]); // ✅ AJOUT: Liste des t
ypes de formation
  const [loading, setLoading] = useState(true);

  // Récupérer Les vraies données depuis L'API
  useEffect(() => {
    const fetchFormationsAdmin = async () => {
      try {
        setLoading(true);
        const response = await fetch('http://localhost:3000/api/admin/statistiques');

        if (!response.ok) {
          throw new Error('Erreur lors du chargement des données');
        }
      }
    }
  })
}
```



```

const data = await response.json();
console.log('📊 Données admin:', data);
setFormations(data);

// ✅ AJOUT: Extraire Les types de formation uniques
const typesUniques = [...new Set(data.map(item => ({
  id: item.id_formation,
  nom: item.nom
})))].filter(item => item.id && item.nom);

setTypesFormation(typesUniques);
console.log('🌀 Types de formation:', typesUniques);

} catch (error) {
  console.error('Erreur chargement données admin:', error);
  setFormations([]);
  setTypesFormation([]);
} finally {
  setLoading(false);
}
};

fetchFormationsAdmin();
}, []);

// Trier Les formations par date
const formationsTriees = [...formations].sort((a, b) => new Date(a.date) - new Date(b.date));

// ✅ MODIFIÉ: Filtrer Les formations selon Les filtres sélectionnés
const formationsFiltrees = formationsTriees.filter(formation => {
  // Filtre par statut (pleines/disponibles)
  let filtreStatutOk = true;
  if (filtreStatut === 'pleines') {
    filtreStatutOk = formation.inscrits >= 30;
  } else if (filtreStatut === 'disponibles') {
    filtreStatutOk = formation.inscrits < 30;
  }

  // ✅ AJOUT: Filtre par type de formation
  let filtreTypeOk = true;
  if (filtreFormation !== 'toutes') {
    filtreTypeOk = formation.id_formation === filtreFormation;
  }
});

```

```
    return filtreStatutOk && filtreTypeOk;
  });

const formatDate = (dateStr) => {
  const date = new Date(dateStr);
  return date.toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
};

const getStatutFormation = (inscrits) => {
  if (inscrits >= 30) {
    return { texte: 'Formation pleine', classe: 'pleine' };
  } else if (inscrits >= 25) {
    return { texte: 'Presque pleine', classe: 'presque-pleine' };
  } else {
    return { texte: 'Places disponibles', classe: 'disponible' };
  }
};

const getPourcentageInscription = (inscrits) => {
  return Math.min((inscrits / 30) * 100, 100);
};

if (loading) {
  return (
    <div className="admin-container">
      <div className="loading-admin">
        <p>Chargement des données administrateur...</p>
      </div>
    </div>
  );
}

return (
  <div className="admin-container">
    <div className="admin-content">
      { /* Header */ }
    <header className="admin-header">
      <div className="header-top">
```

```

<button className="back-button" onClick={onRetourAccueil}>
  ← Retour à l'accueil
</button>
<h1 className="admin-title">Espace Administrateur</h1>
</div>

{/* Statistiques */}
<div className="admin-stats">
  <div className="stat-card">
    <div className="stat-number">{formations.length}</div>
    <div className="stat-label">Sessions ce mois</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.filter(f => f.inscrits >= 30).length}
    </div>
    <div className="stat-label">Sessions pleines</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.reduce((total, f) => total + f.inscrits, 0)}
    </div>
    <div className="stat-label">Total inscriptions</div>
  </div>
</div>
</header>

{/* Filtres */}
<div className="filtres-section">
  <h2>Filtrer les sessions</h2>

  {/* ✅ AJOUT: Filtre par type de formation */}
  <div className="filtre-groupe">
    <label htmlFor="filtre-formation" className="filtre-label">
      📁 Type de formation :
    </label>
    <select
      id="filtre-formation"
      value={filtreFormation}
      onChange={(e) => setFiltreFormation(e.target.value)}
      className="filtre-select"
    >
      <option value="toutes">Tous les logiciels</option>
      {typesFormation.map((type) => (

```

```

      <option key={type.id} value={type.id}>
        {type.nom}
      </option>
    )}}
  </select>
</div>

{/* Filtre par statut */}
<div className="filtre-groupe">
  <label className="filtre-label">📊 Statut :</label>
  <div className="filtres-buttons">
    <button
      className={`filtre-btn ${filtreStatut === 'toutes' ? 'active' : ''}`}
      onClick={() => setFiltreStatut('toutes')}
    >
      Toutes les sessions
    </button>
    <button
      className={`filtre-btn ${filtreStatut === 'disponibles' ? 'active' : ''}`}
      onClick={() => setFiltreStatut('disponibles')}
    >
      Places disponibles
    </button>
    <button
      className={`filtre-btn ${filtreStatut === 'pleines' ? 'active' : ''}`}
      onClick={() => setFiltreStatut('pleines')}
    >
      Sessions pleines
    </button>
  </div>
</div>

{/* ✅ AJOUT: Résumé des filtres */}
<div className="filtres-resume">
  <p>
    Affichage de <strong>{formationsFiltrees.length}</strong> session(s)
    {filtreFormation !== 'toutes' &&
      ` pour "${typesFormation.find(t => t.id === filtreFormation)?.nom ||
filtreFormation}"`
    }
    {filtreStatut !== 'toutes' &&
      ` (${filtreStatut === 'pleines' ? 'pleines' : 'avec places disponible

```

```
s'}}`
```

```

    }
  </p>
</div>
</div>

{/* Liste des formations */}
<main className="admin-main">
  <h2>Sessions du mois prochain</h2>

  {formationsFiltrees.length === 0 ? (
    <div className="no-formations">
      <p>Aucune session ne correspond aux critères sélectionnés.</p>
      <button
        className="reset-filtres-btn"
        onClick={() => {
          setFiltreStatut('toutes');
          setFiltreFormation('toutes');
        }}
      >
        Réinitialiser les filtres
      </button>
    </div>
  ) : (
    <div className="formations-list">
      {formationsFiltrees.map((formation) => {
        const statut = getStatutFormation(formation.inscrits);
        const pourcentage = getPourcentageInscription(formation.inscrits);

        return (
          <div key={formation.id} className="formation-admin-card">
            <div className="formation-header">
              <h3 className="formation-nom">{formation.nom}</h3>
              <span className={`statut-badge ${statut.classe}`}>
                {statut.texte}
              </span>
            </div>

            <div className="formation-details">
              <div className="detail-item">
                <span className="detail-label">📅 Date :</span>
                <span className="detail-value">{formatDate(formation.date)}</span>
              </div>
            </div>
          </div>
        );
      })}
    </div>
  )}
</div>

```

```

<div className="detail-item">
  <span className="detail-label">🕒 Heure :</span>
  <span className="detail-value">{formation.heureDebut} - {form
ation.heureFin}</span>
</div>
<div className="detail-item">
  <span className="detail-label">👤 Inscrits :</span>
  <span className="detail-value">
    {formation.inscrits} / 30 personnes
  </span>
</div>
<div className="detail-item">
  <span className="detail-label">🆔 ID Formation :</span>
  <span className="detail-value formation-id">{formation.id_for
mation}</span>
</div>
</div>

{/* Barre de progression */}
<div className="progression-section">
  <div className="progression-bar">
    <div
      className={`progression-remplissage ${statut.classe}`}
      style={{ width: `${pourcentage}%` }}
    ></div>
  </div>
  <div className="progression-text">
    {formation.inscrits} inscrits ({Math.round(pourcentage)}%)
  </div>
</div>

{/* Actions */}
<div className="formation-actions">
  <button className="action-btn details-btn">
    Voir les détails
  </button>
  <button className="action-btn export-btn">
    Exporter la liste
  </button>
</div>
</div>
);
}}
</div>

```

```

    })
  </main>
</div>
</div>
);
};

export default PageAdministrateur;

```

2. Modifier la route API pour inclure id_formation

Assurez-vous que votre route `/api/admin/statistiques` retourne aussi `id_formation` :

javascript

```

// Dans Backend/routes/formations.js - route /api/admin/statistiques
const query = `
  SELECT
    f.id_formation, // ☒ S'assurer que cette colonne est incluse
    f.nom,
    fi.id_formation_instance,
    fi.jour as date_instance,
    f.heure_debut,
    f.heure_fin,
    COALESCE(tr.reservation_count, 0) as inscrits,
    (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
    (COALESCE(tr.reservation_count, 0) >= 30) as complet
  FROM formation f
  JOIN formation_instance fi ON f.id_formation = fi.id_reference_formation
  LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
  ) tr ON fi.id_formation_instance = tr.id_formation_instance
  WHERE fi.jour >= CURRENT_DATE
    AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
  ORDER BY fi.jour, f.heure_debut
`;

```

3. Ajouter du CSS pour les nouveaux filtres

Ajoutez dans PageAdministrateur.css :

css

```
/* Styles pour les nouveaux filtres */
.filtre-groupe {
    margin-bottom: 20px;
}

.filtre-label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    color: #333;
    font-size: 1em;
}

.filtre-select {
    padding: 10px 15px;
    border: 2px solid #e1e5e9;
    border-radius: 8px;
    font-size: 1em;
    background: white;
    cursor: pointer;
    transition: all 0.3s ease;
    min-width: 250px;
}

.filtre-select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.filtres-resume {
    margin-top: 15px;
    padding: 12px;
    background: #f8f9fa;
    border-radius: 8px;
    border-left: 4px solid #667eea;
}

.filtres-resume p {
    margin: 0;
```



```
    color: #495057;
    font-size: 0.95em;
}

.formation-id {
    font-family: 'Courier New', monospace;
    background: #f1f3f4;
    padding: 2px 6px;
    border-radius: 4px;
    font-size: 0.85em;
    color: #5f6368;
}

.reset-filtres-btn {
    background: #6c757d;
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 6px;
    cursor: pointer;
    font-size: 0.9em;
    margin-top: 10px;
    transition: background 0.3s ease;
}

.reset-filtres-btn:hover {
    background: #5a6268;
}

/* Responsive */
@media (max-width: 768px) {
    .filtre-select {
        min-width: 100%;
    }

    .filtres-buttons {
        flex-direction: column;
        gap: 10px;
    }

    .filtre-btn {
        width: 100%;
    }
}
```

```
}  
}
```

Maintenant vous pouvez :

- Filtrer par type de formation (Blender, Krita, DaVinci Resolve, etc.)
- Filtrer par statut (pleines/disponibles)
- Voir un résumé des filtres appliqués
- Réinitialiser facilement les filtres

La page admin affichera uniquement les sessions du logiciel sélectionné !

Je ne comprends pas trop ce qu'il faut modifier dans le fichier Backend/routes/formations.js. Pour l'instant il ressemble à :

```
// Backend/routes/formations.js  
import express from 'express';  
import formationController from '../formationController.js';  
import formationInstanceController from  
  '../formationInstanceController.js';  
import reservationController from '../reservationController.js';  
import pool from '../db.js'; //  AJOUT: Import du pool
```

```
const router = express.Router();
```

```
// Routes pour les formations  
router.get('/formations', async (req, res) => {  
  try {  
    const formations = await  
      formationController.obtenirToutesFormations();  
    res.json(formations);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
router.get('/formations/:id', async (req, res) => {  
  try {  
    const formation = await  
      formationController.obtenirFormationParId(req.params.id);
```

```
    if (!formation) {  
      return res.status(404).json({ error: 'Formation non trouvée' });  
    }  
    res.json(formation);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
// Routes pour les instances  
router.get('/instances', async (req, res) => {  
  try {  
    const instances = await  
formationInstanceController.obtenirInstancesDetaillees();  
    res.json(instances);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
router.get('/instances/:id', async (req, res) => {  
  try {  
    const instance = await  
formationInstanceController.obtenirInstanceParId(req.params.id);  
    if (!instance) {  
      return res.status(404).json({ error: 'Instance non trouvée' });  
    }  
    res.json(instance);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
// ✅ CORRIGÉ: Une seule route pour les formations disponibles  
router.get('/formations-disponibles', async (req, res) => {  
  try {  
    const { formation } = req.query;  
    console.log('🔍 Recherche instances pour formation:', formation);  
  
    const instances = await
```

```
reservationController.listerFormationsDisponibles(formation);
  res.json(instances);
} catch (error) {
  console.error(' ✖ Erreur récupération instances:', error);
  res.status(500).json({ error: error.message });
}
});
```

// ✅ CORRIGÉ: Une seule route pour les réservations

```
router.post('/reservations', async (req, res) => {
  try {
    const idReservation = await
reservationController.inscrireFormation(req.body);
    res.json({
      success: true,
      message: 'Inscription réussie',
      id_reservation: idReservation
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

// ✖ SUPPRIMER: Les routes dupliquées en bas du fichier

// Route pour récupérer les statistiques admin

```
router.get('/admin/statistiques', async (req, res) => {
  try {
    const query = `
      SELECT
        f.id_formation,
        f.nom,
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        COALESCE(tr.reservation_count, 0) as inscrits,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
```

```

        (COALESCE(tr.reservation_count, 0) >= 30) as complet
FROM formation f
JOIN formation_instance fi ON f.id_formation =
fi.id_reference_formation
LEFT JOIN (
    SELECT id_formation_instance, COUNT(*) as reservation_count
    FROM table_reservation
    GROUP BY id_formation_instance
) tr ON fi.id_formation_instance = tr.id_formation_instance
WHERE fi.jour >= CURRENT_DATE
    AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
ORDER BY fi.jour, f.heure_debut
`;

```

```
const result = await pool.query(query);
```

```

const formations = result.rows.map(row => ({
  id: row.id_formation_instance,
  nom: row.nom,
  date: row.date_instance,
  heureDebut: row.heure_debut?.substring(0, 5),
  heureFin: row.heure_fin?.substring(0, 5),
  inscrits: parseInt(row.inscrits),
  places_restantes: parseInt(row.places_restantes),
  complet: row.complet
}));

```

```

    res.json(formations);
  } catch (error) {
    console.error('Erreur récupération statistiques admin:', error);
    res.status(500).json({ error: error.message });
  }
});

```

```
export default router;
```

Parfait ! Votre route `/admin/statistiques` est déjà correcte - elle inclut bien `f.id_formation` dans la requête SQL. Cependant, je vois que dans le mapping des résultats, vous ne retournez pas `id_formation`. Voici la correction :

Modifier la route /admin/statistiques dans Backend/routes/formations.js

javascript

```
// Route pour récupérer Les statistiques admin
router.get('/admin/statistiques', async (req, res) => {
  try {
    const query = `
      SELECT
        f.id_formation,          // ✅ Déjà présent
        f.nom,
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        COALESCE(tr.reservation_count, 0) as inscrits,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation f
      JOIN formation_instance fi ON f.id_formation = fi.id_reference_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.jour >= CURRENT_DATE
        AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
      ORDER BY fi.jour, f.heure_debut
    `;

    const result = await pool.query(query);

    const formations = result.rows.map(row => ({
      id: row.id_formation_instance,
      id_formation: row.id_formation, // ✅ AJOUT: Inclure l'ID de la formation
      nom: row.nom,
      date: row.date_instance,
      heureDebut: row.heure_debut?.substring(0, 5),
      heureFin: row.heure_fin?.substring(0, 5),
      inscrits: parseInt(row.inscrits),
      places_restantes: parseInt(row.places_restantes),
      complet: row.complet
    }));
```

```

    }));

    console.log('📊 Données admin envoyées:', formations.length, 'sessions');
    res.json(formations);
  } catch (error) {
    console.error('Erreur récupération statistiques admin:', error);
    res.status(500).json({ error: error.message });
  }
});

```

Vérification du fonctionnement

Après cette modification, testez :

1. Redémarrez le backend
2. Allez sur la page admin (tessiern782@gmail.com / Password1)
3. Ouvrez la console du navigateur (F12) et vérifiez les logs

Vous devriez voir dans la console :

text

```

📊 Données admin: [...] sessions
🔗 Types de formation: [...]

```

Et dans le sélecteur de filtres, vous devriez voir tous les types de formation disponibles (Blender, Krita, DaVinci Resolve, etc.).

Si les données ne s'affichent pas

Ajoutez ce debug temporaire dans PageAdministrateur.jsx :

javascript

```

// Dans le useEffect, après setFormations(data)
console.log('🔍 Première formation:', data[0]);
console.log('📄 Colonnes disponibles:', data[0] ? Object.keys(data[0]) : 'Aucune donnée');

```

Cela vous montrera si `id_formation` est bien présent dans les données.

La seule modification nécessaire est d'ajouter `id_formation: row.id_formation` dans le mapping des résultats de la route `/admin/statistiques`.

Cela ne fonctionne pas, c'est même encore pire parce qu'il n'y a aucune formation d'afficher dans la liste même quand j'affiche tous les logiciels. D'ailleurs le menu drop down affiche uniquement l'option tous les logiciels. Il n'y a pas l'option pour afficher un logiciel spécifique. Voici l'erreur que j'ai dans la console :

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend\$ npm start

> start

> node server.js

[dotenv@17.2.3] injecting env (6) from .env -- tip:  load multiple .env files with { path: ['.env.local', '.env'] }

Serveur démarré sur le port 3000

Erreur récupération statistiques admin: error: syntax error at or near "présent"

at

/mnt/c/Users/nicol/Desktop/softghost/Backend/node_modules/pg-pool/index.js:45:11

at process.processTicksAndRejections

(node:internal/process/task_queues:105:5)

at async

file:///mnt/c/Users/nicol/Desktop/softghost/Backend/routes/formations.js:112:20 {

length: 97,

severity: 'ERROR',

code: '42601',

detail: undefined,

hint: undefined,

position: '57',

internalPosition: undefined,

internalQuery: undefined,

where: undefined,

schema: undefined,

table: undefined,


```
column: undefined,  
dataType: undefined,  
constraint: undefined,  
file: 'scan.l',  
line: '1236',  
routine: 'scanner_yyerror'  
}
```

Erreur récupération statistiques admin: error: syntax error at or near
"présent"

```
at  
/mnt/c/Users/nicol/Desktop/softghost/Backend/node_modules/pg-  
pool/index.js:45:11  
at process.processTicksAndRejections  
(node:internal/process/task_queues:105:5)  
at async  
file:///mnt/c/Users/nicol/Desktop/softghost/Backend/routes/formatio  
ns.js:112:20 {  
length: 97,  
severity: 'ERROR',  
code: '42601',  
detail: undefined,  
hint: undefined,  
position: '57',  
internalPosition: undefined,  
internalQuery: undefined,  
where: undefined,  
schema: undefined,  
table: undefined,  
column: undefined,  
dataType: undefined,  
constraint: undefined,  
file: 'scan.l',  
line: '1236',  
routine: 'scanner_yyerror'  
}
```

^C

wizcolas@Purple-

Neon:/mnt/c/Users/nicol/Desktop/softghost/Backend\$

Je vois le problème ! Il y a un commentaire SQL invalide dans la requête. Voici la version corrigée de la route /admin/statistiques :

Corriger Backend/routes/formations.js

javascript

```
// Route pour récupérer Les statistiques admin
router.get('/admin/statistiques', async (req, res) => {
  try {
    const query = `
      SELECT
        f.id_formation,
        f.nom,
        fi.id_formation_instance,
        fi.jour as date_instance,
        f.heure_debut,
        f.heure_fin,
        COALESCE(tr.reservation_count, 0) as inscrits,
        (30 - COALESCE(tr.reservation_count, 0)) as places_restantes,
        (COALESCE(tr.reservation_count, 0) >= 30) as complet
      FROM formation f
      JOIN formation_instance fi ON f.id_formation = fi.id_reference_formation
      LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
      ) tr ON fi.id_formation_instance = tr.id_formation_instance
      WHERE fi.jour >= CURRENT_DATE
        AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
      ORDER BY fi.jour, f.heure_debut
    `;

    console.log('🚦 Exécution de la requête admin...');
    const result = await pool.query(query);
    console.log('✅ Données récupérées:', result.rows.length, 'sessions');

    const formations = result.rows.map(row => ({
      id: row.id_formation_instance,
      id_formation: row.id_formation, // Inclure l'ID de la formation
      nom: row.nom,
      date: row.date_instance,
      heureDebut: row.heure_debut?.substring(0, 5),
    }));
```

```
    heureFin: row.heure_fin?.substring(0, 5),
    inscrits: parseInt(row.inscrits),
    places_restantes: parseInt(row.places_restantes),
    complet: row.complet
  }));

  console.log('📄 Première formation:', formations[0]);
  res.json(formations);
} catch (error) {
  console.error('❌ Erreur récupération statistiques admin:', error);
  res.status(500).json({ error: error.message });
}
});
```

Test de débogage

1. Redémarrez le backend
2. Testez directement l'API dans votre navigateur :

`http://localhost:3000/api/admin/statistiques`

Vous devriez voir un JSON avec toutes les sessions.

Si l'API retourne un tableau vide

Cela signifie qu'il n'y a pas de `formation_instance` dans la base de données pour le mois prochain. Vérifiez avec :

```
sql

-- Vérifier les instances de formation
SELECT
  fi.id_formation_instance,
  fi.jour,
  f.nom,
  f.id_formation
FROM formation_instance fi
JOIN formation f ON fi.id_reference_formation = f.id_formation
WHERE fi.jour >= CURRENT_DATE
      AND fi.jour <= CURRENT_DATE + INTERVAL '1 month'
ORDER BY fi.jour;
```

Si vous n'avez pas de données de test

Ajoutez des instances de test avec :

sql

```
-- Insérer des instances de test pour le mois prochain
INSERT INTO formation_instance (id_formation_instance, jour, id_reference_formation)
VALUES
('formation_instance_test1', CURRENT_DATE + INTERVAL '7 days', 'formation1'),
('formation_instance_test2', CURRENT_DATE + INTERVAL '14 days', 'formation2'),
('formation_instance_test3', CURRENT_DATE + INTERVAL '21 days', 'formation3');
```

Version de secours avec données simulées

Si l'API continue de poser problème, voici une version temporaire avec des données simulées :

javascript

```
// Route pour récupérer les statistiques admin (version simulée temporaire)
router.get('/admin/statistiques', async (req, res) => {
  try {
    console.log('📊 Utilisation de données simulées pour le debug...');

    // Données simulées pour tester
    const formationsSimulees = [
      {
        id: 'formation_instance1',
        id_formation: 'formation1',
        nom: 'Formation Blender 3D',
        date: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString(),
        heureDebut: '14:00',
        heureFin: '17:00',
        inscrits: 15,
        places_restantes: 15,
        complet: false
      },
      {
        id: 'formation_instance2',
        id_formation: 'formation2',
        nom: 'Formation Krita - Dessin Digital',

```

```

    date: new Date(Date.now() + 14 * 24 * 60 * 60 * 1000).toISOString(),
    heureDebut: '18:00',
    heureFin: '21:00',
    inscrits: 28,
    places_restantes: 2,
    complet: false
  },
  {
    id: 'formation_instance3',
    id_formation: 'formation3',
    nom: 'Formation Kdenlive - Montage Vidéo',
    date: new Date(Date.now() + 21 * 24 * 60 * 60 * 1000).toISOString(),
    heureDebut: '19:00',
    heureFin: '22:00',
    inscrits: 30,
    places_restantes: 0,
    complet: true
  }
];

console.log('✅ Données simulées envoyées:', formationsSimulees.length, 'sessions');
res.json(formationsSimulees);
} catch (error) {
  console.error('❌ Erreur récupération statistiques admin:', error);
  res.status(500).json({ error: error.message });
}
});

```

Testez d'abord la version corrigée sans les commentaires SQL invalides. Le problème venait des commentaires // ✅ Déjà présent dans la requête SQL.

C'est parfait le backend fonctionne à merveille. En revanche le frontend a encore un problème. Le problème est que tout les options sont écrits blanc sur du blanc et donc on ne les voit pas. La seule écriture dans le job dom de New qui est écrit noir sur blanc est l'option sur laquelle ma souris est au-dessus. Voici à quoi ressemble pour l'instant le fichier src/PageAdministrateur.css :

```

/* PageAdministrateur.css */
.admin-container {
  min-height: 100vh;

```

```
min-width: 100vw;  
background: linear-gradient(135deg, #2c3e50 0%, #34495e  
100%);  
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
margin: 0;  
padding: 0;  
position: fixed;  
top: 0;  
left: 0;  
right: 0;  
bottom: 0;  
overflow-y: auto;  
}
```

```
.admin-content {  
  max-width: 1200px;  
  margin: 0 auto;  
  padding: 30px 20px;  
  min-height: 100vh;  
  box-sizing: border-box;  
}
```

```
/* Header */  
.admin-header {  
  margin-bottom: 40px;  
}
```

```
.header-top {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 30px;  
}
```

```
.back-button {  
  background: rgba(255, 255, 255, 0.1);  
  border: 2px solid rgba(255, 255, 255, 0.3);  
  color: white;  
  padding: 12px 24px;  
  border-radius: 8px;
```

```
    cursor: pointer;
    font-size: 1em;
    transition: all 0.3s ease;
    backdrop-filter: blur(10px);
}

.back-button:hover {
    background: rgba(255, 255, 255, 0.2);
    transform: translateX(-5px);
}

.admin-title {
    color: white;
    font-size: 2.5em;
    font-weight: 700;
    margin: 0;
    text-align: center;
    flex: 1;
}

/* Statistiques */
.admin-stats {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
    margin-top: 30px;
}

.stat-card {
    background: rgba(255, 255, 255, 0.1);
    backdrop-filter: blur(10px);
    border-radius: 15px;
    padding: 25px;
    text-align: center;
    border: 1px solid rgba(255, 255, 255, 0.2);
    transition: all 0.3s ease;
}

.stat-card:hover {
    transform: translateY(-5px);
```

```
background: rgba(255, 255, 255, 0.15);
}

.stat-number {
  color: white;
  font-size: 2.5em;
  font-weight: 700;
  margin-bottom: 8px;
}

.stat-label {
  color: rgba(255, 255, 255, 0.8);
  font-size: 1em;
  font-weight: 500;
}

/* Filtres */
.filtres-section {
  background: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(10px);
  border-radius: 15px;
  padding: 25px;
  margin-bottom: 30px;
  border: 1px solid rgba(255, 255, 255, 0.2);
}

.filtres-section h2 {
  color: white;
  margin: 0 0 20px 0;
  font-size: 1.5em;
}

.filtres-buttons {
  display: flex;
  gap: 15px;
  flex-wrap: wrap;
}

.filtre-btn {
  background: rgba(255, 255, 255, 0.1);
```



```
border: 2px solid rgba(255, 255, 255, 0.3);
color: white;
padding: 12px 24px;
border-radius: 25px;
cursor: pointer;
font-size: 1em;
font-weight: 500;
transition: all 0.3s ease;
}

.filtre-btn:hover {
  background: rgba(255, 255, 255, 0.2);
  transform: translateY(-2px);
}

.filtre-btn.active {
  background: #3498db;
  border-color: #3498db;
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(52, 152, 219, 0.4);
}

/* Liste des formations */
.admin-main {
  background: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(10px);
  border-radius: 15px;
  padding: 30px;
  border: 1px solid rgba(255, 255, 255, 0.2);
}

.admin-main h2 {
  color: white;
  margin: 0 0 25px 0;
  font-size: 1.8em;
}

. formations-list {
  display: flex;
  flex-direction: column;
```

```
gap: 20px;
}

.formation-admin-card {
  background: rgba(255, 255, 255, 0.95);
  border-radius: 15px;
  padding: 25px;
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.1);
  transition: all 0.3s ease;
  border-left: 5px solid #3498db;
}

.formation-admin-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
}

.formation-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 20px;
}

.formation-nom {
  color: #2c3e50;
  font-size: 1.4em;
  font-weight: 600;
  margin: 0;
  flex: 1;
}

.statut-badge {
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 0.9em;
  font-weight: 600;
  margin-left: 15px;
}
```

```
.statut-badge.disponible {
  background: #e8f6f3;
  color: #27ae60;
  border: 1px solid #a3e4d7;
}

.statut-badge.presque-pleine {
  background: #fef9e7;
  color: #f39c12;
  border: 1px solid #fad7a0;
}

.statut-badge.pleine {
  background: #fdeecf;
  color: #e74c3c;
  border: 1px solid #f5b7b1;
}

.formation-details {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
  margin-bottom: 20px;
}

.detail-item {
  display: flex;
  align-items: center;
  gap: 10px;
}

.detail-label {
  color: #7f8c8d;
  font-weight: 500;
  font-size: 0.95em;
}

.detail-value {
  color: #2c3e50;
  font-weight: 600;
}
```

```
}

/* Barre de progression */
.progression-section {
  margin-bottom: 20px;
}

.progression-bar {
  width: 100%;
  height: 10px;
  background: #ecf0f1;
  border-radius: 5px;
  overflow: hidden;
  margin-bottom: 8px;
}

.progression-remplissage {
  height: 100%;
  border-radius: 5px;
  transition: width 0.3s ease;
}

.progression-remplissage.disponible {
  background: linear-gradient(90deg, #27ae60, #2ecc71);
}

.progression-remplissage.presque-pleine {
  background: linear-gradient(90deg, #f39c12, #f1c40f);
}

.progression-remplissage.pleine {
  background: linear-gradient(90deg, #e74c3c, #c0392b);
}

.progression-text {
  color: #7f8c8d;
  font-size: 0.9em;
  font-weight: 500;
  text-align: right;
}
```

```
/* Actions */
.formation-actions {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
}

.action-btn {
  padding: 10px 20px;
  border: none;
  border-radius: 8px;
  font-size: 0.9em;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
}

.details-btn {
  background: #3498db;
  color: white;
}

.details-btn:hover {
  background: #2980b9;
  transform: translateY(-2px);
}

.export-btn {
  background: #95a5a6;
  color: white;
}

.export-btn:hover {
  background: #7f8c8d;
  transform: translateY(-2px);
}

.no-formation {
  text-align: center;
```

```
padding: 40px 20px;
color: rgba(255, 255, 255, 0.8);
font-size: 1.1em;
}

/* Personnalisation de la barre de scroll */
.admin-container::-webkit-scrollbar {
  width: 12px;
}

.admin-container::-webkit-scrollbar-track {
  background: linear-gradient(135deg, #2c3e50 0%, #34495e 100%);
}

.admin-container::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.3);
  border-radius: 6px;
  border: 2px solid #2c3e50;
}

.admin-container::-webkit-scrollbar-thumb:hover {
  background: rgba(255, 255, 255, 0.5);
}

/* Pour Firefox */
.admin-container {
  scrollbar-width: thin;
  scrollbar-color: rgba(255, 255, 255, 0.3) #2c3e50;
}

/* Responsive */
@media (max-width: 768px) {
  .admin-content {
    padding: 20px 15px;
  }

  .header-top {
    flex-direction: column;
    gap: 15px;
  }
}
```

```
    text-align: center;
  }

.admin-title {
  font-size: 2em;
}

.admin-stats {
  grid-template-columns: 1fr;
}

.filtres-buttons {
  flex-direction: column;
}

.formation-header {
  flex-direction: column;
  gap: 10px;
}

.statut-badge {
  align-self: flex-start;
  margin-left: 0;
}

.formation-details {
  grid-template-columns: 1fr;
}

.formation-actions {
  flex-direction: column;
}
}

@media (max-width: 480px) {
  .admin-title {
    font-size: 1.7em;
  }

  .admin-main {
```

```
padding: 20px 15px;
}

.formation-admin-card {
padding: 20px;
}

.formation-nom {
font-size: 1.2em;
}
}

.login-error {
background: #fee;
border: 1px solid #fcc;
border-radius: 8px;
padding: 12px;
margin: 15px 0;
color: #c33;
font-size: 0.9em;
text-align: center;
}

.loading-admin {
display: flex;
justify-content: center;
align-items: center;
height: 50vh;
font-size: 1.2em;
color: #666;
}

/* Styles pour les nouveaux filtres */
.filtre-groupe {
margin-bottom: 20px;
}

.filtre-label {
display: block;
```



```
margin-bottom: 8px;
font-weight: 600;
color: #333;
font-size: 1em;
}

.filtre-select {
padding: 10px 15px;
border: 2px solid #e1e5e9;
border-radius: 8px;
font-size: 1em;
background: white;
cursor: pointer;
transition: all 0.3s ease;
min-width: 250px;
}

.filtre-select:focus {
outline: none;
border-color: #667eea;
box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.filtres-resume {
margin-top: 15px;
padding: 12px;
background: #f8f9fa;
border-radius: 8px;
border-left: 4px solid #667eea;
}

.filtres-resume p {
margin: 0;
color: #495057;
font-size: 0.95em;
}

.formation-id {
font-family: 'Courier New', monospace;
background: #f1f3f4;
```

```
padding: 2px 6px;
border-radius: 4px;
font-size: 0.85em;
color: #5f6368;
}

.reset-filtres-btn {
background: #6c757d;
color: white;
border: none;
padding: 10px 20px;
border-radius: 6px;
cursor: pointer;
font-size: 0.9em;
margin-top: 10px;
transition: background 0.3s ease;
}

.reset-filtres-btn:hover {
background: #5a6268;
}

/* Responsive */
@media (max-width: 768px) {
.filtre-select {
min-width: 100%;
}

.filtres-buttons {
flex-direction: column;
gap: 10px;
}

.filtre-btn {
width: 100%;
}
}
```

et voici le fichier src/PageAdministrateur.jsx :

```
// PageAdministrateur.jsx
import React, { useState, useEffect } from 'react';
import './PageAdministrateur.css';

const PageAdministrateur = ({
  onRetourAccueil = () => {
    console.log('Retour à l\'accueil');
  }
}) => {
  const [filtreStatut, setFiltreStatut] = useState('toutes'); // 'toutes',
  'disponibles', 'pleines'
  const [filtreFormation, setFiltreFormation] = useState('toutes'); //
  'toutes' ou id_formation spécifique
  const [formations, setFormations] = useState([]);
  const [typesFormation, setTypesFormation] = useState([]); // ✅
  AJOUT: Liste des types de formation
  const [loading, setLoading] = useState(true);

  // Récupérer les vraies données depuis l'API
  useEffect(() => {
    const fetchFormationsAdmin = async () => {
      try {
        setLoading(true);
        const response = await
        fetch('http://localhost:3000/api/admin/statistiques');

        if (!response.ok) {
          throw new Error('Erreur lors du chargement des données');
        }

        const data = await response.json();
        console.log('📊 Données admin:', data);
        setFormations(data);

        // ✅ AJOUT: Extraire les types de formation uniques
        const typesUniques = [...new Set(data.map(item => ({
          id: item.id_formation,
          nom: item.nom
        })))].filter(item => item.id && item.nom);
```

```
    setTypesFormation(typesUniques);
    console.log('🔗 Types de formation:', typesUniques);

  } catch (error) {
    console.error('Erreur chargement données admin:', error);
    setFormations([]);
    setTypesFormation([]);
  } finally {
    setLoading(false);
  }
};

fetchFormationsAdmin();
}, []);

// Trier les formations par date
const formationsTriees = [...formations].sort((a, b) => new
Date(a.date) - new Date(b.date));

// ✅ MODIFIÉ: Filtrer les formations selon les filtres sélectionnés
const formationsFiltrees = formationsTriees.filter(formation => {
  // Filtre par statut (pleines/disponibles)
  let filtreStatutOk = true;
  if (filtreStatut === 'pleines') {
    filtreStatutOk = formation.inscrits >= 30;
  } else if (filtreStatut === 'disponibles') {
    filtreStatutOk = formation.inscrits < 30;
  }

  // ✅ AJOUT: Filtre par type de formation
  let filtreTypeOk = true;
  if (filtreFormation !== 'toutes') {
    filtreTypeOk = formation.id_formation === filtreFormation;
  }

  return filtreStatutOk && filtreTypeOk;
});

const formatDate = (dateStr) => {
```

```
const date = new Date(dateStr);
return date.toLocaleDateString('fr-FR', {
  weekday: 'long',
  year: 'numeric',
  month: 'long',
  day: 'numeric'
});
};

const getStatutFormation = (inscrits) => {
  if (inscrits >= 30) {
    return { texte: 'Formation pleine', classe: 'pleine' };
  } else if (inscrits >= 25) {
    return { texte: 'Presque pleine', classe: 'presque-pleine' };
  } else {
    return { texte: 'Places disponibles', classe: 'disponible' };
  }
};

const getPourcentageInscription = (inscrits) => {
  return Math.min((inscrits / 30) * 100, 100);
};

if (loading) {
  return (
    <div className="admin-container">
      <div className="loading-admin">
        <p>Chargement des données administrateur...</p>
      </div>
    </div>
  );
}

return (
  <div className="admin-container">
    <div className="admin-content">
      { /* Header */ }
      <header className="admin-header">
        <div className="header-top">
          <button className="back-button" onClick=
```

```

{onRetourAccueil}>
  ← Retour à l'accueil
</button>
<h1 className="admin-title">Espace Administrateur</h1>
</div>

{/* Statistiques */}
<div className="admin-stats">
  <div className="stat-card">
    <div className="stat-number">{formations.length}</div>
    <div className="stat-label">Sessions ce mois</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.filter(f => f.inscrits >= 30).length}
    </div>
    <div className="stat-label">Sessions pleines</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.reduce((total, f) => total + f.inscrits, 0)}
    </div>
    <div className="stat-label">Total inscriptions</div>
  </div>
</div>
</header>

{/* Filtres */}
<div className="filtres-section">
  <h2>Filtrer les sessions</h2>

  {/* ✅ AJOUT: Filtre par type de formation */}
  <div className="filtre-groupe">
    <label htmlFor="filtre-formation" className="filtre-label">
      📅 Type de formation :
    </label>
    <select
      id="filtre-formation"
      value={filtreFormation}
      onChange={(e) => setFiltreFormation(e.target.value)}
    >

```

```

        className="filtre-select"
      >
        <option value="toutes">Tous les logiciels</option>
        {typesFormation.map((type) => (
          <option key={type.id} value={type.id}>
            {type.nom}
          </option>
        ))}
      </select>
    </div>

    { /* Filtre par statut */}
    <div className="filtre-groupe">
      <label className="filtre-label"> 🏠 Statut :</label>
      <div className="filtres-buttons">
        <button
          className={`filtre-btn ${filtreStatut === 'toutes' ? 'active'
: ''}}
          onClick={() => setFiltreStatut('toutes')}
        >
          Toutes les sessions
        </button>
        <button
          className={`filtre-btn ${filtreStatut === 'disponibles' ?
'active' : ''}}
          onClick={() => setFiltreStatut('disponibles')}
        >
          Places disponibles
        </button>
        <button
          className={`filtre-btn ${filtreStatut === 'pleines' ?
'active' : ''}}
          onClick={() => setFiltreStatut('pleines')}
        >
          Sessions pleines
        </button>
      </div>
    </div>

    { /* ✅ AJOUT: Résumé des filtres */}

```

```

<div className="filtres-resume">
  <p>
    Affichage de <strong>{formationsFiltrees.length}</strong>
    session(s)
    {filtreFormation !== 'toutes' &&
      ` pour "${typesFormation.find(t => t.id ===
filtreFormation)?.nom || filtreFormation}`
    }
    {filtreStatut !== 'toutes' &&
      ` (${filtreStatut === 'pleines' ? 'pleines' : 'avec places
disponibles'})`
    }
  </p>
</div>
</div>

```

```

{ /* Liste des formations */ }
<main className="admin-main">
  <h2>Sessions du mois prochain</h2>

  {formationsFiltrees.length === 0 ? (
    <div className="no-formations">
      <p>Aucune session ne correspond aux critères
sélectionnés.</p>
      <button
        className="reset-filtres-btn"
        onClick={() => {
          setFiltreStatut('toutes');
          setFiltreFormation('toutes');
        }}
      >
        Réinitialiser les filtres
      </button>
    </div>
  ) : (
    <div className="formations-list">
      {formationsFiltrees.map((formation) => {
        const statut = getStatutFormation(formation.inscrits);
        const pourcentage =
getPourcentageInscription(formation.inscrits);

```



```

return (
  <div key={formation.id} className="formation-admin-
card">
    <div className="formation-header">
      <h3 className="formation-nom">{formation.nom}
</h3>
      <span className={`statut-badge ${statut.classe}`}>
        {statut.texte}
      </span>
    </div>

    <div className="formation-details">
      <div className="detail-item">
        <span className="detail-label"> 📅 Date :</span>
        <span className="detail-value">
{formatDate(formation.date)}</span>
      </div>
      <div className="detail-item">
        <span className="detail-label"> 🕒 Heure :</span>
        <span className="detail-value">
{formation.heureDebut} - {formation.heureFin}</span>
      </div>
      <div className="detail-item">
        <span className="detail-label"> 👥 Inscrits :</span>
        <span className="detail-value">
          {formation.inscrits} / 30 personnes
        </span>
      </div>
      <div className="detail-item">
        <span className="detail-label"> 🆔 ID Formation :
</span>
        <span className="detail-value formation-id">
{formation.id_formation}</span>
      </div>
    </div>

    {/* Barre de progression */}
    <div className="progression-section">
      <div className="progression-bar">

```

```

        <div
            className={`progression-remplissage
${statut.classe}`}
            style={{ width: `${pourcentage}%` }}
        ></div>
    </div>
    <div className="progression-text">
        {formation.inscrits} inscrits
        ({Math.round(pourcentage)}%)
    </div>
</div>

    {/* Actions */}
    <div className="formation-actions">
        <button className="action-btn details-btn">
            Voir les détails
        </button>
        <button className="action-btn export-btn">
            Exporter la liste
        </button>
    </div>
</div>
    );
    }}}
</div>
    )}
</main>
</div>
</div>
    );
};

export default PageAdministrateur;

```

Le problème vient des couleurs dans la section des filtres. Les labels et textes sont en couleur claire sur fond clair. Voici les corrections CSS :

Modifier PageAdministrateur.css - Section des filtres

CSS

```
/* Styles pour les nouveaux filtres */
.filtre-groupe {
  margin-bottom: 20px;
}

.filtre-label {
  display: block;
  margin-bottom: 8px;
  font-weight: 600;
  color: white; /* ✅ CHANGEMENT: Blanc pour être visible sur fond sombre */
  font-size: 1em;
}

.filtre-select {
  padding: 10px 15px;
  border: 2px solid rgba(255, 255, 255, 0.3); /* ✅ CHANGEMENT: Bordure visible */
  border-radius: 8px;
  font-size: 1em;
  background: rgba(255, 255, 255, 0.1); /* ✅ CHANGEMENT: Fond semi-transparent */
  color: white; /* ✅ CHANGEMENT: Texte blanc */
  cursor: pointer;
  transition: all 0.3s ease;
  min-width: 250px;
  backdrop-filter: blur(10px);
}

.filtre-select:focus {
  outline: none;
  border-color: #3498db; /* ✅ CHANGEMENT: Bleu pour le focus */
  box-shadow: 0 0 0 3px rgba(52, 152, 219, 0.3);
}

/* ✅ AJOUT: Styles pour les options du select */
.filtre-select option {
  background: #2c3e50; /* Fond sombre pour les options */
  color: white; /* Texte blanc pour les options */
  padding: 10px;
}

.filtres-resume {
  margin-top: 15px;
  padding: 12px;
```

```
background: rgba(255, 255, 255, 0.1); /* ✅ CHANGEMENT: Fond semi-transparent */
border-radius: 8px;
border-left: 4px solid #3498db; /* ✅ CHANGEMENT: Bleu pour correspondre au thème
*/
backdrop-filter: blur(10px);
}

.filtres-resume p {
margin: 0;
color: white; /* ✅ CHANGEMENT: Texte blanc */
font-size: 0.95em;
}

.formation-id {
font-family: 'Courier New', monospace;
background: rgba(255, 255, 255, 0.1);
padding: 2px 6px;
border-radius: 4px;
font-size: 0.85em;
color: rgba(255, 255, 255, 0.8);
}

.reset-filtres-btn {
background: rgba(108, 117, 125, 0.8);
color: white;
border: none;
padding: 10px 20px;
border-radius: 6px;
cursor: pointer;
font-size: 0.9em;
margin-top: 10px;
transition: background 0.3s ease;
backdrop-filter: blur(10px);
}

.reset-filtres-btn:hover {
background: rgba(90, 98, 104, 0.9);
}

/* ✅ AJOUT: Style pour le placeholder du select */
.filtre-select::placeholder {
color: rgba(255, 255, 255, 0.7);
}
```

```

/*  AJOUT: Style pour les options au survol */
.filtre-select option:hover {
  background: #3498db;
}

/* Responsive */
@media (max-width: 768px) {
  .filtre-select {
    min-width: 100%;
  }

  .filtres-buttons {
    flex-direction: column;
    gap: 10px;
  }

  .filtre-btn {
    width: 100%;
  }
}

```

Alternative si les options restent invisibles

Si les options du select restent blanches sur blanc, vous pouvez utiliser cette version alternative avec un composant custom :

javascript

```

// Dans PageAdministrateur.jsx - Remplacer le select par cette version
<div className="filtre-groupe">
  <label htmlFor="filtre-formation" className="filtre-label">
     Type de formation :
  </label>
  <select
    id="filtre-formation"
    value={filtreFormation}
    onChange={(e) => setFiltreFormation(e.target.value)}
    className="filtre-select"
    style={{
      backgroundColor: 'rgba(44, 62, 80, 0.9)',
      color: 'white',
      border: '2px solid rgba(255, 255, 255, 0.3)',

```

```

        borderRadius: '8px',
        padding: '10px 15px',
        minWidth: '250px'
      }}
    >
    <option value="toutes" style={{ backgroundColor: '#2c3e50', color: 'white' }}>
      Tous les logiciels
    </option>
    {typesFormation.map((type) => (
      <option
        key={type.id}
        value={type.id}
        style={{ backgroundColor: '#2c3e50', color: 'white' }}
      >
        {type.nom}
      </option>
    ))}
  </select>
</div>

```

Solution ultime : Custom Dropdown

Si le select natif pose toujours problème, créez un dropdown custom :

javascript

```

// Alternative avec dropdown custom
const [showDropdown, setShowDropdown] = useState(false);

// Dans le JSX, remplacer le select par :
<div className="custom-dropdown-container">
  <label className="filtre-label"> 📌 Type de formation :</label>
  <div className="custom-dropdown">
    <button
      className="dropdown-toggle"
      onClick={() => setShowDropdown(!showDropdown)}
    >
      {filtreFormation === 'toutes'
        ? 'Tous les logiciels'
        : typesFormation.find(t => t.id === filtreFormation)?.nom
      }
    <span className="dropdown-arrow">▼</span>
  </button>

```

```

{showDropdown && (
  <div className="dropdown-menu">
    <div
      className={`dropdown-item ${filtreFormation === 'toutes' ? 'active' : ''}`}
      onClick={() => {
        setFiltreFormation('toutes');
        setShowDropdown(false);
      }}
    >
      Tous les logiciels
    </div>
    {typesFormation.map((type) => (
      <div
        key={type.id}
        className={`dropdown-item ${filtreFormation === type.id ? 'active' : ''}`}
        onClick={() => {
          setFiltreFormation(type.id);
          setShowDropdown(false);
        }}
      >
        {type.nom}
      </div>
    ))}
  </div>
)}
</div>
</div>

```

Et ajouter le CSS correspondant. Mais **essayez d'abord la correction CSS simple** - cela devrait résoudre le problème d'affichage des options du select.

Parfait maintenant je sens que mon projet est presque terminé. Je voudrais faire quelques finitions avec toi. Premièrement dans la page d'administrateur je voudrais qu'on enlève les boutons voir les détails et exporter la liste. Pour l'instant la page d'administrateur ressemble à ça :

```

// PageAdministrateur.jsx
import React, { useState, useEffect } from 'react';
import './PageAdministrateur.css';

```

```
const PageAdministrateur = ({
  onRetourAccueil = () => {
    console.log('Retour à l\'accueil');
  }
}) => {
  const [filtreStatut, setFiltreStatut] = useState('toutes'); // 'toutes',
  'disponibles', 'pleines'
  const [filtreFormation, setFiltreFormation] = useState('toutes'); //
  'toutes' ou id_formation spécifique
  const [formations, setFormations] = useState([]);
  const [typesFormation, setTypesFormation] = useState([]); // ✅
  AJOUT: Liste des types de formation
  const [loading, setLoading] = useState(true);

  // Récupérer les vraies données depuis l'API
  useEffect(() => {
    const fetchFormationsAdmin = async () => {
      try {
        setLoading(true);
        const response = await
fetch('http://localhost:3000/api/admin/statistiques');

        if (!response.ok) {
          throw new Error('Erreur lors du chargement des données');
        }

        const data = await response.json();
        console.log('📊 Données admin:', data);
        setFormations(data);

        // ✅ AJOUT: Extraire les types de formation uniques
        const typesUniques = [...new Set(data.map(item => ({
          id: item.id_formation,
          nom: item.nom
        })))].filter(item => item.id && item.nom);

        setTypesFormation(typesUniques);
        console.log('🎯 Types de formation:', typesUniques);

      } catch (error) {
```



```
        console.error('Erreur chargement données admin:', error);
        setFormations([]);
        setTypesFormation([]);
    } finally {
        setLoading(false);
    }
};

fetchFormationsAdmin();
}, []);

// Trier les formations par date
const formationsTriees = [...formations].sort((a, b) => new
Date(a.date) - new Date(b.date));

// ✅ MODIFIÉ: Filtrer les formations selon les filtres sélectionnés
const formationsFiltrees = formationsTriees.filter(formation => {
    // Filtre par statut (pleines/disponibles)
    let filtreStatutOk = true;
    if (filtreStatut === 'pleines') {
        filtreStatutOk = formation.inscrits >= 30;
    } else if (filtreStatut === 'disponibles') {
        filtreStatutOk = formation.inscrits < 30;
    }

    // ✅ AJOUT: Filtre par type de formation
    let filtreTypeOk = true;
    if (filtreFormation !== 'toutes') {
        filtreTypeOk = formation.id_formation === filtreFormation;
    }

    return filtreStatutOk && filtreTypeOk;
});

const formatDate = (dateStr) => {
    const date = new Date(dateStr);
    return date.toLocaleDateString('fr-FR', {
        weekday: 'long',
        year: 'numeric',
        month: 'long',
```

```
    day: 'numeric'
  });
};
```

```
const getStatutFormation = (inscrits) => {
  if (inscrits >= 30) {
    return { texte: 'Formation pleine', classe: 'pleine' };
  } else if (inscrits >= 25) {
    return { texte: 'Presque pleine', classe: 'presque-pleine' };
  } else {
    return { texte: 'Places disponibles', classe: 'disponible' };
  }
};
```

```
const getPourcentageInscription = (inscrits) => {
  return Math.min((inscrits / 30) * 100, 100);
};
```

```
if (loading) {
  return (
    <div className="admin-container">
      <div className="loading-admin">
        <p>Chargement des données administrateur...</p>
      </div>
    </div>
  );
}
```

```
return (
  <div className="admin-container">
    <div className="admin-content">
      { /* Header */ }
      <header className="admin-header">
        <div className="header-top">
          <button className="back-button" onClick=
{onRetourAccueil}>
            ← Retour à l'accueil
          </button>
          <h1 className="admin-title">Espace Administrateur</h1>
        </div>
      </header>
    </div>
  </div>
);
```

```

{ /* Statistiques */ }
<div className="admin-stats">
  <div className="stat-card">
    <div className="stat-number">{formations.length}</div>
    <div className="stat-label">Sessions ce mois</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.filter(f => f.inscrits >= 30).length}
    </div>
    <div className="stat-label">Sessions pleines</div>
  </div>
  <div className="stat-card">
    <div className="stat-number">
      {formations.reduce((total, f) => total + f.inscrits, 0)}
    </div>
    <div className="stat-label">Total inscriptions</div>
  </div>
</div>
</header>

```

```

{ /* Filtres */ }
<div className="filtres-section">
  <h2>Filtrer les sessions</h2>

  { /* ✅ AJOUT: Filtre par type de formation */ }
  <div className="filtre-groupe">
    <label htmlFor="filtre-formation" className="filtre-label">
      📁 Type de formation :
    </label>
    <select
      id="filtre-formation"
      value={filtreFormation}
      onChange={(e) => setFiltreFormation(e.target.value)}
      className="filtre-select"
    >
      <option value="toutes">Tous les logiciels</option>
      {typesFormation.map((type) => (
        <option key={type.id} value={type.id}>

```

```

        {type.nom}
      </option>
    )}}
  </select>
</div>

```

```

{ /* Filtre par statut */ }
<div className="filtre-groupe">
  <label className="filtre-label"> 🏠 Statut :</label>
  <div className="filtres-buttons">
    <button
      className={ `filtre-btn ${filtreStatut === 'toutes' ? 'active'
: ''} ` }
      onClick={() => setFiltreStatut('toutes')}
    >
      Toutes les sessions
    </button>
    <button
      className={ `filtre-btn ${filtreStatut === 'disponibles' ?
'active' : ''} ` }
      onClick={() => setFiltreStatut('disponibles')}
    >
      Places disponibles
    </button>
    <button
      className={ `filtre-btn ${filtreStatut === 'pleines' ?
'active' : ''} ` }
      onClick={() => setFiltreStatut('pleines')}
    >
      Sessions pleines
    </button>
  </div>
</div>

```

```

{ /* ✅ AJOUT: Résumé des filtres */ }
<div className="filtres-resume">
  <p>
    Affichage de <strong>{formationsFiltrees.length}</strong>
    session(s)
    {filtreFormation !== 'toutes' &&

```

```

        ` pour "${typesFormation.find(t => t.id ===
filtreFormation)?.nom || filtreFormation}"`
      }
      {filtreStatut !== 'toutes' &&
        ` (${filtreStatut === 'pleines' ? 'pleines' : 'avec places
disponibles'})`
      }
    </p>
  </div>
</div>

```

```

{ /* Liste des formations */ }
<main className="admin-main">
  <h2>Sessions du mois prochain</h2>

  {formationsFiltrees.length === 0 ? (
    <div className="no-formations">
      <p>Aucune session ne correspond aux critères
sélectionnés.</p>
      <button
        className="reset-filtres-btn"
        onClick={() => {
          setFiltreStatut('toutes');
          setFiltreFormation('toutes');
        }}
      >
        Réinitialiser les filtres
      </button>
    </div>
  ) : (
    <div className="formations-list">
      {formationsFiltrees.map((formation) => {
        const statut = getStatutFormation(formation.inscrits);
        const pourcentage =
getPourcentageInscription(formation.inscrits);

        return (
          <div key={formation.id} className="formation-admin-
card">
            <div className="formation-header">

```

```

</h3>
<h3 className="formation-nom">{formation.nom}

<span className={`statut-badge ${statut.classe}`}>
  {statut.texte}
</span>
</div>

<div className="formation-details">
  <div className="detail-item">
    <span className="detail-label"> 📅 Date :</span>
    <span className="detail-value">
      {formatDate(formation.date)}</span>
    </div>
    <div className="detail-item">
      <span className="detail-label"> 🕒 Heure :</span>
      <span className="detail-value">
        {formation.heureDebut} - {formation.heureFin}</span>
      </div>
    <div className="detail-item">
      <span className="detail-label"> 👥 Inscrits :</span>
      <span className="detail-value">
        {formation.inscrits} / 30 personnes
      </span>
    </div>
    <div className="detail-item">
      <span className="detail-label"> ID Formation :
    </span>
      <span className="detail-value formation-id">
        {formation.id_formation}</span>
      </div>
    </div>

    {/* Barre de progression */}
    <div className="progression-section">
      <div className="progression-bar">
        <div
          className={`progression-remplissage
            ${statut.classe}`}
          style={{ width: `${pourcentage}%` }}
        ></div>

```

```

    </div>
    <div className="progression-text">
      {formation.inscrits} inscrits
      ({Math.round(pourcentage)}%)
    </div>
  </div>

  {/* Actions */}
  <div className="formation-actions">
    <button className="action-btn details-btn">
      Voir les détails
    </button>
    <button className="action-btn export-btn">
      Exporter la liste
    </button>
  </div>
</div>
);
}}
</div>
)}
</main>
</div>
</div>
);
};

export default PageAdministrateur;

```

Parfait ! Voici la version simplifiée de `PageAdministrateur.jsx` sans les boutons "Voir les détails" et "Exporter la liste" :

PageAdministrateur.jsx simplifié

javascript

```

// PageAdministrateur.jsx
import React, { useState, useEffect } from 'react';
import './PageAdministrateur.css';

```

```
const PageAdministrateur = ({
  onRetourAccueil = () => {
    console.log('Retour à l\'accueil');
  }
}) => {
  const [filtreStatut, setFiltreStatut] = useState('toutes'); // 'toutes', 'disponibl
es', 'pleines'
  const [filtreFormation, setFiltreFormation] = useState('toutes'); // 'toutes' ou id
_formation spécifique
  const [formations, setFormations] = useState([]);
  const [typesFormation, setTypesFormation] = useState([]);
  const [loading, setLoading] = useState(true);

  // Récupérer les vraies données depuis l'API
  useEffect(() => {
    const fetchFormationsAdmin = async () => {
      try {
        setLoading(true);
        const response = await fetch('http://localhost:3000/api/admin/statistiques');

        if (!response.ok) {
          throw new Error('Erreur lors du chargement des données');
        }

        const data = await response.json();
        console.log('📊 Données admin:', data);
        setFormations(data);

        // Extraire les types de formation uniques
        const typesUniques = [...new Set(data.map(item => ({
          id: item.id_formation,
          nom: item.nom
        })))].filter(item => item.id && item.nom);

        setTypesFormation(typesUniques);
        console.log('🔗 Types de formation:', typesUniques);

      } catch (error) {
        console.error('Erreur chargement données admin:', error);
        setFormations([]);
        setTypesFormation([]);
      } finally {
        setLoading(false);
      }
    }
  }, []);
}
```



```
    });

    fetchFormationsAdmin();
  }, []);

  // Trier les formations par date
  const formationsTriees = [...formations].sort((a, b) => new Date(a.date) - new Date(b.date));

  // Filtrer les formations selon les filtres sélectionnés
  const formationsFiltrees = formationsTriees.filter(formation => {
    // Filtre par statut (pleines/disponibles)
    let filtreStatutOk = true;
    if (filtreStatut === 'pleines') {
      filtreStatutOk = formation.inscrits >= 30;
    } else if (filtreStatut === 'disponibles') {
      filtreStatutOk = formation.inscrits < 30;
    }

    // Filtre par type de formation
    let filtreTypeOk = true;
    if (filtreFormation !== 'toutes') {
      filtreTypeOk = formation.id_formation === filtreFormation;
    }

    return filtreStatutOk && filtreTypeOk;
  });

  const formatDate = (dateStr) => {
    const date = new Date(dateStr);
    return date.toLocaleDateString('fr-FR', {
      weekday: 'long',
      year: 'numeric',
      month: 'long',
      day: 'numeric'
    });
  };

  const getStatutFormation = (inscrits) => {
    if (inscrits >= 30) {
      return { texte: 'Formation pleine', classe: 'pleine' };
    } else if (inscrits >= 25) {
      return { texte: 'Presque pleine', classe: 'presque-pleine' };
    } else {

```

```

    return { texte: 'Places disponibles', classe: 'disponible' };
  }
};

const getPourcentageInscription = (inscrits) => {
  return Math.min((inscrits / 30) * 100, 100);
};

if (loading) {
  return (
    <div className="admin-container">
      <div className="loading-admin">
        <p>Chargement des données administrateur...</p>
      </div>
    </div>
  );
}

return (
  <div className="admin-container">
    <div className="admin-content">
      { /* Header */ }
      <header className="admin-header">
        <div className="header-top">
          <button className="back-button" onClick={onRetourAccueil}>
            ← Retour à l'accueil
          </button>
          <h1 className="admin-title">Espace Administrateur</h1>
        </div>

        { /* Statistiques */ }
        <div className="admin-stats">
          <div className="stat-card">
            <div className="stat-number">{formations.length}</div>
            <div className="stat-label">Sessions ce mois</div>
          </div>
          <div className="stat-card">
            <div className="stat-number">
              {formations.filter(f => f.inscrits >= 30).length}
            </div>
            <div className="stat-label">Sessions pleines</div>
          </div>
          <div className="stat-card">
            <div className="stat-number">

```

```

        {formations.reduce((total, f) => total + f.inscrits, 0)}
      </div>
      <div className="stat-label">Total inscriptions</div>
    </div>
  </div>
</header>

```

```

{ /* Filtrés */ }

```

```

<div className="filtres-section">
  <h2>Filtrer les sessions</h2>

```

```

{ /* Filtre par type de formation */ }

```

```

<div className="filtre-groupe">
  <label htmlFor="filtre-formation" className="filtre-label">
     Type de formation :
  </label>
  <select
    id="filtre-formation"
    value={filtreFormation}
    onChange={(e) => setFiltreFormation(e.target.value)}
    className="filtre-select"
  >
    <option value="toutes">Tous les logiciels</option>
    {typesFormation.map((type) => (
      <option key={type.id} value={type.id}>
        {type.nom}
      </option>
    ))}
  </select>
</div>

```

```

{ /* Filtre par statut */ }

```

```

<div className="filtre-groupe">
  <label className="filtre-label"> Statut :</label>
  <div className="filtres-buttons">
    <button
      className={`filtre-btn ${filtreStatut === 'toutes' ? 'active' : ''}`}
      onClick={() => setFiltreStatut('toutes')}
    >
      Toutes les sessions
    </button>
    <button
      className={`filtre-btn ${filtreStatut === 'disponibles' ? 'active' : ''}}`
    >

```

```

        onClick={() => setFiltreStatut('disponibles')}}
      >
        Places disponibles
    </button>
    <button
      className={`filtre-btn ${filtreStatut === 'pleines' ? 'active' : ''}
    `}

      onClick={() => setFiltreStatut('pleines')}}
    >
      Sessions pleines
    </button>
  </div>
</div>

{/* Résumé des filtres */}
<div className="filtres-resume">
  <p>
    Affichage de <strong>{{formationsFiltrees.length}}</strong> session(s)
    {filtreFormation !== 'toutes' &&
      ` pour "${typesFormation.find(t => t.id === filtreFormation)?.nom ||
filtreFormation}"`
    }
    {filtreStatut !== 'toutes' &&
      ` (${filtreStatut === 'pleines' ? 'pleines' : 'avec places disponible
s'})`
    }
  </p>
</div>
</div>

{/* Liste des formations */}
<main className="admin-main">
  <h2>Sessions du mois prochain</h2>

  {formationsFiltrees.length === 0 ? (
    <div className="no-formations">
      <p>Aucune session ne correspond aux critères sélectionnés.</p>
      <button
        className="reset-filtres-btn"
        onClick={() => {
          setFiltreStatut('toutes');
          setFiltreFormation('toutes');
        }}
      >

```

```

        Réinitialiser les filtres
    </button>
</div>
) : (
<div className="formations-list">
    {formationsFiltrees.map((formation) => {
        const statut = getStatutFormation(formation.inscrits);
        const pourcentage = getPourcentageInscription(formation.inscrits);

        return (
            <div key={formation.id} className="formation-admin-card">
                <div className="formation-header">
                    <h3 className="formation-nom">{formation.nom}</h3>
                    <span className={`statut-badge ${statut.classe}`}>
                        {statut.texte}
                    </span>
                </div>

                <div className="formation-details">
                    <div className="detail-item">
                        <span className="detail-label">📅 Date :</span>
                        <span className="detail-value">{formatDate(formation.date)}</span>
                    </div>
                    <div className="detail-item">
                        <span className="detail-label">🕒 Heure :</span>
                        <span className="detail-value">{formation.heureDebut} - {formation.heureFin}</span>
                    </div>
                    <div className="detail-item">
                        <span className="detail-label">👤 Inscrits :</span>
                        <span className="detail-value">
                            {formation.inscrits} / 30 personnes
                        </span>
                    </div>
                    <div className="detail-item">
                        <span className="detail-label">🆔 ID Formation :</span>
                        <span className="detail-value formation-id">{formation.id_formation}</span>
                    </div>
                </div>

                {/* Barre de progression */}
                <div className="progression-section">

```

```

<div className="progression-bar">
  <div
    className={`progression-remplissage ${statut.classe}`}
    style={{ width: `${pourcentage}%` }}
  ></div>
</div>
<div className="progression-text">
  {formation.inscrits} inscrits ({Math.round(pourcentage)}%)
</div>
</div>

{/*  SUPPRIMÉ: Section des actions */}
</div>

);
}}
</div>

)}
</main>
</div>
</div>
);
};

export default PageAdministrateur;

```

Modifications apportées :

1. Supprimé la section des actions : J'ai entièrement retiré ce bloc :

javascript

```

{/* Actions */}
<div className="formation-actions">
  <button className="action-btn details-btn">
    Voir les détails
  </button>
  <button className="action-btn export-btn">
    Exporter la liste
  </button>
</div>

```

2. **Nettoyé les commentaires** : Retiré les commentaires  AJOUT qui n'étaient plus nécessaires

3. **Conservé toutes les fonctionnalités essentielles** :

- Filtres par type de formation et statut
- Statistiques en temps réel
- Barres de progression visuelles
- Design responsive

La page est maintenant plus épurée et se concentre sur l'essentiel : **visualiser rapidement l'état des réservations** pour chaque session de formation.

Maintenant je veux que l'utilisateur reçoive un courriel de confirmation pour sa réservation à une formation lorsque la réservation est confirmée. Je veux que le message de confirmation affiché sur la page disent que le courriel a été envoyé à son adresse courriel qu'elle a précédemment rentrer pour son inscription.

La page de description de l'événement pour l'instant ressemble à ça :

```
// DescriptionFormation.jsx
import React, { useState, useEffect } from 'react';
import './DescriptionFormation.css';

const DescriptionFormation = ({
  titreEvenement = "Formation React Avancé",
  descriptionEvenement = "Cette formation vous permettra de maîtriser les concepts avancés de React...",
  heureEvenement = "14:00 - 17:00",
  jourSemaine = 1,
  images = [],
  formationId,
  onRetour = () => {
    window.history.back();
  },
  onReservationSuccess = null, //  NOUVEAU: Callback pour succès
  onReservationError = null //  NOUVEAU: Callback pour erreur
```

```

}) => {
  const [imageIndex, setImageIndex] = useState(0);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    nom: '',
    prenom: '',
    email: '',
    telephone: '',
    dateSelectionnee: ''
  });
  const [errors, setErrors] = useState({});
  const [instancesDisponibles, setInstancesDisponibles] =
  useState([]);
  const [loading, setLoading] = useState(false);

  const jours = ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi',
'Samedi', 'Dimanche'];

  // ✅ NOUVEAU: Récupérer les instances disponibles depuis l'API
  useEffect(() => {
    const fetchInstancesDisponibles = async () => {
      if (!formationId) return;

      try {
        setLoading(true);
        const response = await
fetch('http://localhost:3000/api/formations-disponibles?
formation=${formationId}');

        if (!response.ok) {
          throw new Error(`Erreur HTTP: ${response.status}`);
        }

        const data = await response.json();
        console.log('📅 Instances disponibles:', data);
        setInstancesDisponibles(data);
      } catch (error) {
        console.error('❌ Erreur chargement des instances:', error);
        // En cas d'erreur, on génère des dates par défaut
        setInstancesDisponibles(genererDatesParDefaut());
      }
    };
  }, [formationId]);

```



```
    } finally {
      setLoading(false);
    }
  };

  fetchInstancesDisponibles();
}, [formationId]);

// ✅ NOUVEAU: Générer des dates par défaut si l'API échoue
const genererDatesParDefaut = () => {
  const dates = [];
  const aujourdHui = new Date();
  const dansUnMois = new Date();
  dansUnMois.setMonth(aujourdHui.getMonth() + 1);

  for (let i = 0; i < 56; i++) {
    const date = new Date(aujourdHui);
    date.setDate(aujourdHui.getDate() + i);

    if (date.getDay() === jourSemaine && date <= dansUnMois) {
      dates.push({
        id_instance: `instance-${i}`,
        date: date.toISOString().split('T')[0],
        places_restantes: 30, // Par défaut
        complet: false
      });
      if (dates.length >= 8) break;
    }
  }

  return dates;
};

// ✅ CORRIGÉ: Formater la date pour l'affichage
const formatDateAffichage = (dateString) => {
  try {
    // Gérer les dates ISO avec timezone
    const date = new Date(dateString);
    return date.toLocaleDateString('fr-FR', {
      weekday: 'long',
```

```
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
} catch (error) {
  console.error('Erreur formatage date:', dateString, error);
  return 'Date invalide';
}
};
```

```
//  CORRIGÉ: Formater la date pour la valeur du select
const formatDatePourSelect = (dateString) => {
  try {
    const date = new Date(dateString);
    return date.toISOString().split('T')[0]; // Format YYYY-MM-DD
  } catch (error) {
    console.error('Erreur formatage date select:', dateString, error);
    return dateString;
  }
};
```

```
const nextImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev + 1) % images.length);
  }
};
```

```
const prevImage = () => {
  if (images.length > 0) {
    setImageIndex((prev) => (prev - 1 + images.length) %
images.length);
  }
};
```

```
const validateName = (name) => {
  return /^[A-Za-zÀ-ÿ\ -]+$/.test(name);
};
```

```
const validateEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
};
```

```

};

const validatePhone = (phone) => {
  if (!phone) return true;
  return /^[+]?([0-9]{1,4})?[-\s\.]?[0-9]{1,4}[-\s\.]?[0-9]{1,9}$/.test(phone);
};

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));

  if (errors[name]) {
    setErrors(prev => ({
      ...prev,
      [name]: ''
    }));
  }
};

// DescriptionFormation.jsx - MODIFICATIONS DANS LA FONCTION
handleSubmit

// ✅ MODIFIÉ: Soumission avec redirection
const handleSubmit = async (e) => {
  e.preventDefault();
  const newErrors = {};

  // Validation
  if (!formData.nom.trim()) {
    newErrors.nom = 'Le nom est requis';
  } else if (!validateName(formData.nom)) {
    newErrors.nom = 'Le nom ne doit contenir que des lettres et des
traits d\'union';
  }

  if (!formData.prenom.trim()) {

```

```
newErrors.prenom = 'Le prénom est requis';
} else if (!validateName(formData.prenom)) {
  newErrors.prenom = 'Le prénom ne doit contenir que des lettres
et des traits d\'union';
}

if (!formData.email.trim()) {
  newErrors.email = 'L\'email est requis';
} else if (!validateEmail(formData.email)) {
  newErrors.email = 'Format d\'email invalide';
}

if (!formData.dateSelectionnee) {
  newErrors.dateSelectionnee = 'Veuillez sélectionner une date';
}

if (formData.telephone && !validatePhone(formData.telephone)) {
  newErrors.telephone = 'Format de téléphone invalide';
}

if (Object.keys(newErrors).length > 0) {
  setErrors(newErrors);
  return;
}

try {
  // Trouver l'instance sélectionnée
  const instanceSelectionnee = instancesDisponibles.find(
    instance => formatDatePourSelect(instance.date) ===
formData.dateSelectionnee
  );

  if (!instanceSelectionnee) {
    throw new Error('Instance non trouvée');
  }

  // Envoyer la réservation à l'API
  const response = await
fetch('http://localhost:3000/api/reservations', {
  method: 'POST',
```

```
headers: {
  'Content-Type': 'application/json',
},
body: JSON.stringify({
  prenom: formData.prenom,
  nom: formData.nom,
  courriel: formData.email,
  telephone: formData.telephone,
  id_formation_instance: instanceSelectionnee.id_instance
})
});

if (!response.ok) {
  const errorData = await response.json();
  throw new Error(errorData.error || 'Erreur lors de la réservation');
}

const result = await response.json();

// ✅ SUCCÈS: Rediriger vers la page de confirmation
if (onReservationSuccess) {
  const dateFormatee =
formatDateAffichage(formData.dateSelectionnee + 'T00:00:00');
  onReservationSuccess({
    id_reservation: result.id_reservation,
    date: dateFormatee,
    formation: titreEvenement,
    prenom: formData.prenom,
    nom: formData.nom,
    email: formData.email
  });
}

} catch (error) {
  console.error('Erreur réservation:', error);

  // ✅ ERREUR: Rediriger vers la page d'erreur
  if (onReservationError) {
    onReservationError({
      message: error.message,
```

```

        formation: titreEvenement,
        date: formData.dateSelectionnee
    });
}
}
};

```

```

const handleBack = () => {
    if (onRetour) {
        onRetour();
    }
};

```

```

const closeForm = () => {
    setShowForm(false);
    setErrors({});
    setFormData({
        nom: '',
        prenom: '',
        email: '',
        telephone: '',
        dateSelectionnee: ''
    });
};

```

```

return (
    <div className="formation-container">
        <div className="formation-content">
            <button className="back-button" onClick={handleBack}>
                ← Retour aux formations
            </button>

            <div className="formation-grid">
                { /* Section Galerie d'images (inchangée) */ }
                <div className="gallery-section">
                    <div className="gallery-container">
                        {images.length > 0 ? (
                            <>
                                <div className="image-main-container">
                                    <img

```

```

src={images[imageIndex]}
alt={` ${titreEvenement} - Image ${imageIndex + 1} `}
className="gallery-image"
/>

{images.length > 1 && (
  <>
    <button className="nav-button prev" onClick=
{prevImage}><</button>
    <button className="nav-button next" onClick=
{nextImage}>></button>
    <div className="image-counter">{imageIndex + 1} /
{images.length}</div>
  </>
)}
</div>

{images.length > 1 && (
  <div className="thumbnail-container">
    {images.map((img, index) => (
      <img
        key={index}
        src={img}
        alt={` Miniature ${index + 1} `}
        className={`thumbnail ${index === imageIndex ?
'active' : ''} `}
        onClick={() => setImageIndex(index)}
      />
    ))}
  </div>
)}
</>
): (
  <div className="no-images">
    <p>Aucune image disponible</p>
  </div>
)}
</div>
</div>

```

```

{ /* Section Informations */}
<div className="info-section">
  <div className="formation-header">
    <h1 className="formation-title">{titreEvenement}</h1>
    <div className="formation-meta">
      <div className="meta-item">
        <span className="meta-icon"> 📅 </span>
        <span>Tous les {jours[jourSemaine]}</span>
      </div>
      <div className="meta-item">
        <span className="meta-icon"> ⌚ </span>
        <span>{heureEvenement}</span>
      </div>
    </div>
  </div>

  <div className="description-section">
    <h2>Description</h2>
    <p className="description-text">{descriptionEvenement}
  </p>
</div>

  <button
    className="inscription-button"
    onClick={() => setShowForm(true)}
    disabled={loading}
  >
    {loading ? 'Chargement...' : 'S\'inscrire à la formation'}
  </button>

  { /* ✅ AJOUT: Affichage du nombre d'instances disponibles
*/}

  {instancesDisponibles.length > 0 && (
    <div className="disponibilite-info">
      <p>
        <strong>{instancesDisponibles.filter(i =>
!i.complet).length}</strong>
        session(s) disponible(s) dans le prochain mois
      </p>
    </div>
  )}

```



```

    })
  </div>
</div>

```

```

{ /* Formulaire d'inscription */ }

```

```

{showForm && (

```

```

  <div className="form-fullscreen-overlay">

```

```

    <div className="form-container">

```

```

      <div className="form-header">

```

```

        <h3>Inscription à la formation</h3>

```

```

        {loading && <p>Chargement des dates disponibles...</p>}

```

```

      </div>

```

```

    <form className="inscription-form" onSubmit=

```

```

    {handleSubmit}>

```

```

      <div className="form-group">

```

```

        <label htmlFor="prenom">Prénom *</label>

```

```

        <input

```

```

          type="text"

```

```

          id="prenom"

```

```

          name="prenom"

```

```

          value={formData.prenom}

```

```

          onChange={handleInputChange}

```

```

          className={errors.prenom ? 'error' : ''}

```

```

          placeholder="Entrez votre prénom"

```

```

        />

```

```

        {errors.prenom && <span className="error-message">

```

```

        {errors.prenom}</span>}

```

```

      </div>

```

```

    <div className="form-group">

```

```

      <label htmlFor="nom">Nom *</label>

```

```

      <input

```

```

        type="text"

```

```

        id="nom"

```

```

        name="nom"

```

```

        value={formData.nom}

```

```

        onChange={handleInputChange}

```

```

        className={errors.nom ? 'error' : ''}

```

```

        placeholder="Entrez votre nom"

```

```

    />
    {errors.nom && <span className="error-message">
{errors.nom}</span>}
    </div>

    <div className="form-group">
    <label htmlFor="email">Email *</label>
    <input
    type="email"
    id="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    className={errors.email ? 'error' : ''}
    placeholder="entrez@votre.email"
    />
    {errors.email && <span className="error-message">
{errors.email}</span>}
    </div>

    /*  MODIFIÉ: Sélecteur de dates avec places restantes
*/}

    <div className="form-group">
    <label htmlFor="dateSelectionnee">Date de formation *
</label>
    <select
    id="dateSelectionnee"
    name="dateSelectionnee"
    value={formData.dateSelectionnee}
    onChange={handleInputChange}
    className={errors.dateSelectionnee ? 'error' : ''}
    disabled={loading || instancesDisponibles.length === 0}
    >
    <option value="">
    {loading ? 'Chargement...' : 'Sélectionnez une date'}
    </option>
    {instancesDisponibles
    .filter(instance => !instance.complet)
    .map((instance, index) => (
    <option

```

```

        key={index}
        value={formatDatePourSelect(instance.date)} // ✅ Utiliser
formatDatePourSelect
    >
        {formatDateAffichage(instance.date)}
        {instance.places_restantes !== undefined &&
        ` (${instance.places_restantes} place(s) restante(s))`}
    </option>
))
}
</select>

{/* ✅ AJOUT: Affichage de la date sélectionnée */}
{formData.dateSelectionnee && (
    <div className="selected-date-display">
        📅 Date sélectionnée : <strong>
{formatDateAffichage(formData.dateSelectionnee + 'T00:00:00')}
        </strong>
    </div>
)}

{/* Affichage si aucune date disponible */}
{!loading && instancesDisponibles.filter(i => !i.complet).length ===
0 && (
    <div className="warning-message">
        Aucune session disponible dans le prochain mois
    </div>
)}

{errors.dateSelectionnee && <span className="error-message">
{errors.dateSelectionnee}</span>}
    </div>

    <div className="form-group">
        <label htmlFor="telephone">Téléphone (optionnel)
    </label>
        <input
            type="tel"
            id="telephone"
            name="telephone"

```

```

        value={formData.telephone}
        onChange={handleInputChange}
        className={errors.telephone ? 'error' : ''}
        placeholder="+33 1 23 45 67 89"
      />
      {errors.telephone && <span className="error-
message">{errors.telephone}</span>}
    </div>

    <div className="form-buttons">
      <button
        type="submit"
        className="submit-button"
        disabled={loading || instancesDisponibles.filter(i =>
!i.complet).length === 0}
      >
        {loading ? 'Traitement...' : 'Confirmer l\'inscription'}
      </button>
      <button type="button" className="cancel-button"
onClick={closeForm}>
        Annuler
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
</div>
);
};

```

```
export default DescriptionFormation;
```

la page de confirmation ressemble à ça :

```

function MaPage() {
  const handleClick = () => {
    alert("OK cliqué !");
  };
}

```

```
return (  
  <div className="ma-page">  
    {/* Ton HTML converti en JSX ici */}  
    <h1>Confirmation</h1>  
    <button style={{ border: '2px solid green' }} onClick=  
{handleClick}>  
      OK  
    </button>  
  </div>  
);  
}
```

```
export default MaPage;
```

et il y a un fichier qui permet de envoyer des courriels. C'est le fichier Backend/emailer.js :

```
import nodemailer from "nodemailer";  
import dotenv from "dotenv";
```

```
// Charger le .env dans process.env  
dotenv.config({ path: '../.env' })
```

```
console.log("EMAIL_USER:", process.env.EMAIL_USER);  
console.log("EMAIL_PASS:", process.env.EMAIL_PASS);
```

```
async function sendEmail({ from, to, subject, text, html }) {  
  try {  
    const transporter = nodemailer.createTransport({  
      service: "gmail",  
      auth: {  
        user: process.env.EMAIL_USER,  
        pass: process.env.EMAIL_PASS,  
      },  
    });  
  }  
});
```

```

    await transporter.verify();
    console.log("✅ SMTP connecté avec succès !");

    const info = await transporter.sendMail({ from, to, subject, text,
    html });
    console.log("✅ Email envoyé :", info.messageId);
  } catch (error) {
    console.error("❌ Erreur lors de l'envoi :", error);
  }
}

// ✅ Exporter la fonction pour qu'elle soit utilisable ailleurs
export { sendEmail };

```

Parfait ! Je vais intégrer l'envoi d'email de confirmation et modifier la page de confirmation. Voici les modifications :

1. Modifier Backend/routes/formations.js - Ajouter l'envoi d'email

javascript

```

// Dans Backend/routes/formations.js - modifier la route POST /reservations
import { sendEmail } from '../emailer.js'; // ✅ AJOUT: Import de l'emailer

// Route pour créer une réservation
router.post('/reservations', async (req, res) => {
  try {
    const { prenom, nom, courriel, telephone, id_formation_instance } = req.body;

    // Validation des données requises
    if (!prenom || !nom || !courriel || !id_formation_instance) {
      return res.status(400).json({ error: 'Tous les champs obligatoires doivent être remplis' });
    }

    // Vérifier que l'instance existe et a des places disponibles
    const checkInstanceQuery = `
      SELECT

```

```

        fi.id_formation_instance,
        fi.jour,
        f.nom as formation_nom,
        f.heure_debut,
        f.heure_fin
    FROM formation_instance fi
    JOIN formation f ON fi.id_reference_formation = f.id_formation
    LEFT JOIN (
        SELECT id_formation_instance, COUNT(*) as reservation_count
        FROM table_reservation
        GROUP BY id_formation_instance
    ) tr ON fi.id_formation_instance = tr.id_formation_instance
    WHERE fi.id_formation_instance = $1
        AND (COALESCE(tr.reservation_count, 0) < 30)
`;

const instanceResult = await pool.query(checkInstanceQuery, [id_formation_instance]);

if (instanceResult.rows.length === 0) {
    return res.status(400).json({ error: 'Cette session est complète ou n\'existe pas' });
}

const instance = instanceResult.rows[0];

// Commencer une transaction
await pool.query('BEGIN');

// 1. Créer la réservation
const insertReservationQuery = `
    INSERT INTO reservation (prenom, nom, courriel, telephone)
    VALUES ($1, $2, $3, $4)
    RETURNING id_reservation
`;

const reservationResult = await pool.query(insertReservationQuery, [
    prenom, nom, courriel, telephone || null
]);

const idReservation = reservationResult.rows[0].id_reservation;

// 2. Lier la réservation à l'instance
const insertTableReservationQuery = `

```

```
INSERT INTO table_reservation (id_reservation, id_formation_instance)
VALUES ($1, $2)
`;

await pool.query(insertTableReservationQuery, [idReservation, id_formation_instance]);

// Valider la transaction
await pool.query('COMMIT');

// ✅ AJOUT: Envoyer l'email de confirmation
try {
  const dateFormatee = new Date(instance.jour).toLocaleDateString('fr-FR', {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });

  const emailHtml = `
    <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
      <h2 style="color: #2c3e50; text-align: center;">Confirmation de votre inscription</h2>

      <div style="background: #f8f9fa; padding: 20px; border-radius: 10px; margin: 20px 0;">
        <h3 style="color: #2c3e50; margin-top: 0;">Détails de votre réservation</h3>

        <p><strong>Formation :</strong> ${instance.formation_nom}</p>
```


